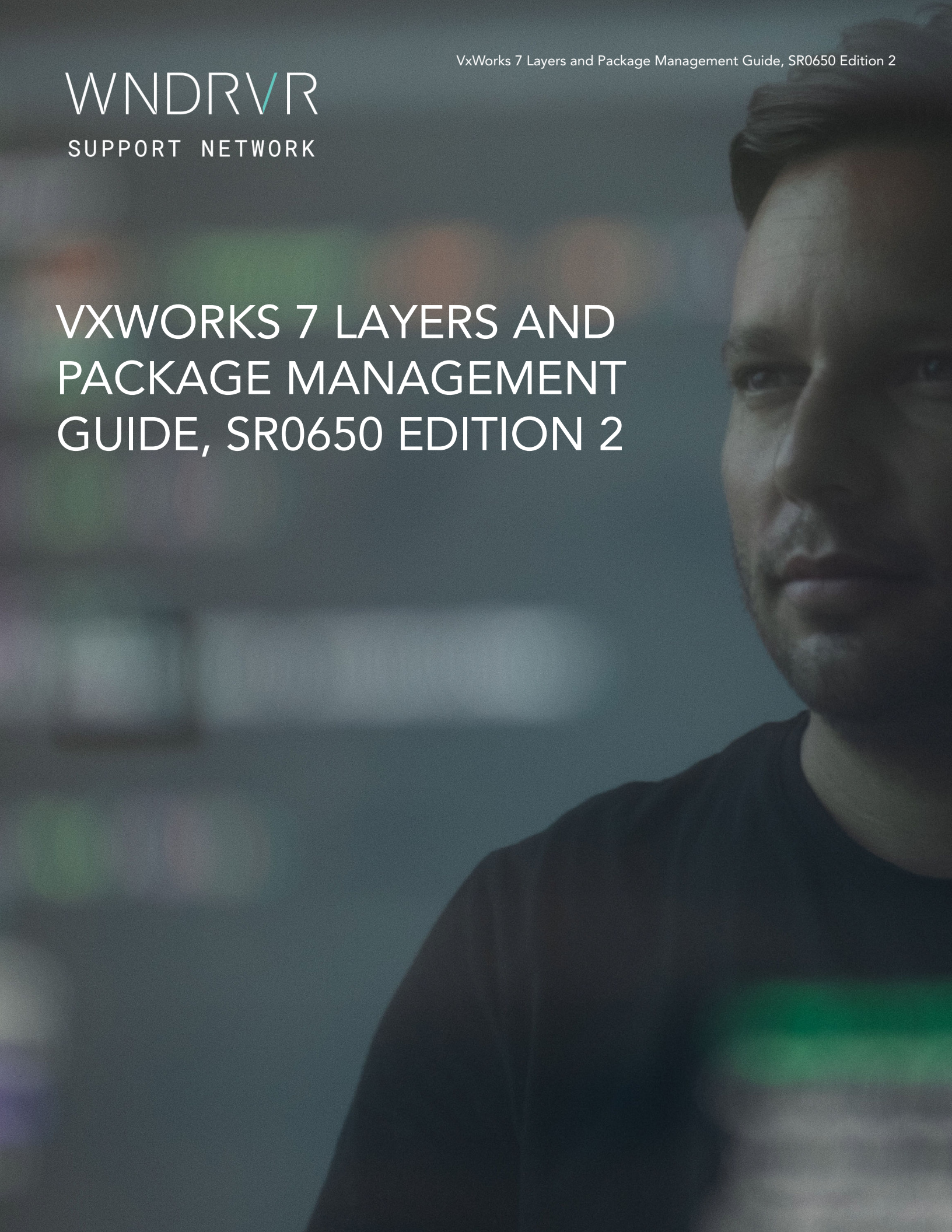
WNDRVR
SUPPORT NETWORK

# VXWORKS 7 LAYERS AND PACKAGE MANAGEMENT GUIDE, SR0650 EDITION 2



**Copyright Notice**

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

**Corporate Headquarters**

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

*VxWorks 7 Layers and Package Management Guide, SR0650 Edition 2*

4 August 2020

# 1. INTRODUCING LAYERS AND PACKAGE MANAGEMENT

VxWorks® is built into discrete units called layers, to enable make packaging and testing of features and functionality. Individual features, like USB, file system, and Board support packages (BSPs), are built, tested and released independently using layers.

## Purpose

VxWorks layers are containers for features including kernel code or additional user space libraries and processes that can be incorporated into the VxWorks runtime. They typically consist of source code with configuration and build support. Wind River constructs the VxWorks runtime from layers, and so this mechanism exists within your Wind River installation.

VxWorks source build (VSB) is a Workbench project that defines the source and binary libraries that your VxWorks runtime will support. The Workbench Source Build Configurator gives a graphical representation of your runtime library configuration, and each library represented there has a layer behind its implementation. Layers are used because they offer a flexible mechanism for adding libraries to a VSB.

Layers are contained within RPM packages. RPM packages typically contain one or more related layers. These layers are installed into *installDir* **/vxworks-7/pkgs_v2** directory tree.

You do not need to understand layers in order to benefit from them because Wind River uses them already. Yet the layers infrastructure also presents many powerful opportunities to customers who decide to also make use of it themselves.

- The layers framework permits multiple versions of a feature set to be installed within a single Wind River installation.
- Layers let you decide which version of any feature set should go into the VxWorks kernel libraries.
- Each layer version may contain incremental bug fixes and feature enhancements.
- A layer may modify a pre-existing layer directly without changing the contents of the pre-existing layer.
- A layer may safely encapsulate an open source application without risk of GPL contamination of either the VxWorks kernel or surrounding BSP, driver or application layers.
- A layer may have sub-layers.
- A layer may be shipped as source only, binary only, or both, thus protecting the software ip contained within the layer.

## Audience

You should be familiar with the VxWorks layer infrastructure if you are a:

- Wind River software partner that develops middleware that runs on VxWorks.
- VxWorks developer that is responsible for generating the VxWorks kernel libraries or BSPs.
- VxWorks developer that needs to ship binary applications to customers.

# 2. LAYER OVERVIEW

VxWorks layers provide the infrastructure for the code that produces individual VxWorks features. Individual features, such as USB, file system, and board support packages, may be built, tested, and released independently from each other using layers.

## Purpose

VxWorks layers are containers for features including kernel code or additional user space libraries and processes that can be incorporated into the VxWorks runtime. They offer a flexible mechanism for adding components to a VxWorks source build (VSB). They typically consist of source code with configuration and build support. A layer may have sub-layers and every layer or sub-layer has a version.

A layer is presented as a unique selectable feature when configuring a VSB project, and by using explicit configuration files, you can customize a layer with multiple selectable features.

Binary layers are also supported to allow delivery of pre-built objects in the case where licensing restrictions prevent the sharing of source code.

RPM packages will typically install one or more related layers into *installDir* **/vxworks-7/pkgs_v2** directory tree.

VxWorks BSPs are packaged as layers

# 3. VXWORKS LAYER SYSTEM

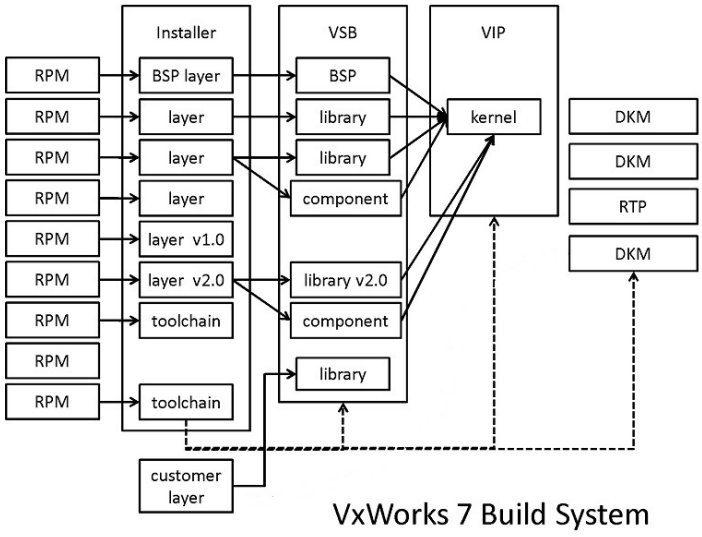
The VxWorks layer system receives source and binary components from RPM files that may be included in a VSB and therefore define the available features within a VxWorks VIP project.

Source and binary components are presented to the VxWorks build system as RPM files containing layers. A layer encapsulates a piece of functionality that may be built into the VxWorks kernel.

Prior to VxWorks product installation, the functionality within each layer is packaged as an RPM supplied either on installation media, or retrieved from the Wind River servers by the installer. At fresh installation time, the installer takes the RPM files, unpacks them and assembles the initial set of VxWorks layers into the Wind River installation directory structure. The installer may also be used to incrementally add new layers into the Wind River installation after a fresh product installation has completed.

The VxWorks layer system can support multiple versions of the same packages and can group individual packages together based on dependencies.

Binary layers are also supported to allow delivery of pre-built objects in the case where it is required by licensing restrictions.



# 4. DESCRIBING THE VXWORKS 7 LAYERS INSTALLATION

The Wind River installer unpacks the Wind River supplied layers inside the installation directory and arranged in functional groups.

All Wind River supplied layers are grouped by function at *installBase* **/vxworks-7/pkgs_v2**. For example, *installBase* **/vxworks-7/pkgs_v2/os** contains Wind River supplied layers that deal with the core of the VxWorks operating system. Within this directory you will find layers like:

- The core VxWorks layer, found at *installBase* **/vxworks-7/pkgs_v2/os/core-2.0.0.0**. This layer holds a number of elements including the VxWorks kernel source code, header files and CDF files and conflglettes. Source files are grouped into individual directories within the **kernel/src** directory, cdf files for this layer are in **kernel/cdf** and configlettes are in **kernel/configlette**.
- The core VxWorks I/O subsystem, found at *installBase* **/vxworks-7/pkgs_v2/os/core-2.0.0.0**. This layer contains I/O header files in the **io/h** directory, source files in the **io/src** directory and cdf files in the **io/cdf** directory.
- The Wind River supplied VxWorks Board Support Packages (BSP) are found at *installBase* **/vxworks-7/pkgs_v2/os/board** and are grouped by CPU archiecture.

An overview of the layers found in *installBase* **/vxworks-7/pkgs_v2** is described in the following table. The actual content of your **pkgs_v2** directory will depend on which products your have licensed from Wind River.

Table 1. Wind River Supplied Layers

| | |
|---|---|
| app | Contains:<br>**XML kernel and user space support library.**<br>**Web Command Line Interface (WEBCLI) for VxWorks web server.**<br>**SNMP agent and engine.**<br>**SOAP web services.**<br>**Open Source Computer Vision (OpenCV) software library.** |
| boot | Contains bootloaders for select IA32, IA64, Power-PC and ARM targets. |
| connectivity | Contains:<br>**CAN Bus protocol stack.**<br>**USB Host and Target stack.** |
| multimedia | Contains different layers that provide the multimedia libraries:<br>**Audio stack with drivers.**<br>**Graphics drivers - fbdev (frame buffer) and GPU specific (gpudev).**<br>**evdev event loop.**<br>**Tilcon UI builder stack.** |
| net | Contains the network stack and associated network drivers and protocol middleware. |
| os | Contains the core VxWorks operating system and services. |
| debug_analysis | Contains: |

| | |
|---|---|
| | **System Viewer agent.**<br>**Stop Mode Debug agent.**<br>**Target Communication Framework (TCF) agent.** |
| security | Contains:<br>**Open SSL components.**<br>**Crypto and Hash components.**<br>**User Management components.** |
| storage | Contains:<br>**VxWorks file systems (cdromfs, dosfs, hrfs, nfs, romfs vrfs)**<br>**Core support libraries.** |

Custom layers may be added to *installBase* **/vxworks-7/pkgs_v2** as required. It is recommended that your custom layers are contained within a directory named *installBase* **/vxworks-7/pkgs_v2/custom** that you create. These custom layers are referred to as *inside layers*, because they are positioned inside the installation directory structure.

Custom layers may also be contained within user defined directories that are external to the Wind River installation itself. These user defined locations must be communicated to Workbench thru your host workstation's *WIND_LAYER_PATHS* environment variable. These custom layers are referred to as *outside layers* because they are located outside of the installation directory.

# 5. DESCRIBING THE VXWORKS 7 LAYER

A layer is a container for functionality that can be added to a VxWorks VSB project.

## Introduction

On the workstation filesystem, a layer is a directory containing different files and sub-directories. The purpose of a layer is to implement functionality like software drivers, stacks or parts of the VxWorks kernel itself which may be individually included into a VSB project. It is common therefore for a layer to contain sub-directories holding header files and source files.

Kernel functionality may be implemented in a modular way using layers. Layers are often implemented recursively. Layer sub-directories may contain child layers that implement different related pieces of parent layer functionality. Layers and child layers have the same structure; the only difference between them is that they are related. One layer is the parent of the other.

## Top of a Layer

The top directory of all layers will always contain a **Makefile** and a related **layer.vsbl** file which passes layer metadata to the VSB build system and defines how the parts of a layer relate. When looking to identify where on the host filesystem a layer begins, it is helpful to look for these two files, which act as a marker to the top of all layers. In a recursive layer, this marker will appear at the boundary between every parent layer and its children.

The layer makefile in the layer's top directory always includes the rule file *$(WIND_KRNL_MK)* **/rules.layers.mk**. For more details on the available makefile rules, see Layer Makefile Rules Reference on page 96.

For a description of all the files commonly found within a layer directory, see Common Layer Contents on page 10.

## Layer Naming

Only the following basic character set can be used when naming a layer:

- nondigit (upper and lower case letters)
- digit (zero to nine)
- underscore

All layers have a name and a current version number. The layer directory name is always the layer name followed by the version number. When a layer is included into a VSB project, the build system identifies it by a string composed of three elements:

- The name and version number of any parent layer.
- The name of this layer.
- The version number of this layer.

## Layer Location

Wind River supplied layers are located within the Wind River installation at *installDir* **/vxworks-7/pkgs_v2**. Third party or customized layers may either be located in the same place, or they may be located elsewhere on the host workstation file system as defined by an environment variable. For further details, refer to Using Layers at a Custom Location on page 50.

## Parallel Layer Versions

After you use the Wind River Product Maintenance Tool to update the Wind River installation to a newer release, it is normal for this update process to download new versions of existing layers into your Wind River installation directory and therefore leave behind multiple layer versions. Wind River updates system layers regularly to add new features, and the vehicle for receiving these updates is

the periodic update release downloaded by the Product Maintenance Tool. The tool's job is to update all layers that have changed, and leave the original layer versions intact.

VxWorks layers and the VSB project they are inserted into have been designed to be both modular and flexible. Multiple versions of a layer are allowed to exist in the Wind River installation directory at the same time. You can observe this by exploring *installDir* **/ vxworks-7/pkgs_v2** following a Wind River product update. You may find new directories there with the same name, but different version numbers. These are layers that have been updated during the Wind River product update process. The newer layer versions co-exist with the original layer version.

For example, following a Wind River update, suppose the vxsim architecture support library layer was updated from **simulator-2.0.0.0** to **simulator-2.0.7.5**. It would therefore be normal to find the following directories in an updated installation directory *installDir* **/vxworks-7/pkgs_v2/os/arch**:

```
$tree -L 1
.
├── 020.conf
├── arm-2.0.0.0
├── ia-2.0.0.0
├── ppc-2.0.0.0
├── simulator-2.0.0.0
├── simulator-2.0.7.5
```

When multiple versions of a layer are resident on the host filesystem, the build system will determine which version of a layer to include into a VSB project based on any dependencies that exist in the other system layers (e.g. the BSP layer).

The user is permitted to override the VSB build system's choice of layer version. However, if any dependency is left unresolved within the VSB project, the project will not build successfully. For this reason, it is recommended not to override the VSB system default. Also, do not manually delete old versions of layers from your installation diectory as you may need them again in the future.

## Private, Protected and Public Interfaces

Private interfaces are functions and data that are internal to a layer and are not part of the layer's external user interface. Definitions of private interfaces must be placed in a header file that remains in the layer's installation directory. Macros available in a layer's Makefile for exporting header files must not specify any of the private header files nor the directories containing the private header files. Private header files cannot be accessed by source files outside of the layer because they do not get copied into the VSB project directory.

Protected interfaces are functions and data that are exported to a select set of layers known as friends. Definitions of protected interfaces must be placed in a header file that is exported by one of the following macros available to a layer's Makefile:

- **KERNEL_PROTECTED_H_DIRS**
- **SHARED_PROTECTED_H_DIRS**
- **USER_PROTECTED_H_DIRS**
- **KERNEL_PROTECTED_H_FILES**
- **SHARED_PROTECTED_H_FILES**
- **USER_PROTECTED_H_FILES**

In addition, the exporting layer must use the **FRIEND** keyword in the **layer.vsbl** file to specify the set of friend layers. Protected header files specified in any of the above macros will be copied into the VSB project directory, and can be accessed only by the set of friend layers.

Public interfaces are functions and data that are available to any other layer and to applications. Definitions of public interfaces must be placed in a header file that is exported by one of the following macros available to a layer's Makefile:

- **KERNEL_PUBLIC_H_DIRS**
- **SHARED_PUBLIC_H_DIRS**

- **USER_PUBLIC_H_DIRS**
- **KERNEL_PUBLIC_H_FILES**
- **SHARED_PUBLIC_H_FILES**
- **USER_PUBLIC_H_FILES**

Public header files specified in any of the above macros will be copied into the VSB project directory, and can be accessed by all other source files.

## Example Layer and Makefile

The following example illustrates the structure of the utils layer that contains all the VxWorks utilities. The layer is located in the Wind River installation at *installDir* **/vxworks-7/pkgs_v2/os/utils-2.0.0.0** directory. This layer is identified as **UTILS**.

```
$tree -L 1
.
├── 060.conf
├── archive-3.3.2.10
├── boardlib
├── bootapp
├── common.vxconfig
├── jansson-2.7.0.0
├── layer.vsbl
├── loader
├── Makefile
├── ostools
├── rbuff
├── shell
├── tbb-20.18.1.10
├── unix
├── utf
├── vi_editor-3.7a.0.0
├── zlib-1.2.8.20
```

The **UTILS** layer directory does not directly contain source file or header sub-directories. Rather, it has directories containing child layers. The **Makefile** includes two important layer files:

```
# Makefile - UTILS layer Makefile

include $(WIND_KRNL_MK)/defs.layers.mk

...

include $(WIND_KRNL_MK)/rules.layers.mk
```

The **WIND_KRNL_MK** environment variable currently expands to the *installDir* **/vxworks-7/build/mk/krnl/** directory. All the layer kernel space makefile rule files are contained in this directory.

## Example Child Layer and Makefile

The following example illustrates the recursive nature of the VxWorks layer.

The zlib layer is contained within in the parent utils layer. It is located in the Wind River installation at *installDir* **/vxworks-7/pkgs_v2/os/utils-2.0.0.0/zlib-1.2.8.20** directory. This layer is identified as **UTILS_ZLIB**.

```
$tree -L 1
.
├── cdf
├── cfg
├── common.vxconfig
├── h
├── layer.vsbl
├── Makefile
├── src
```

The zlib sublayer **Makefile** is as follows. Notice that it has the same form as the parent utils layer makefile. The only difference is that this layer directly contains source, header, configlette and cdf directories. Therefore, this layer makefile specifies make macros while the parent makefile does not:

```
# Makefile - Wind River ZLIB Layer Makefile

include $(WIND_KRNL_MK)/defs.layers.mk

SHARED_PUBLIC_H_DIRS = shareh

BUILD_DIRS = src

ifdef _WRS_CONFIG_RTP
BUILD_USER_DIRS = src
endif

POST_NOBUILD_CDFDIRS = cdf

include $(WIND_KRNL_MK)/rules.layers.mk
```

# 6. COMMON LAYER CONTENTS

To qualify as a layer, a directory must contain some particular files and sub-directories.

A layer is a directory positioned within the directory structure at *installDir* **/vxworks-7/pkgs_v2**, or somewhere else defined by an environment variable. This directory contains certain important elements that make it a layer.

The files that are found in a layer directory are described in the following table.

Table 1. Layer Root Directory File Elements

| File | Required / Optional | Description |
|---|---|---|
| **common.vxconfig** | Optional | Specifies VSB kernel-side configurations and dependencies in Kconfig syntax. Use to add additional VSB configurable items to layer. |
| **common.vxusrconfig** | Optional | Specifies VSB user-side configurations and dependencies in Kconfig syntax. Use to add additional VSB configurable items to layer. |
| **Makefile** | Required | **This is the layer makefile.**<br>**Must be named Makefile.**<br>**Always appears in the same directory as layer.vsbl.**<br>**Specifies the relationship between the layer and its sub-directories. These sub-directories contain headers/src/cdf/configlettes etc.**<br>**The makefile controls how the layer is built as part of a VSB project.**<br>**For more details, refer to** Layer Makefile on page 17**.** |
| **layer.vsbl** | Required | **Always appears in the same directory as the layer Makefile.**<br>**Contains layer metadata.** |

Various directories, some optional, may be found within a layer. These directories may contain child layers, source code, header files, cdf files or configlettes.

Layers are built as part of a VxWorks Source Build (VSB) project build. A VSB build occurs in three stages:

- Stage 1 - PREBUILD
- Stage 2 - BUILD
- Stage 3 - POSTBUILD

Layers are built at one or more of these build stages. The directories that exist within a root layer directory will be actioned during these build stages; each directory relates to a particular build stage.

The directories that are found within a layer root directory are described in the following table.

Table 2. Layer Root and Container Directory Elements

| Directory | Description | Build Stage |
|---|---|---|
| **h** | Contains kernel side only public header files. | PREBUILD |
| **shareh** | Contains both kernel and user side public header files. | PREBUILD |
| **userh** | Contains user side only public header files. | PREBUILD |
| **src** | The source directory for the layer. The build results in kernel libraries. Contains a source makefile to control the build. | BUILD |
| **user_src** | The source directory for the layer. The build results in kernel libraries. Contains a source makefile to control the build. | BUILD |
| **cdf** | Contains the CDF file configuration information. | POSTBUILD |
| **configlettes** | Contains the configlettes for the features implemented by this layer. | POSTBUILD |
| **dkm** | Contains dkm files that should be built as a layer. Contains a source makefile to control the build. Manual editing of this makefile is common. | POSTBUILD |
| **rtp** | Contains user side rtp project files that should be built as a layer. Contains a source makefile to control the build. Manual editing of this makefile is common. | POSTBUILD |
| <layer_01>, <layer_02>, <layer_xx> | Child layers named by the user. If one or more of these are present, this becomes a parent layer. | |

For more information on how these directories are used during the VSB project build, please refer to Layer Makefile on page 17 and Layer Makefile Macros Reference on page 83.

## Example Layer

The following example illustrates the structure of the SNMP parent layer that is located in the Wind River installation at *installDir* **/ vxworks-7/pkgs_v2/app/snmp-2.0.0.0** directory. The SNMP layer includes **agent**, **engine**,**std-mibs**,**tools** and **wrsnmp** elements. These are not child layers, but elements of the SNMP layer.

```
$tree -L 1
.
├── agent
├── engine
├── std-mibs
├── tools
├── wrsnmp
├── layer.vsbl
└── Makefile
```

# 7. LAYER METADATA FILE

A layer is defined and also configured by the metadata in the **layer.vsbl** file in its root directory.

The **layer.vsbl** metadata file contains the name and version of the layer. Other optional fields provide information on hierarchy and dependencies. Layer metadata dependencies configure the VSB project hierarchy and override any install dependencies in RPMs, component dependencies used to configure VIPs, or object link time dependencies on libraries. If a layer is properly defined, the dependencies at each level of the install, configuration and build system will result in an optimally configured system.

The configuration system of a VSB project uses information in **layer.vsbl** and the location of the layer in the *installDir* **/vxworks-7/pkgs_v2** directory tree to determine how the layer will be displayed in the Workbench Source Build Configuration window.

You include a layer in a VSB by using a configuration parameter with the concatenated form:*LayerName _VERSION* . In the VSB auto-generated configuration, each layer is prefixed with the parent layer's *LayerName _VERSION* , building a unique key for each VSB configuration.

For example:

```
Layer PARENT_A {
 VERSION 1.0.0.0
 ...
}
Layer CHILD_B {
  VERSION 1.0.0.0
  PARENT PARENT_A_1_0_0_0
  ...
}
```

These would be represented in the VSB configuration as **_WRS_CONFIG_PARENT_A_1_0_0_0** for the layer **PARENT_A** and **_WRS_CONFIG_PARENT_A_1_0_0_0_CHILD_B_1_0_0_0** for layer **CHILD_B**.

The parent-child relationship between layers is determined by the directory layout. This relationship can be overridden by explicitly specifying the PARENT Layer in the layer.vsbl meta-data. Having modified the meta-data in this way, the specified PARENT takes precedence over the directory layout.This layout is used to populate the *PARENT* statement in the **layer.vsbl** file. For example, the directories **a/b/c** represent layers A, B and C. In this hierarchy, the parent of layer B is layer A, and the parent of layer C is layer B.

Note: The naming standard of a layer is to use uppercase for names with underscore ("_") as the separators.

## Layer Metadata Structure Format

The following example illustrates the metadata format in the **layer.vsbl** file.

```
Layer LayerName {
  SYNOPSIS A longer name displayed in VSB editor
  HELP A description of what the layer contains and provides, that is displayed in the VSB editor
  VERSION 1.0.0
  VSB_REQUIRES VSB_COMPONENT_1 VSB_COMPONENT_2
  LAYER_REQUIRES LAYER_NAME_1 LAYER_NAME_2
  OPTIONAL YES|NO
  DEFAULT YES|NO
  DOC_BUILD YES|NO
  FASTBUILD YES|NO
  PARENT LAYER_NAME_1_0_0_SUBLAYER_1_0_0
}
```

Since many of these fields are optional, a minimal **layer.vsbl** file might contain:.

```
Layer MY_LAYER {
  SYNOPSIS My first test layer
  HELP This layer provides a simple hello world example, and is only for demonstration
  VERSION 1.0.0.1
  OPTIONAL YES
  DEFAULT YES
  DOC_BUILD NO
  FASTBUILD NO
  }
```

The default value for **DOC_BUILD** is *YES*. **DOC_BUILD** *NO* prevents the layer from being included in the document generation process.

The default value for **FASTBUILD** is *YES*, and users creating complex layers that are not **FASTBUILD** capable must set the vaue to *NO* in the **layer.vsbl** file.

## Metadata Elements

See Layer Metadata Elements Reference on page 98 for a complete listing and description of available elements.

## Conditional Layer Dependencies

A layer may be dependent on other layers for the APIs provided by headers or objects and for symbols used during RPT linking. This dependency is expressed with the following syntax:

```
LAYER REQUIRES LAYER LAYER ...
```

For example:

```
LAYER_REQUIRES IPNET_COREIP SHELL
```

Layer requirements can be conditional. For instance, it may depend on a flag such as **SMP** or **LP64** being enabled, or another layer being included.

The syntax for basing a condition on a flag is:

```
LAYER_REQUIRES  {Test_FLAGs ,  True_Layers ,  False_Layers }
```

The logic group is wrapped with {}, and contains 3 sections.

*Test_FLAGs*

The list of FLAGs to test.

This is Required.

*True_Layers*

The list of layers to be included if the *test_FLAGs* are enabled.

This is optional if a *False_layers* list is present.

*False_Layers*

The list of layer to be included if the *test_FLAGs* are disabled

This is optional if a *True_layers* list is present.

This conditional syntax can also be stated as shown below, however the former example is preferred:

```
LAYER_REQUIRES {FLAG_A, LAYER_X, } \
                          {FLAG_A, , LAYER_Y}
```

This is interpreted as "The layer requires **LAYER_X** if **FLAG_A** is enabled and **LAYER_Y** if not".

If you want to conditionally include a layer, with no alternate, simply leave either the true or false clause empty, as in the following example:

```
LAYER_REQUIRES {SMP, LAYER_A, }
```

In the kConfig syntax used in the generated VSB configuration files this would be:

```
((FLAG_A && LAYER_X) || (!FLAG_A && LAYER_Y))
```

## Debugging Layer Logic

Because layers may relate to other layers, it is sometimes useful to debug the logic that your layer is expressing into a VSB. This can be done in two ways.

First, from command-line utility **wrtool** use prj vsb info <LAYER_X> to show the layer relationships that are used to build the VSB project **vsb.vxconfig** file.

Second, open the VSB project **vsb.vxconfig** file. This file is assembled into raw kConfig code from the supplied layer metadata during the VSB build. This raw code defines the layer relationships for the purpose of the VSB build.

Multiple conditional layers can be specified, along with multiple flags, a complex example would be:

```
LAYER_REQUIRES {FLAG_A FLAG_B,  LAYER_X LAYER_Y,  LAYER_W LAYER_Z} LAYER_V
```

This is interpreted as "The layer requires **LAYER_V** in all cases, and requires **LAYER_X** and **LAYER_Y** if **FLAG_A** and **FLAG_B** are enabled, and **LAYER_W** and **LAYER_Z** when **FLAG_A** and **FLAG_B** are not set"

# 8. LAYER VSB CONFIGURATION OPTIONS FILE

The developer of a VxWorks layer may optionally define the file **common.vxconfig**, which declares layer macros and switches and strings that are fed into the VSB build system.

All layer configuration options are fed into the VxWorks VSB build system as a kconfig script. The syntax of the kconfig script is documented by the linux **kernel.org**. You can find the document at kconfig Language.

The contents of the **layer.vsbl** meta-data file is dynamically converted into kconfig script during VSB build time. But you also have the ability to write your own additional kconfig script code to help define the parameters of your layer. You define this code in the optional **common.vxconfig** file which is placed in the layer root directory alongside the **layer.vsbl** meta-data file.

The purpose of the kconfig script code within **common.vxconfig** file is to allow the developer of layers to explicitly declare macros and strings and switches that are customisable by users and are ultimately fed into the VSB build system. The objects you define will be visible from the VSB Source Build Configurator. Layer makefiles and source files will reference these objects by way of ifdef statements and makefile logic.

The state of the macros, strings and switches defined in **common.vxconfig** must often depend on other settings within the VSB, perhaps the presence or absence of other VxWorks layers. The **common.vxconfig** file therefore operates as a switch mechanism that appropriately sets the value of variables that control the VSB build.

For example, the VxWorks PPP layer declares a **common.vxconfig** file. You can find this at *installBase* **/vxworks-7/pkgs_v2/net/ipnet/linkproto-2.0.0.0**. In this file you see the following:

```
config COMPONENT_IPPPP

     bool
     default y

config IPPPP_MP
       bool "Enable PPP with MP"
       default n
       depends on COMPONENT_IPPPP
       help
           Enable PPP with MP

config IPPPP_DEBUG
       bool "Enable PPP Debug mode"
       default n
       depends on COMPONENT_IPPPP && IPCOM_DEBUG_ADVANCED
       help
           This option enables PPP Debug mode and uses the specified DEBUG Build-time Syslog leve
l for this component.

config IPPPP_SYSLOG_PRIORITY
        int
        default IPCOM_DEBUG_SYSLOG_PRIORITY if IPPPP_DEBUG
        default IPCOM_DEFAULT_SYSLOG_PRIORITY

config SUPPORTS_PPP
        bool
        default y
```

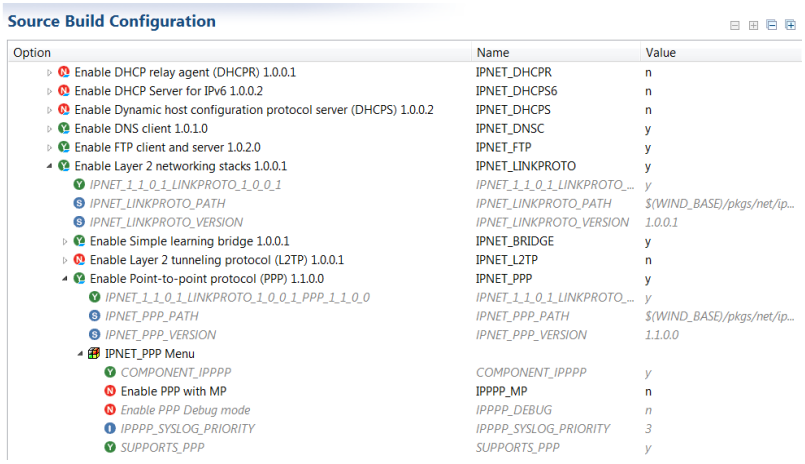
The boolean macro **COMPONENT_IPPPP** is defined and set to y whenever this layer is included.

The boolean macro **IPPPP_MP** is defined whenever this layer is included and therefore **COMPONENT_IPPPP** is set to y. **IPPPP_MP** is set to n by default.

The integer macro **IPPP_SYSLOG_PRIORITY** is defined if this layer is included. It is set to a default value of **IPCOM_DEFAULT_SYSLOG_PRIORITY** unless the macro **IPPPP_DEBUG** is defined. In that case, **IPPP_SYSLOG_PRIORITY** defaults to **IPCOM_DEBUG_SYSLOG_PRIORITY**.

Macros are visible within the VSB Source Build Configuration. And the values of all defined macros can also be modified by the user from within the VSB Source Build Configuration.

Notice that, while all the PPP layer macros are visible in the Source Build Configuration, only **IPPPP_MP** is defined. Others, for example **IPPPP_DEBUG**, are undefined, and therefore greyed out. We can discover why **IPPPP_DEBUG** is undefined by looking at the logic of the kconfig script above. While **COMPONENT_IPPPP** will be defined, presumably **IPCOM_DEBUG_ADVANCED** is not enabled in the VSB. And therefore by the logic in the PPP layer **common.vxconfig**, this means that **IPPPP_DEBUG** is undefined for this VSB.



**Source Build Configuration**

| Option | Name | Value |
|---|---|---|
| Enable DHCP relay agent (DHCPR) 1.0.0.1 | IPNET_DHCPR | n |
| Enable DHCP Server for IPv6 1.0.0.2 | IPNET_DHCPS6 | n |
| Enable Dynamic host configuration protocol server (DHCPS) 1.0.0.2 | IPNET_DHCPS | n |
| Enable DNS client 1.0.1.0 | IPNET_DNSC | y |
| Enable FTP client and server 1.0.2.0 | IPNET_FTP | y |
| Enable Layer 2 networking stacks 1.0.0.1 | IPNET_LINKPROTO | y |
| IPNET_1_1_0_1_LINKPROTO_1_0_0_1 | IPNET_1_1_0_1_LINKPROTO_... | y |
| IPNET_LINKPROTO_PATH | IPNET_LINKPROTO_PATH | $(WIND_BASE)/pkgs/net/ip... |
| IPNET_LINKPROTO_VERSION | IPNET_LINKPROTO_VERSION | 1.0.0.1 |
| Enable Simple learning bridge 1.0.0.1 | IPNET_BRIDGE | y |
| Enable Layer 2 tunneling protocol (L2TP) 1.0.0.1 | IPNET_L2TP | n |
| Enable Point-to-point protocol (PPP) 1.1.0.0 | IPNET_PPP | y |
| IPNET_1_1_0_1_LINKPROTO_1_0_0_1_PPP_1_1_0_0 | IPNET_1_1_0_1_LINKPROTO_... | y |
| IPNET_PPP_PATH | IPNET_PPP_PATH | $(WIND_BASE)/pkgs/net/ip... |
| IPNET_PPP_VERSION | IPNET_PPP_VERSION | 1.1.0.0 |
| IPNET_PPP Menu | | |
| COMPONENT_IPPPP | COMPONENT_IPPPP | y |
| Enable PPP with MP | IPPPP_MP | n |
| Enable PPP Debug mode | IPPPP_DEBUG | n |
| IPPPP_SYSLOG_PRIORITY | IPPPP_SYSLOG_PRIORITY | 3 |
| SUPPORTS_PPP | SUPPORTS_PPP | y |

Wind River has reused the kconfig script technology from the linux kernel configuration tools available with Wind River Linux. The difference between Linux kconfig and VxWorks kconfig is simply in the naming of the variables that are defined in the script. When a Linux script declares a macro, that macro name is automatically extended to **CONFIG_macro_name**. However, the VxWorks kconfig extends variable names differently. They are declared as **_WRS_CONFIG_macro_name**. All Makefiles and source files that refer to the variables must do so as **_WRS_CONFIG_macro_name**.

# 9. LAYER MAKEFILE

A layer must have a makefile that contains macro definitions and included rule files that describe how the layer content is delivered to the VSB build system.

## Introduction

All layers must have a file called **Makefile** and a second file called **layer.vsbl** positioned at the layer's top directory. The makefile in the layer's root directory controls how the layer is built as part of a VSB project. It is essential that the layer makefile and the **layer.vsbl** file are both in the same top level directory. The layer makefile uses macros which define up to three different types of content:

Static Content Macro

Some macros in the makefile specify static content to be delivered to a VSB project like headers and component descriptor files. The macro determines which directory the content originates from before being copied to the VSB. An example of this type of macro is **KERNEL_PUBLIC_H_DIRS**.

Directory Specifications Macro

Some macros specify source directories that are used to build code. The specified directories must contain a makefile in the appropriate format for the library or project source it contains. Typically, a layer makefile will contain a Directory Specification macro to declare the presence of layer source code that must be compiled during the VSB build process. An example of this type of macro is **BUILD_DIRS**.

Toolchain Specification Macro

Sometimes you may wish to override the default compiler settings of the VSB project for your layer. This is done using a Toolchain Specification macro that declares which compiler(s) to use at the layer's chosen build stage(s). An example of this type of macro is **BUILD_TOOLS**.

If the layer provides no source directories or header directories and only contains other layers, the layer makefile is still required because the makefile defines how this parent layer fits into a VSB project. It is common for layer makefiles like this to simply include a rule file. Refer to Describing the VxWorks 7 Layer on page 6 for a description and an example of this makefile.

## Layer Makefiles vs Source Makefiles

Although they have the same filename, layer makefiles are very different from source makefiles. A layer makefile controls the stages at which a layer appears in the VSB build. However, the source makefile does the job of ensuring that source files are compiled properly and libraries are assembled as needed within the layer. Source makefiles are traditional makefiles like the ones found in previous Wind River products (e.g. VxWorks 6). Source makefiles are always located within a layer within source directories pointed to by Directory Specification macros.

Unlike the layer makefile, the source makefile will do the job of actually building the VxWorks library. It does this by defining the macro **LIB_BASE_NAME**. The source makefile is not restricted to building one library. It may contribute to multiple libraries. For example, the source makefile could contain the following lines that incrementally add to the **LIB_BASE_NAMES** library list:

```
ifdef _WRS_CONFIG_CERT_RTP_OBJECT
    LIB_BASE_NAMES += cert
else
    LIB_BASE_NAME = $(LIB_VX_BASE_NAME)
```

One similarity between layer makefiles and source makefiles is that they both include make rules and reference system variables and macros. However, the make rules and macros available to a source makefile are different from the layer makefile ones. For a discussion of the makefile rules, refer to Layer Makefile Rules Reference on page 96.

While source makefiles are essential for ensuring the correct compilation of the layer source and header files, layer makefiles are essential for ensuring that the layer is built as part of a VSB project. The remainder of this section focusses on the layer makefile.

## Sequential Build

Every VSB project build happens in a sequence of three build stages which are subdivided into seven build sub-steps. The layer is built at one or more of these sub-steps, as defined by the choice of layer makefile build macros. For a summary of the available build macros, refer to the table below. For more details on the VSB build stages, please refer to Relationship Between Layers and the VSB Project on page 32.

When deciding which macros to use when writing your layer makefile, it is important to understand the sequential nature of VSB project build stages. Your layer will be built as part of a wider VSB project build, and so you will need to decide which of these build sub-steps is appropriate for it. To ensure that your layer builds properly, all dependant libraries must already have been built by the time the VSB build system reaches your layer.

For example, if you build your dynamic shared library layer using **BUILD_USER_DIRS**, which is scheduled for build sub-stage 4, then you may find your code relies on libraries that have not been built yet. If you were to experience VSB project build link errors, a useful strategy for dealing with the error would be to move your layer code to a later stage in the VSB build pipeline and then rebuild the VSB project again. Move the layer code to a later user stage by removing the definition of **BUILD_USER_DIRS** in the layer makefile and then adding the **POSTBUILD_USERLIB_DIRS** macro to the layer makefile and setting it to the same layer source code directory.

The layer makefile causing VSB build problems could be:

```
# layer Makefile
BUILD_USER_DIRS = src

include $(WIND_KRNL_MK)/rules.layers.mk
```

The layer makefile fixing VSB build problems by delaying the layer build could be:

```
# layer Makefile
POSTBUILD_USERLIB_DIRS = src

include $(WIND_KRNL_MK)/rules.layers.mk
```

Note: It is hard to predict what order layers are compiled in a single sub-stage. This is because layers in a sub-stage don't compile sequentially but in parallel. Most workstations today are multi-core machines. It is common for the host workstation to have a CPU with multiple cores, and be running a host OS (Windows or Linux) that takes advantage of these cores. The VSB project build system on that type of host OS will typically be configured to take advantage of these cores too and parallel up the compilation of the VSB layers in each sub-stage.

## Simple Layers in Kernel Space and User Space

A layer can be configured to build in VxWorks kernel space, where it builds to a library that links and runs in the VxWorks kernel. Alternatively, it may configure itself for user space build. A user space layer will typically implement a static or dynamic user library to support user space RTPs. The layer may also build to an RTP vxe file.

Simple layers will pick one build stage and register for build activity to take place at that stage.

A simple layer makefile that builds for kernel space may look as follows:

```
# layer Makefile
SHARED_PUBLIC_H_DIRS = h
BUILD_DIRS = src
```

```
include $(WIND_KRNL_MK)/rules.layers.mk
```

The source directory makefile must also include the correct makefile rules. If **src** is built for kernel space:

```
# source Makefile
.
.
.
# build rules for source directory
.
.
.
include $(WIND_KRNL_MK)/rules.library.mk
endif
```

Alternatively, if **src** is built for user space:

```
# source Makefile
.
.
.
# build rules for source directory
.
.
.
include $(WIND_USR_MK)/rules.library.mk
endif
```

## Complex Layers - Either Kernel Space or User Space Build

Another application layer might instead decide at VSB project build time whether to build its source directory either for kernel space or user space. For example, assume that a layer contains an audio software stack. This layer and all its child layers can either be built in kernel space or user space, depending on the VSB project build option. Each child layer must therefore also be flexible enough to build its source directories for either kernel or user space. The child layer makefile must check how the audio software stack build macros are set and then choose either kernel space (**BUILD_DIRS**) or user space (**BUILD_USER_DIRS**).

In the demo child layer **Makefile**:

```
# layer Makefile
SHARED_PUBLIC_H_DIRS = h
BUILD_DIRS = src
ifdef _WRS_CONFIG_AUDIO_SW_STACK_USER_TOOL_BUILD
BUILD_USER_DIRS = src
endif
include $(WIND_KRNL_MK)/rules.layers.mk
```

The source directory makefile also needs to be aware of which space it is building in. It is the job of the source makefile in this complex case to correctly compile the available source files to the appropriate libraries. It needs to be able to build the correct libraries and include the correct makefile rules. One approach to this is for the source makefile to use the *SPACE* variable. The bottom lines of the source makefile may look like this:

```
# source Makefile
.
.
.
# build rules for source directory
```

```
.
.
.
ifeq ($(SPACE),user)
include $(WIND_USR_MK)/rules.library.mk
else
include $(WIND_KRNL_MK)/rules.library.mk
endif
```

## Complex Layers - Both Kernel Space and User Space Build

Alternatively, another layer might need to go further and build its source files in both kernel and also user space during the VSB build. This is permitted by the layer build system, and shows its power and flexibility. Our layer may easily register for build work to be done at more than one stage of the VSB build.

We might want to build a layer in both kernel and user space for convenience reasons. We have functionality for kernel space and user space that can share source and header files. We could create two separate layers that build for each space. But it might be more economical to have a single layer building for both kernel and user space. The kernel space build will always happen first.

Configure the layer this way in the layer makefile using the build macros:

```
# layer Makefile
SHARED_PUBLIC_H_DIRS = h
BUILD_DIRS = src
BUILD_USER_DIRS = src

include $(WIND_KRNL_MK)/rules.layers.mk
```

This approach places a bigger demand on the source makefile in the **src** directory. It must handle both kernel and user space builds from the same source directory. It is the job of the source makefile to compile the available source to the appropriate libraries. The source makefile must also include a different make rule file depending on whether it is building in user space or kernel space. It chooses the correct rule file by checking the value of the *SPACE* variable.

The bottom lines of the source makefile might look as follows:

```
# source Makefile
.
.
.
# build rules for source directory
.
.
.
ifeq ($(SPACE),user)
include $(WIND_USR_MK)/rules.library.mk
else
include $(WIND_KRNL_MK)/rules.library.mk
endif
```

## Makefile Rules

Layer makefiles and source makefiles must include an appropriate rule file. It is this rule that ensures the layer is built during a VSB project build. The following line must appear at the bottom of every layer **Makefile**:

**include $(WIND_KRNL_MK)/rules.layers.mk**

A number of other layer rule files exist for different purposes within the layer. Source directory makefiles include different rule files depending on where they occur in the VSB build pipeline. For more details on available makefile rules, see Layer Makefile Rules Reference on page 96.

## Strategy for Reduction in Layer Complexity

The hierarchical structure of complex layers, with the parent layer housing multiple child layers, must be avoided and layer architecture kept simple. One way of simplifying complex layers is to combine conditional compilation techniques with layer build macros in the makefile. This approach establishes the extent of the layer that will be compiled dynamically at VSB build time. The relevant definitions are set in the VSB project, and the layer makefile build macros are assembled to reflect the VSB project definitions.

An example of this approach is the CAN layer, located at *installDir* **/vxworks-7/pkgs_v2/connectivity/can**.

```
$tree -L 1
.
├── core
├── ctlr
├── vxTest
├── can.exclude
├── can.spec
├── common.vxconfig
├── layer.vsbl
├── Makefile
```

The makefile is located at *installDir* **/vxworks-7/pkgs_v2/connectivity/can/Makefile**.

```
# Makefile - CAN Layer Makefile

include $(WIND_KRNL_MK)/defs.layers.mk

ifdef _WRS_CONFIG_CAN_CORE
KERNEL_PUBLIC_H_DIRS += core/h
BUILD_DIRS += core/src
POST_NOBUILD_CDFDIRS += core/cdf
endif

ifdef _WRS_CONFIG_CAN_CTLR
KERNEL_PUBLIC_H_DIRS += ctlr/h
BUILD_DIRS += ctlr/src
POST_NOBUILD_CDFDIRS += ctlr/cdf
endif

ifneq ("$(wildcard vxTest/src/Makefile)","")
ifdef _WRS_CONFIG_CAN_VXTEST
KERNEL_PUBLIC_H_DIRS += vxTest/h
BUILD_DIRS += vxTest/src
POST_NOBUILD_CDFDIRS += vxTest/cdf
endif
endif

include $(WIND_KRNL_MK)/rules.layers.mk
```

Three independent CAN layer elements exist, and each is individually selected by the VSB using the macros **_WRS_CONFIG_CAN_CORE**, **_WRS_CONFIG_CAN_CTLR** and **_WRS_CONFIG_CAN_VXTEST**. For each macro that is defined, the CAN layer makefile appends the relevant directories to the makefile layer build macros that control the compilation of this particular

independent CAN layer element. This approach ensures that the correct parts of the layer are compiled, and the unnecessary parts are omitted when the VSB is built.

Note: This makefile does not set the value of each layer build macro, rather it appends to it. This allows multiple, independent layer elements to be selected and built together when the VSB project is built.

Table 1. CAN Layer Optional Elements

| Macro Set in VSB Project | CAN Layer Code to be Built | Header Files Required | CAN Layer Component Details |
|---|---|---|---|
| **_WRS_CONFIG_CAN_CORE** | **BUILD_DIRS** += **core/src** | **KERNEL_PUBLIC_H_DIRS** += **core/h** | **POST_NOBUILD_CDFDIRS** += **core/cdf** |
| **_WRS_CONFIG_CAN_CTLER** | **BUILD_DIRS** += **ctlr/src** | **KERNEL_PUBLIC_H_DIRS** += **ctlr/h** | **POST_NOBUILD_CDFDIRS** += **ctlr/cdf** |
| **_WRS_CONFIG_CAN_VXTEST** | **BUILD_DIRS** += **vxTest/src** | **KERNEL_PUBLIC_H_DIRS** += **vxTest/h** | **POST_NOBUILD_CDFDIRS** += **vxTest/cdf** |

## Example Layers

One way to learn how to construct a custom layer makefile is to explore how the Wind River VxWorks layers have been constructed. Customer layers will use the same VSB project build system that Wind River product layers use. When developing a custom layer, it is instructive to explore how these Wind River layers are written.

For example, to explore how **PRE_BUILD_GENH_DIRS** macro is used by the Wind River supplied layers on Linux host:

```
~/WindRiver/vxworks-7$ cd pkgs_v2
~/WindRiver/vxworks-7/pkgs_v2$
~/WindRiver/vxworks-7/pkgs_v2$ grep -r PRE_BUILD_GENH_DIRS *
net/end-2.0.0.0/Makefile:PRE_BUILD_GENH_DIRS += drv/genh
os/hv/guest_support_vx7-1.0.0.0/Makefile:PRE_BUILD_GENH_DIRS += virtio/genh
os/core-2.0.0.0/Makefile:PRE_BUILD_GENH_DIRS += kernel/genh
os/psl/other/xen-2.0.0.0/Makefile:# 14jun18,yat  changed PRE_BUILD_GENH_DIRS to genh directory
os/psl/other/xen-2.0.0.0/Makefile:PRE_BUILD_GENH_DIRS = genh
os/psl/intel/itl_x86_common-1.0.0.0/Makefile:PRE_BUILD_GENH_DIRS = genh
os/drv/vxbus-2.0.0.0/Makefile:# 14jun18,yat  changed PRE_BUILD_GENH_DIRS to genh directory
os/drv/vxbus-2.0.0.0/Makefile:PRE_BUILD_GENH_DIRS += buslib/genh
os/drv/vxbus-2.0.0.0/Makefile:PRE_BUILD_GENH_DIRS += core/genh
os/drv/vxbus-2.0.0.0/Makefile:PRE_BUILD_GENH_DIRS += subsystem/genh
security/crypto_misc-2.0.0.0/Makefile:PRE_BUILD_GENH_DIRS += tpm/genh
storage/bdm-2.0.0.0/Makefile:PRE_BUILD_GENH_DIRS += sdmmc/core/genh
~/WindRiver/vxworks-7/pkgs_v2$
```

The files listed are working examples of the use of the **PRE_BUILD_GENH_DIRS** makefile macro.

## About Layer Makefile Macros

Table 2. Layer Makefile Macro Overview - Build Stages

| Build Stage | Build Sub-Stage | Macro Name | Overview |
|---|---|---|---|
| **PREBUILD** | 1 | **PRE_BUILD_GENH_DIRS** | Directory Specification Macro<br>Assemble header files in these directories.<br>Recommended layer directory name is **genh**. |
| | 1 | **KERNEL_PUBLIC_H_DIRS** | Static Content Macro<br>Public header files needed during kernel space build are found in these directories.<br>Recommended layer directory name is **h**. |
| | 1 | **USER_PUBLIC_H_DIRS** | Static Content Macro<br>Public header files needed during user space build are found in these directories.<br>Recommended layer directory name is **usr_h**. |
| | 1 | **SHARED_PUBLIC_H_DIRS** | Static Content Macro<br>Public header files needed during both kernel and user space build are found in these directories.<br>Recommended layer directory name is **share_h**. |
| | 1 | **KERNEL_PUBLIC_H_FILES** | Static Content Macro<br>Specific public header files for kernel build are listed here. |
| | 1 | **USER_PUBLIC_H_FILES** | Static Content Macro<br>Specific public header files for user space build are listed here. |
| | 1 | **SHARED_PUBLIC_H_FILES** | Static Content Macro<br>Specific public header files for both kernel and user space build are listed here. |
| | 1 | **KERNEL_PROTECTED_H_DIRS** | Static Content Macro<br>Protected header files needed during kernel space build are found in these directories.<br>Recommended layer directory name is **prot_h**. |
| | 1 | **USER_PROTECTED_H_DIRS** | Static Content Macro<br>Protected header files needed during user space build are found in these directories.<br>Recommended layer directory name is **usr_prot_h**. |
| | 1 | **SHARED_PROTECTED_H_DIRS** | Static Content Macro |

| Build Stage | Build Sub-Stage | Macro Name | Overview |
|---|---|---|---|
| | | | Protected header files needed during both kernel and user space build are found in these directories.<br>Recommended layer directory name is **share_prot_h**. |
| | 1 | **KERNEL_PROTECTED_H_FILES** | Static Content Macro<br>Specific protected header files for kernel build are listed here. |
| | 1 | **USER_PROTECTED_H_FILES** | Static Content Macro<br>Specific protected header files for user space build are listed here. |
| | 1 | **SHARED_PROTECTED_H_FILES** | Static Content Macro<br>Specific protected header files for both kernel and user space build are listed here. |
| | 1 | **KERNEL_SYSTEM_H_DIRS** | Static Content Macro<br>System header files needed during kernel space build are found in these directories. |
| | 1 | **KERNEL_SYSTEM_H_FILES** | Static Content Macro<br>Specific system header files for kernel space build are listed here. |
| | 1 | **POST_NOBUILD_CDFDIRS** | Static Content Macro<br>Directories containing cdf files to be used when building a VIP.<br>Recommended layer directory name is **cdf**. |
| | 2 | **PRE_BUILD** | Directory Specification Macro<br>Directories that contain the source of small, stand alone libraries.<br>Recommended layer directory name is **pre_src**. |
| **BUILD** | 3 | **BUILD_DIRS** | Directory Specification Macro<br>Directories containing code to be built into kernel libraries.<br>Recommended layer directory name is **src**. |
| | 4 | **BUILD_USER_DIRS** | Directory Specification Macro<br>Directories containing code to be built into user space libraries.<br>Recommended layer directory name is **usr_src**. |
| **POSTBUILD** | 5 | **POSTBUILD_USERLIB_DIRS** | Directory Specification Macro<br>Directories containing code to be built and linked with user space libraries. |

| Build Stage | Build Sub-Stage | Macro Name | Overview |
|---|---|---|---|
| | 5 | **POST_NOBUILD_CFGDIRS** | Static Content Macro<br>Directories containing configelette files to be used when building a VIP.<br>Recommended layer directory name is **configlette**. |
| | 6 | **POSTBUILD_DKM_DIRS** | Directory Specification Macro<br>Directories containing code to be built into DKM object files.<br>Recommended layer directory name is **dkm**. |
| | 7 | **POSTBUILD_RTP_DIRS** | Directory Specification Macro<br>Directories containing code to be built into RTP vxe files.<br>Recommended layer directory name is **rtp**. |

The compiler used to build the layer is specified in the VSB project global configuration. This is usually set in the VSB configurator tool. However, it is possible to override this setting. Different parts of the layer can be compiled using different tools if necessary. Depending on which type of content you wish to override, and which VSB build stage it appears in, assign one of the macros in the table below in the layer makefile.

Table 3. Layer Makefile Macro Overview - Toolchain

| Macro Name | Build Sub-Stages | Description |
|---|---|---|
| **PRE_BUILD_TOOLS** | 1, 2 | Defines the tools used for content specified by the **PRE_BUILD_GENH_DIRS** and **PRE_BUILD_DIRS** macros. |
| **BUILD_TOOLS** | 3 | Defines the tools used for content specified by the **BUILD_DIRS** macro. |
| **BUILD_USER_TOOLS** | 4,5 | Defines the tools used for content specified by the **BUILD_USER_DIRS** and **POSTBUILD_USERLIB_DIRS** macros. |
| **DKM_TOOLS** | 6 | Defines the tools used for the content specified by the **POSTBUILD_DKM_DIRS** macro. |
| **RTP_TOOLS** | 7 | Defines the tools used for the content specified by the **POSTBUILD_RTP_DIRS** macro. |

The value assigned to the toolchain macro is one of the following:

- **$(_WRS_CONFIG_TOOL)** is the default setting and selects the compiler as chosen in the VSB configurator tool.
- **$(_WRS_CONFIG_APP_TOOL)** selects the compiler as chosen in the VSB configurator tool.
- **$(_WRS_CONFIG_ALL_TOOLS)** selects the compiler as chosen in the VSB configurator tool.
- **$(_WRS_CONFIG_llvm_VERSION)** selects the Clang (LLVM) compiler only.
- **$(_WRS_CONFIG_gnu_VERSION)** selects the GNU GCC compiler only.

- If no value is assigned; the default compiler as chosen in the VSB configurator tool is used.

If you intend to use the toolchain macros, you must ensure that the following makefile rule is also included first at the top of your layer makefile:

**include $(WIND_KRNL_MK)/defs.layers.mk**

See Layer Makefile Macros Reference on page 83 for a description of all available macros and Layer Makefile Rules Reference on page 96 for a description of the available makefile rule files.

## Example Layer Makefile

The following example illustrates the structure of the USB core layer that is contained in the USB layer. The layer is located in the Wind River installation at *installDir* **/vxworks-7/pkgs_v2/connectivity/usb-2.0.0.0** directory.

```
$tree -L 1
.
├── common.vxconfig
├── core
├── ctlr
├── host
├── layer.vsbl
├── Makefile
├── otg
├── phy
├── target
```

The USB layer uses the complexity reduction technique described earlier. The layer makefile contains conditional compilation that is driven by the VSB project configuration, and the layer build macros are constructed dynamically based on this configuration. This approach dynamically selects which elements of the USB layer are required and which can be omitted when the VSB is built. These optional elements are not child layers, they are part of the USB layer:

```
# Makefile - Wind River USB Core Layer Makefile #

include $(WIND_KRNL_MK)/defs.layers.mk

ifdef _WRS_CONFIG_USB_CORE
KERNEL_PUBLIC_H_DIRS += core/h
BUILD_DIRS += core/src
POST_NOBUILD_CFGDIRS += core/cfg
POST_NOBUILD_CDFDIRS += core/cdf
endif

ifdef _WRS_CONFIG_USB_HCORE
KERNEL_PUBLIC_H_DIRS += host/core/h
BUILD_DIRS += host/core/src
POST_NOBUILD_CFGDIRS += host/core/cfg
POST_NOBUILD_CDFDIRS += host/core/cdf
endif

...

ifdef _WRS_CONFIG_USB_OHCI
BUILD_DIRS += ctlr/ohci/src
POST_NOBUILD_CDFDIRS += ctlr/ohci/cdf
endif
...
```

```
include $(WIND_KRNL_MK)/rules.layers.mk
```

The **WIND_KRNL_MK** environment variable currently expands to the *installDir* **/vxworks-7/build/mk/krnl/** directory. All the layer kernel space makefile rule files are contained in this directory.

# 10. BINARY LAYERS

A binary layer is a VxWorks layer that contains binary objects. Use a binary layer to prevent users from seeing the underlying source code for your feature.

## About Binary Layers

Due to security concerns, you may decide that your VxWorks components cannot be released as sublayers containing source code. In this case you can release the prebuilt binary objects as binary layers. This allows you to deliver functionality without giving users the ability to see your source code.

A binary layer's syntax and requirements are similar to a standard layer that delivers headers or source code, regardless of differences in content.

Most prebuilt binary objects only support certain CPU types and operating environments such as UP or SMP. You must specify the supported types in the **layer.vsbl** file for the binary layer.

## Creating Binary Layers

Wind River recommends that you use **wrtool** command-line utility to automatically generate the binary layer from a source layer. This is done by using variations of the command prj vsb createBinary.

Wind River also recommends that you add the required variations you want the binary layer to be built in, to the **VSB_BINARY_TOOL** metadata of the source layer.

The **VSB_BINARY_TOOL** metadata has the following format:

```
{CPU TOOL FP ENDIAN UP/SMP LP64}
```

You can specify multiple configurations. Each configuration is enclosed in curly braces ({}) and the configuration variables are defined as follows:

CPU

The CPU used to build the binary layer.

TOOL

The compiler (llvm/gnu) used to build the binary layer.

FP

The floating point designation. This can be FP_khard, FP_hard, FP_ksoft, FP_soft or "" (empty). When set to "", the default value for the CPU is used.

ENDIAN

The endian value to be used. This can be little, big; or "" (empty). When set to "", the default value for the CPU is used.

UP/SMP

The multiprocessing setting to use. This can be set to uniprocessor (UP) or symmetric multiprocessor (SMP).

LP64

When specified the build uses the LP64 flag. If set to "" (empty), ILP32 is used.

With the above definitions in place in the source layer, you can use the prj vsb listBinaryCmds command to list the options to build the binary layers. This makes the generation of the binary layer much easier to replicate, for example in a script environment.

```
workspace> prj vsb listBinaryCmds

vxprj vsb createBinary -cfg  PPC603     diab  ""     ""  UP  ""  compat69 -force -buildPPC_1_2_0_2
_PPCMATH_1_0_2_1
vxprj vsb createBinary -cfg  PPC603     diab  soft   ""  UP  ""  compat69 -force -buildPPC_1_2_0_2
_PPCMATH_1_0_2_1
vxprj vsb createBinary -cfg  PPC604     diab  ""     ""  UP  ""  compat69 -force -buildPPC_1_2_0_2
_PPCMATH_1_0_2_1
vxprj vsb createBinary -cfg  PPC604     diab  soft   ""  UP  ""  compat69 -force -buildPPC_1_2_0_2
_PPCMATH_1_0_2_1
.
.
.
```

The build commands generated include the full name of the layer and its version. The -force option is used to delete the previously built binary, and the -build option is used to invoke the build itself. Also note that each dataset from **VSB_BINARY_TOOL** resulted in a unique configuration that is prefaced with -cfg.

The prj vsb createBinary command can also be used manually to create and build binary layers.

The new binary layer is generated in the same location as the source layer. As soon as the binary layer is generated, it can be used in a VxWorks source build (VSB).

## Directory Structure

A binary layer can contain the following subdirectories and files:

**cdf**

Subdirectory containing any component description files (CDFs) that the binary layer wants to introduce for VxWorks image project (VIP) creation.

**cfg**

Subdirectory containing any configlettes that can be compiled in the VIP based on the CDF selections.

**genh**

Subdirectory containing any code that is required to generate the header files needed during any part of the build.

**h**

Subdirectory containing public headers for the kernel library build.

**prebuilt**

Subdirectory containing the binary objects.

> Note: These files should not be edited.

**src**

Subdirectory containing a makefile to deliver the objects to the VSB project.

**user_src**

Subdirectory containing any source and nonpublic header files to be compiled for the user side libraries.

**layer.vsbl**

File containing the metadata information for the binary layer.

**Makefile**

Defines the layer as a binary layer.

**src/Makefile**

Subdirectory containing all of the makefiles from the source layer. These makefiles are used to propagate the objects to the VSB into proper archives.

## The Binary Layer Makefile

The makefile for a binary layer is automatically generated and must not be modified.

If you use the standard directory structure described previously, the makefile is straightforward. In this case, **Makefile** sources the following file:

```
include $(WIND_KRNL_MK)/rules.binlayer.mk
```

No other declarations are necessary to ensure that the contents of the **prebuilt** library are copied into the VSB directory at the correct time during the build. All the headers, CDF files, and configlettes are processed properly. The makefile in the **src** directory copies and archives objects based on a **__BIN_LAYER** command-line macro passed to it by this **Makefile**.

## Binary Layers layer.vsbl file

The **layer.vsbl** file for a binary layer is automatically generated and must not be modified. The layer metadata indicates the architecture and compiler that are compatible with the layer using the **VSB_BINARY** macro. The following example is for PowerPC math libraries compiled with the GNU compiler:

```
/* Automatically generated Binary target do not edit */
/* layer.vsbl for PPCMATH PPC604 gnu */


Layer  PPCMATH {
SYNOPSIS        PPC math libraries
HELP            This provides the PPC math libraries which are only used for kernel space.
VENDOR          Wind River
VERSION         2.0.0.0_B_PPC604gnu
VSB_REQUIRES    !PPC_PPCMATH_DISABLED && (ARCH_ppc && !LP64 && !(CPU_PPCE500V2 && FP_khard))
LAYER_REQUIRES
SELECT
SUBSCRIBE
FEATURE_PROVIDES
FEATURE_REQUIRES
LAYER_TYPE      GENERIC
FEATURE         COREOS
RETROFIT
VSB_BINARY      CPU_PPC604  \
                && gnu_SUPPORTED \
                && !FP_ksoft \
                && !FP_soft \
                && !LP64 \
                && !SMP \
```

```
PARENT          PPC_2_0_0_0
DOC_BUILD       NO
OPTIONAL        NO
DEFAULT         YES
LAYER_CONTENT   BINARY
}
```

# 11. RELATIONSHIP BETWEEN LAYERS AND THE VSB PROJECT

VxWorks layers are integrated into the build of a VxWorks source build (VSB) project in a structured way.

Layers are supported directly by the Workbench VSB project infrastructure. A layer is included in a VSB by using either the command line tool **wrtool** or the Workbench Source Build Configuration.

The look of the layers in the Workbench Source Build Configuration is controlled by layer meta-data, and the directory arrangement in the **pkgs_v2** directory. The process of building each layer into the VSB is done in a staged process. The various file and directory components of the layer are introduced into the VSB build pipeline at three different stages.

The three VSB layer build stages are as follows:

- Stage 1 - PREBUILD. For all the layers enabled in the VSB, all of the public and system header files are copied from the layer directories into the VSB build area. Any header files that must be auto-generated are also done so at this stage.
- Stage 2 - BUILD. At this stage, all kernel side source code is built into the VSB as libraries. The library names are derived from the macro *LIB_BASE_NAME* as defined in the layer's **src** directory makefile. Once kernel side libraries are built, the user side source code is subsequently compiled.
- Stage 3 - POSTBUILD. For all the layers enabled in the VSB, the available configlettes and cdf files are processed into the VSB at this stage. Also, any DKM and RTP code present in the layers is also built into the VSB.

To ensure the layer is built and included into a VSB project, the layer must register itself with the VSB project build infrastructure and it must request that build work be done at the correct time during VSB project build. This register step is done by the layer makefile. This makefile specifies one or more makefile macros which register for layer work to be done by the VSB project build infrastructure.

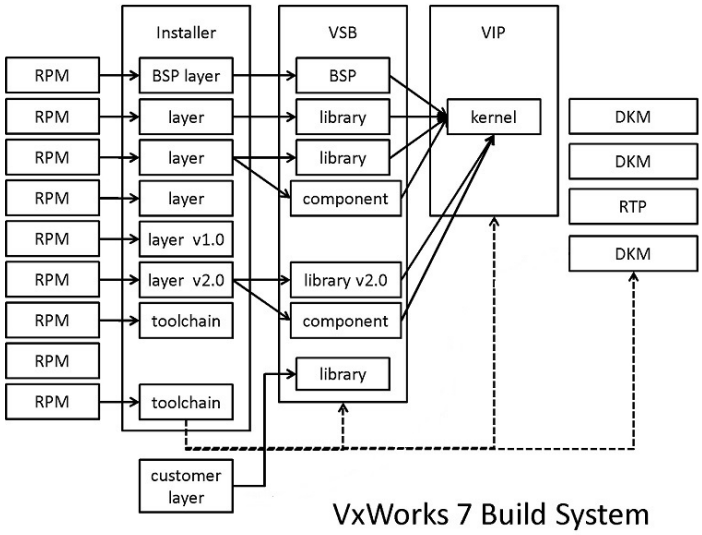
When deciding which macros to use when writing your layer makefile, it is important to understand the sequential nature of VSB project build stages. Your layer will need to be built as part of a wider VSB project, and so you will need to choose the most appropriate VSB build stage to build your layer at. To ensure a successful build, all dependant libraries that your layer relies on must already have been built when the VSB build system reaches your layer.

For more information on layer makefile macros, please refer to Layer Makefile on page 17 and Layer Makefile Macros Reference on page 83.

# 12. INSIDE AND OUTSIDE LAYERS

VxWorks software components are delivered as RPM files, packaged as layers and may reside inside or outside of the Wind River installation directory.

Source and binary components are presented to the VxWorks build system as RPM files containing layers. A layer encapsulates a piece of functionality that may be built into the VxWorks kernel.




Prior to VxWorks product installation, the functionality within each layer is packaged as an RPM supplied either on installation media, or retrieved from the Wind River servers by the installer or **Product Maintenance** tool. At fresh installation time, the installer takes the RPM files, unpacks them and assembles the initial set of VxWorks layers. The **Product Maintenance** uses the installer to incrementally add new layers into the Wind River installation after a fresh product installation has completed. This is a useful mechanism for both partners and customers who wish to easily add new functionality to VxWorks like a BSP, an API, or a driver.

Layers that exist within the installation directory are located at *installDir* **/vxworks-7/pkgs_v2/**. These layers are referred to as *inside layers*, because they are positioned inside the installation directory structure.

Layers are also a useful way of encapsulating software applications which have been developed by a Wind River customer. However, this customer may not want to be forced to deliver their software application to their own end customers as an internal layer. They may have good reason to decouple their own product installation from the Wind River installation. For this reason, layers may also be positioned outside of the installation. Layers can exist somewhere else on the file system, external to *installDir* **/vxworks-7/pkgs_v2/**. These layers are referred to as *outside layers* because they are located outside of the installation directory.

Outside Layers have all the features and benefits of inside layers except one. The installer cannot install the outside layer to the file system, it can only tell the VxWorks build system that the outside layer is expected to reside there. It is therefore your responsibility to ensure that the RPM file is extracted to the correct directory before the installer reads the RPM and identifies it as an outside layer.

For example tutorials that lead you through the creation of inside and outside layers, please refer to VxWorks 7 Inside Layers Tutorial and VxWorks 7 Outside Layers Tutorial and VxWorks 7 Layer Version Management Tutorial

# 13. ABOUT BSPS

Board support packages (BSPs) are located in the VxWorks **pkgs_v2** directory and use a different metadata file format from other layers.

All VxWorks BSPs are located under the *installDir* **/vxworks-7/pkgs_v2/os/board/** directory.

BSPs are further categorized into subdirectories by their corresponding architectures. For example, the Intel-based BSPs are stored under the **ia** directory.

```
$ tree -f -L 1 vxworks-7/pkgs_v2/os/board/
vxworks-7/pkgs/os/board
├── vxworks-7/pkgs_v2/os/board/freescale
├── vxworks-7/pkgs_v2/os/board/intel
├── vxworks-7/pkgs_v2/os/board/nxp
├── vxworks-7/pkgs_v2/os/board/other
├── vxworks-7/pkgs_v2/os/board/renesas
├── vxworks-7/pkgs_v2/os/board/sifive
├── vxworks-7/pkgs_v2/os/board/ti
├── vxworks-7/pkgs_v2/os/board/wrs
├── vxworks-7/pkgs_v2/os/board/xilinx
```

A VxWorks BSP uses a metadata file called **bsp.vsbl**. The BSP metadata file contains the name of the BSP, a version and its dependencies. Other metadata fields found in layers are not required. An example**bsp.vsbl** file is as follows:

```
/* bsp.vsbl - Xilinx zynq 7000 BSP metadata file */

Layer xlnx_zynq7k
    {
    SYNOPSIS            Xilinx zynq 7000 BSP
    HELP                This layer provides the Xilinx Zynq-7000 board support package (BSP).
    VERSION             2.0.0.0
    VENDOR              Wind River
    LAYER_REQUIRES      FDT UTILS XLNX_ZYNQ VXBUS_DRV ARM
    FEATURE_REQUIRES
    }
```

A VSB project can be based either upon a BSP or a supported VxWorks CPU architecture. The **LAYER_REQUIRES** metadata in a BSP **bsp.vsbl** file behaves differently depending on how the VSB project has been built.

VSB Project Based on a BSP

When a VSB is based on a BSP, **LAYER_REQUIRES** is used to assemble the additional layers that the BSP needs in the VSB project. Each required layer is automatically added to the VSB project as if prj vsb add had been called for each one.

VSB Project Based on a CPU Architecture

When a VSB is based on a CPU architecture, **LAYER_REQUIRES** is used to test if a BSP is capable of being supported by the VSB project. To be supported by the VSB, all the layers specified by **LAYER_REQUIRES** in **bsp.vsbl** must be buildable for the architecture already selected for this VSB.

The **FEATURE_REQUIRES** in the above **bsp.vsbl** assumes that the LAYER being required has a **FEATURE_PROVIDES** list that meets the feature requirements.

# 14. CREATING A LAYER

## 1. Preparing to Create a Layer

Creating a custom layer requires careful planning before coding and layer construction can begin.

### About This Task

If you are a Wind River partner who develops VxWorks middleware, or a customer responsible for writing and delivering software libraries or VxWorks BSPs, then the layer may be an attractive way to package and deliver technology to your users.

The layer mechanism allows you to encapsulate feature sets and deliver them to your end customers in source or binary form. Your users can safely update to your latest layer version without breaking their current one. Further, layers are a framework letting you modify and customize existing Wind River or third party source layers. Lastly, you may take your existing application or middleware code and easily port it to the layers framework, making a modularized layer out of your proprietry or GPL code. Layers are a powerful construct, and you are encouraged to take advantage of them.

When preparing to create a new VxWorks layer you must consider the following issues.

### Procedure

1. What will your layer do?

   Some layers contain BSPs, other layers contain operating system libraries while others implement libraries to support target board development tools. Try to pin down the specific function of your layer. If this is difficult, then perhaps it is because you have more than one layer that you must create. Identify the different functions, and you will have identified the different layers as well.

   Once you have decided what your layer will do, you must then consider where it will be positioned on your workstation filesystem. Usually layers are positioned in the *installDir* **/vxworks-7/pkgs_v2** directory. The function of the layer determines which sub directory the layer is positioned in.

   For example:

   - BSP layers are operating system components and so are positioned at **pkgs_v2/os/board**.
   - Debug Agents for development tools are positioned at **pkgs_v2/os/debug_analysis/rttools**.
   - Network Stack protocols are positioned at **pkgs_v2/net**.

   Based on the purpose and function of your intended layer, decide where it will live under the **pkgs_v2** directory. If possible, position it in one of the existing sub-directories.

2. Think about how the layer's functionality will be used by the VxWorks system that eventually contains it.

   For example, consider the following questions:

   - Will your layer provide library code for the system to use, or will the layer implement a specific system function or will it do both?
   - Will your layer contain a VxWorks driver, or will there be a combination of driver plus application code in your layer?
   - If there is application code contained in this layer, will this code run in kernel space or user space or both?
   - Will this layer publish its software API to the other layers in the VxWorks system? Or will it be private?

- How much functionality is this layer bringing to the VSB? Is it a device driver, or is it a complete software stack? The heavier the layer's functionality, the more likely it is that you will need to sub-divide the layer into multiple sub-layers. Must we sub-divide the parent layer into multiple children layers? Or is one layer level enough?

It is important that you have an understanding of these issues. The answers to these sorts of questions will determine how you structure your layer directory and how you position software components within it.

For example, whether or not your layer code runs in kernel space or user space will determine which layer sub-directories and layer makefile macros you must populate your layer directory with.

3. Test the code that you intend to encapsulate by your layer.

   Any code that your layer contains will be built during VSB project build. The VSB is essentially a heavy weight, complete VxWorks system build from source code. The layer code you build into a VSB can only be exercised once a VIP has been created based on the VSB and booted by a compatible target.

   While it is certainly possible to debug your layer code when it is contained within a VSB project, this is not an easy workflow. It will take more effort and time to test your layer code this way. Instead, why not test your layer code as thoroughly as you can before you encapsulate it by your layer and your VSB project?

   The quickest way to test your code, is to take advantage of the rapid software development features provied by VxWorks and Workbench. For example, build the layer's kernel space components as a VxWorks DKM project. This gives you the benefit of a short edit-compile-debug cycle with the option to test DKM code easily on either a target board or on a simulated target board. Similarly, build a user space layer component as an RTP and test it that way first.

   In summary, the more tested your software components are before you begin the process of constructing your layer from them, the easier and quicker this process will be.

4. Decide whether or not your layer code will be compatible with one or more Workbench compiler toolchains.

   Workbench ships with multiple compiler toolchains. It is recommended that layers are presented to the system as fully tested as possible. The choice of compiler toolchain is part of this testing process.

   It is perfectly possible to write code that is compatible with and will be compiled successfully by all of these compilers. You can ship your layer so that it will compile with any toolchain. However, achieving this may take some extra time and effort that you cannot afford.

   The way to deal with this is to limit your layer so that, whatever setting the VSB has, the layer is always built with a single predictable toolchain you have tested the layer with. You do this by setting the appropriate toolchain macro in the layer makefile as you construct your layer.

   If you don't specify a toolchain macro in your layer makefile, then you are saying that you are happy for your layer to be compiled by whichever toolchain your user decides to use in their VSB project.

5. How will you know your layer is functioning as expected once it is deployed on the target?

   If your layer runs code during the VxWorks boot stages, it might be useful if your layer code output some log messages that could be turned off once the system is working as expected. If your layer contains an API that is accessed by other layers in the system, it may be useful to use the public header macros to make these functional calls public so that they are callable directly from the VxWorks shell and so are easy to test. If this is not possible, then you may want to consider building some test code into your layer which exercises the API functions, checks that they are functioning correctly and reports a result.

## 2. Create a Directory for Your Layer

The right location for a layer project is important and must be decided upon before the layer project creation is done.

## About This Task

Before we create our layer project, we must create an empty directory for it.

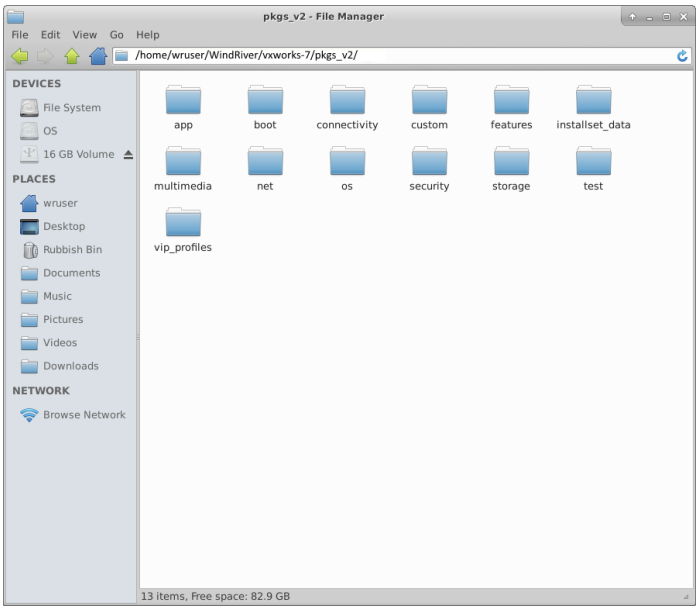
## Procedure

1. Choose a location for the layer project.

   A layer project is not usually created in the workspace directory. Most layers exist in the *installDir* **/vxworks7/pkgs_v2** directory. Layers are built as part of a VSB project and so must be positioned where the VSB build system can find them.

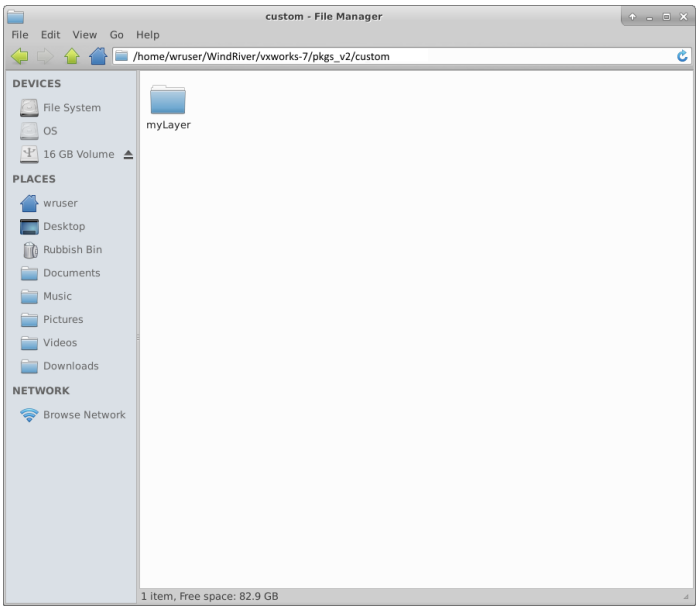
   The **pkgs_v2** directory contains a sub-directory for each functional area within the VxWorks system. Decide where your new layer will live.

   > Note: Layers usually exist within the **pkgs_v2** directory but they can also be located at any other custom location on the workstation filesystem. For more information on creating layers at a custom location refer to Using Layers at a Custom Location on page 50.




2. Create a directory for your layer project.

   Using the explorer or the command line, create an empty directory to contain your new layer. If possible, position your layer in the **pkgs_v2/custom** directory. In the instructions that follow, **myLayer** will be created. So create the empty directory **pkgs_v2/custom/myLayer**.




# 3. Creating a Layer Project Using Workbench

To create a layer using Workbench involves choosing and configuring a Workbench **layer** project.

## Before You Begin

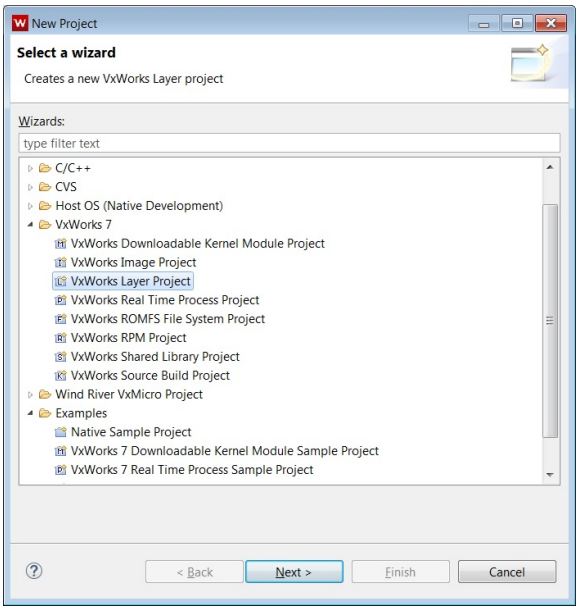
Before you begin, you must have selected a location for your layer and created a directory there.

## About This Task

The construction of your layer may be done within Workbench. The steps are as follows:

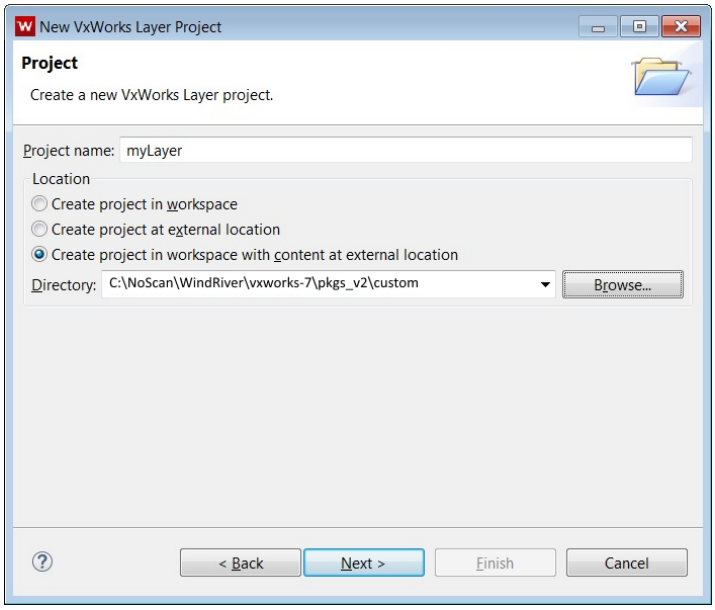
## Procedure

1. In Workbench, select **New Project**.
2. In the wizard, select the VxWorks Layer Project and click **Next**.



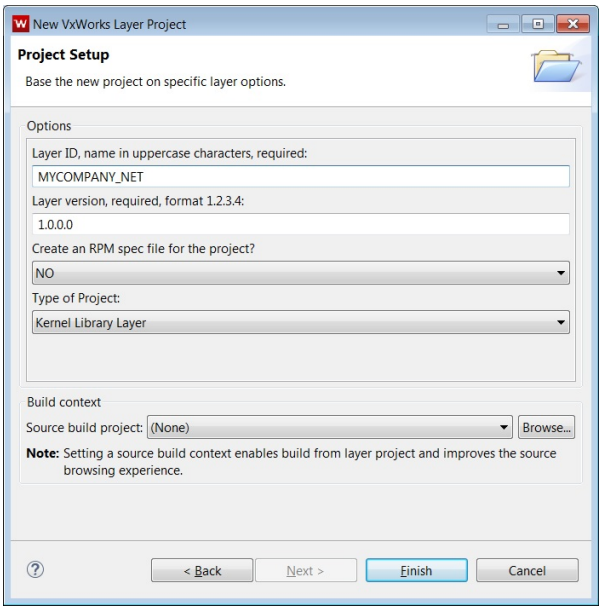

3. Create the layer project in the new directory.
   1. Enter a project name.
   2. Choose to create it in the empty layer directory you created earlier.

   Select the option **Create project in workspace with content at external location** and click **Browse** to select your empty layer directory.

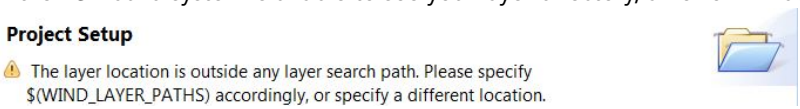



   3. Select **Next** to continue.
4. Enter a layer ID and then click **Finish**.

   The layer ID must be unique. Consider prefixing your company name to the ID to ensure uniqueness.




If the VSB build system is unable to see your layer directory, an error will be reported at the top of this window:



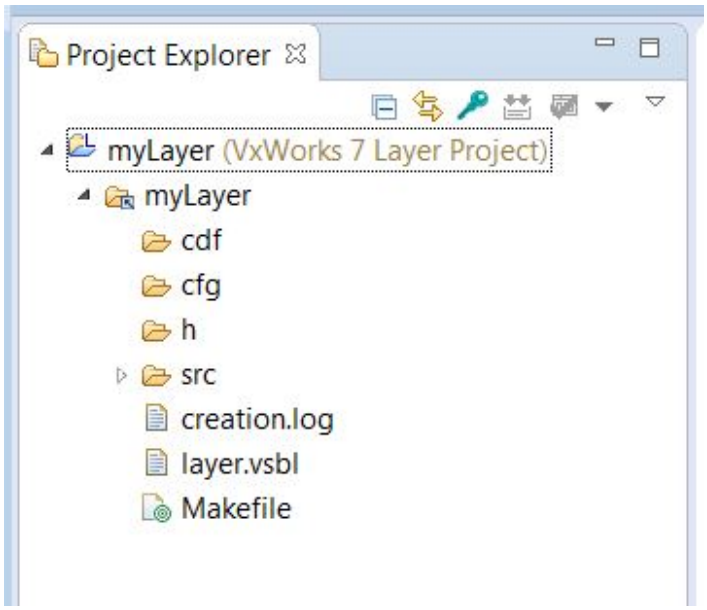

The error means while you may build this layer project in your chosen directory, you will be unable to add this layer to any VSB projects afterwards. The VSB build system will not be able to see it.

It is recommended that you cease layer project creation, rethink the layer directory position, and start project creation again.

> **Note:** If you chose to create the layer project in your Wind River Workbench workspace directory, under normal circumstances you will see this error. It is recommended that you change the layer location to a directory under *installDir* **/vxworks-7/pkgs_v2** instead.
>
> Note that you will not get this error if you have forced the VSB build system to search your project workspace for layers.

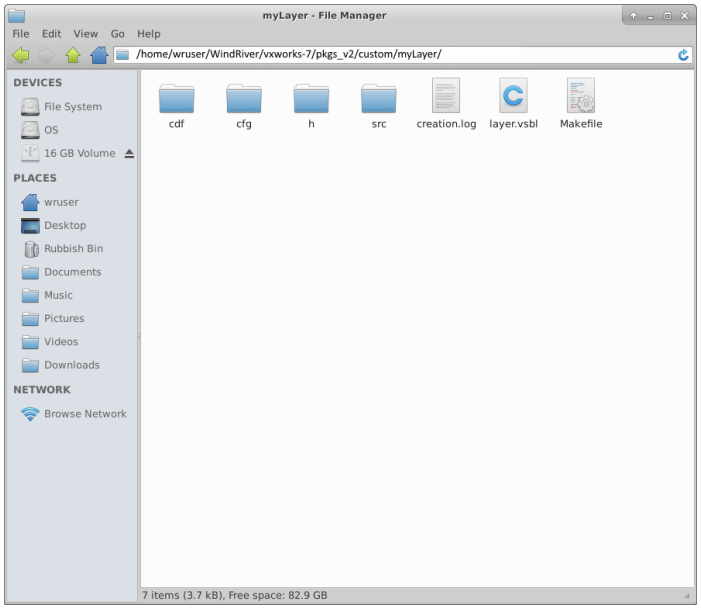
Following the creation of your layer, the empty layer is now visible in three different ways:

- Workbench Project Explorer
  While the **myLayer** project metadata lives in the Workbench workspace folder, all of your layer project files are located correctly in *installDir* **/vxworks-7/pkgs_v2/custom/myLayer**. To verify this, right click on the **Makefile**, select **Properties** and note the file location is not the workspace directory, but instead it is your chosen layer directory.



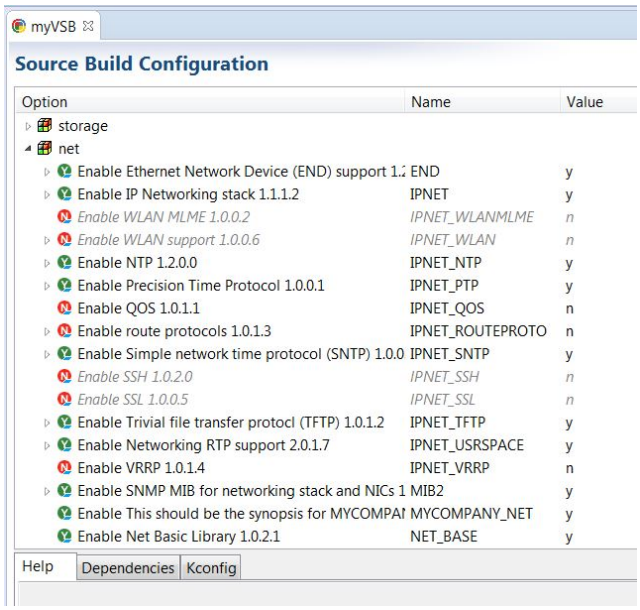

- Workstation Filesystem

  The layer project files and directories are located in the correct place on the workstation filesystem and may be browsed from the command line or explorer. The VSB build system will find the layer if it is located within *installDir* **/vxworks-7/pkgs_v2/**.




- VxWorks Source Build Configurator

  If we now create a VSB project, and then open the **Source Build Configuration** tool, we will see **MYCOMPANY** listed as one of the layers that may be included into our VSB.




> **Note:** Even though our new layer is empty and does not do anything, it is still a valid layer and so may be included into a VSB and then built successfully.

5. Explore the layer files in Workbench.
   1. Examine the top level layer files.

      A directory structure, **layer.vsbl** configuration metadata file, and a **Makefile** are created. See Layer Metadata File on page 12 and Layer Makefile on page 17 for a detailed description of these files.

   2. Examine the directories.
      - **/cdf** must contain any cdf files required by a VIP project containing this layer.
      - **/cfg** must contain any configlette files required by a VIP project containing this layer.
      - **/h** must only contain public header files used during kernel library build.
      - **/src** must contain source files and private header files used during kernel library build.

      The Workbench layer project is created with a minimal, example layer directory structure. If the user does not have any need for one or more of the default directories, then they may be deleted. Ensure that the reference to every deleted directory in the layer Makefile is also removed or commented out.

6. (Optional step). Make kernel space header files public.
   1. Copy any headers that must have a public scope, and so must be visible to the VIP and other layers, to a directory called **h**.

      Public header files are often used to declare software API functions that are visible to the rest of the system.

      The path of each header file must be unique. If you cannot rename headers for uniqueness, create a subdirectory under your layer's **h** directory and move the header files in there. Alternatively, consider prefixing your company name to the header files to ensure uniqueness.

   2. Un-comment the following line in the layer **Makefile**:

      ```
      KERNEL_PUBLIC_H_DIRS = h
      ```

7. (Optional step). Enable local CDF support

1. Move any **CDF** files associated with this library to a directory called **cdf** found under the root of your layer.
2. Uncomment the following line in the layer **Makefile**:

```
POST_NOBUILD_CDFDIRS = cdf
```

8. (Optional step). Enable configlettes support.
   1. Move any configlettes associated with this library to a directory called **cfg** found under the root of your layer.
   2. Uncomment the following line in the layer **Makefile**:

```
POST_NOBUILD_CFGDIRS = cfg
```

9. Create any additional layer directories that are required.
   1. Decide which additional directories your layer requires.

   Additional directories allow the layer to build its components at different stages of the overall VSB project build.

   You can create these directories either by using Wind River Workbench and the context menu within your workspace, or you may create them directly from the command line. If you create any files or folders this way, then Workbench must be restarted in order for its workspace to be updated so that it reflects your changes.

   Each additional directory has a related makefile macro. For a more detailed list of these macros and directories, and a discussion of the sequential nature of VSB project compilation, refer to Layer Makefile on page 17. The additional directories are:

   - **/genh** is where source files and headers files are generated for kernel and user space.

     Added layer makefile line:

```
PRE_BUILD_GENH_DIRS = genh
```

   - **/pre_src** is where tools are build.

     Added layer makefile line:

```
PRE_BUILD_DIRS = pre_src
```

   - **/dkm** contains source and private headers that compile to Downloadable Kernel Module (DKM).

     Added layer makefile line:

```
POSTBUILD_DKM_DIRS = dkm
```

   - **/usr_src** contains source and private headers that compile to user mode libraries.

     Added layer makefile line:

```
BUILD_USER_DIRS = usr_src
```

   - **/usr_lib** is for source and private headers that must be built after kernel and user libraries are built.

     Added layer makefile line:

```
POSTBUILD_USRLIB_DIRS = usr_lib
```

   - **/rtp** contains source and private headers that compile to a Real Time Process (RTP) vxe file.

Added layer makefile line:

```
POSTBUILD_RTP_DIRS = rtp
```

- **/usr_h** must contain header files that are public in user space.

  Added layer makefile line:

  ```
  USER_PUBLIC_H_DIRS = usr_h
  ```

- **/share_h** must contain header files that are public in both kernel and user space.

  Added layer makefile line:

  ```
  SHARED_PUBLIC_H_DIRS = share_h
  ```

2. Create all the layer sub-directories you need and give them the correct name.
3. Add the appropriate macro lines into the layer makefile.

You must manually add new macro definitions to your layer makefile; your layer makefile does not specify these additional macros by default, you need to add them.

10. Migrate your code bases into your layer directories.

    Each of your source directories, for example **src** and **usr_src**, must contain their own makefiles. The function of these makefiles is to compile the source files contained inside that directory. But source makefiles will also need to reference some layer rules and macros and variables.

    For guidance on the construction of source makefiles, refer to Layer Makefile on page 17 and Layer Makefile Rules Reference on page 96.

11. Build and test your layer.

    Refer to Preparing to Test a Layer on page 63 for more information.

# 4. Creating a Layer Project Using the Command Line

To create a layer from the command line involves using the wrtool utility to create a manage the project.

## About This Task

## Before You Begin

Before you begin, you must have selected a location for your layer and created a directory there.

## About This Task

In this example the layer to be created will reside within *installDir* **/vxworks-7/pkgs_v2/custom/myLayer**, which has already been created.

## Procedure

1. Setup the command shell.

```
$ cd installDir
$ wrenv -p vxworks-7
$ cd vxworks-7/pkgs_v2
$ wrtool -data custom
$ cd custom
```

2. Create the layer project.

```
$ prj layer create -force -name myLayer myLayer
$ cd myLayer
```

3. Explore the layer files.
   1. Examine the top level layer files.

   A directory structure, **layer.vsbl** configuration metadata file, and a **Makefile** are created. Open these files either from the wrtool command line or your favourite editor.

   See Layer Metadata File on page 12 and Layer Makefile on page 17 for a detailed description of these files.

```
myLayer> cat Makefile
## Makefile for Project MYLAYER


# Modify the following settings to change the header directories that get copied into the VS
B
# KERNEL_PUBLIC_H_DIRS Files in these directories are copied into the VSB public kernel head
er directory; preferred directory name is h
KERNEL_PUBLIC_H_DIRS = h

# Modify the following settings to change the header files that get copied into the VSB
# KERNEL_PUBLIC_H_FILES These files are copied into the VSB public kernel header directory
# KERNEL_PUBLIC_H_FILES =



# Modify the following settings if your layer is being used to generate kernel libraries in
the VSB
# BUILD_DIRS Directories for Kernel library source; preferred directory name is src
BUILD_DIRS = src

# BUILD_TOOLS Kernel compile tools
# BUILD_TOOLS =                            -primary compiler this is the default
# BUILD_TOOLS = $(_WRS_CONFIG_ALL_TOOLS)    -Both VSB compilers LIB_COMMON = false must be s
et in src Makefile
# BUILD_TOOLS = $(_WRS_CONFIG_gnu_VERSION)  -gnu only, add VSB_REQUIRES gnu_SUPPORTED to lay
er.vsbl
# BUILD_TOOLS = $(_WRS_CONFIG_llvm_VERSION)  -llvm only, add VSB_REQUIRES llvm_SUPPORTED to
layer.vsbl
# BUILD_TOOLS =


# Modify the following settings if your layer has cdf files to be delivered in the VSB
```

```
# POST_NOBUILD_CDFDIRS Post-no-build CDF directories; preferred directory name is cdf
POST_NOBUILD_CDFDIRS = cdf

# Modify the following settings if your layer has configlettes to be delivered in the VSB
# POST_NOBUILD_CFGDIRS Post-no-build config directories; preferred directory name is cfg
POST_NOBUILD_CFGDIRS = cfg


include $(WIND_KRNL_MK)/rules.layers.mk
myLayer>
```

2. Examine the directories.
   - **/cdf** must contain any cdf files required by a VIP project containing this layer.
   - **/cfg** must contain any configlette files required by a VIP project containing this layer.
   - **/h** must only contain public header files used during kernel library build.
   - **/src** must contain source files and private header files used during kernel library build.

The Workbench layer project is created with a minimal, example layer directory structure. If the user does not have any need for one or more of the default directories, then they may be deleted. Ensure that the reference to every deleted directory in the layer Makefile is also removed or commented out.

4. (Optional step). Make kernel space header files public.
   1. Copy any headers that must have a public scope, and so must be visible to the VIP and other layers, to a directory called **h**.

   Public header files are often used to declare software API functions that are visible to the rest of the system.

   The path of each header file must be unique. If you cannot rename headers for uniqueness, create a subdirectory under your layer's **h** directory and move the header files in there. Alternatively, consider prefixing your company name to the header files to ensure uniqueness.

   2. Un-comment the following line in the layer **Makefile**:

```
KERNEL_PUBLIC_H_DIRS = h
```

5. (Optional step). Enable local CDF support
   1. Move any **CDF** files associated with this library to a directory called **cdf** found under the root of your layer.
   2. Uncomment the following line in the layer **Makefile**:

```
POST_NOBUILD_CDFDIRS = cdf
```

6. (Optional step). Enable configlettes support.
   1. Move any configlettes associated with this library to a directory called **cfg** found under the root of your layer.
   2. Uncomment the following line in the layer **Makefile**:

```
POST_NOBUILD_CFGDIRS = cfg
```

7. Create any additional layer directories that are required.
   1. Decide which additional directories your layer requires.

   Additional directories allow the layer to build its components at different stages of the overall VSB project build.

   You can create these directories either by using Wind River Workbench and the context menu within your workspace, or you may create them directly from the command line. If you create any files or folders this way, then Workbench must be restarted in order for its workspace to be updated so that it reflects your changes.

Each additional directory has a related makefile macro. For a more detailed list of these macros and directories, and a discussion of the sequential nature of VSB project compilation, refer to Layer Makefile on page 17. The additional directories are:

- **/genh** is where source files and headers files are generated for kernel and user space.

  Added layer makefile line:

  ```
  PRE_BUILD_GENH_DIRS = genh
  ```

- **/pre_src** is where tools are build.

  Added layer makefile line:

  ```
  PRE_BUILD_DIRS = pre_src
  ```

- **/dkm** contains source and private headers that compile to Downloadable Kernel Module (DKM).

  Added layer makefile line:

  ```
  POSTBUILD_DKM_DIRS = dkm
  ```

- **/usr_src** contains source and private headers that compile to user mode libraries.

  Added layer makefile line:

  ```
  BUILD_USER_DIRS = usr_src
  ```

- **/usr_lib** is for source and private headers that must be built after kernel and user libraries are built.

  Added layer makefile line:

  ```
  POSTBUILD_USRLIB_DIRS = usr_lib
  ```

- **/rtp** contains source and private headers that compile to a Real Time Process (RTP) vxe file.

  Added layer makefile line:

  ```
  POSTBUILD_RTP_DIRS = rtp
  ```

- **/usr_h** must contain header files that are public in user space.

  Added layer makefile line:

  ```
  USER_PUBLIC_H_DIRS = usr_h
  ```

- **/share_h** must contain header files that are public in both kernel and user space.

  Added layer makefile line:

  ```
  SHARED_PUBLIC_H_DIRS = share_h
  ```

2. Create all the layer sub-directories you need and give them the correct name.
3. Add the appropriate macro lines into the layer makefile.

You must manually add new macro definitions to your layer makefile; your layer makefile does not specify these additional macros by default, you need to add them.

8. Migrate your code bases into your layer directories.

   Each of your source directories, for example **src** and **usr_src**, must contain their own makefiles. The function of these makefiles is to compile the source files contained inside that directory. But source makefiles will also need to reference some layer rules and macros and variables.

   For guidance on the construction of source makefiles, refer to Layer Makefile on page 17 and Layer Makefile Rules Reference on page 96.

9. Build and test your layer.

   Refer to Preparing to Test a Layer on page 63 for more information.

# 5. The Effect of Layer Creation on VSB Projects

Which layers are enabled by default in a VSB project configuration is determined by which project is created first and by how the layer's **DEFAULT** metadata value is set.

## Introduction

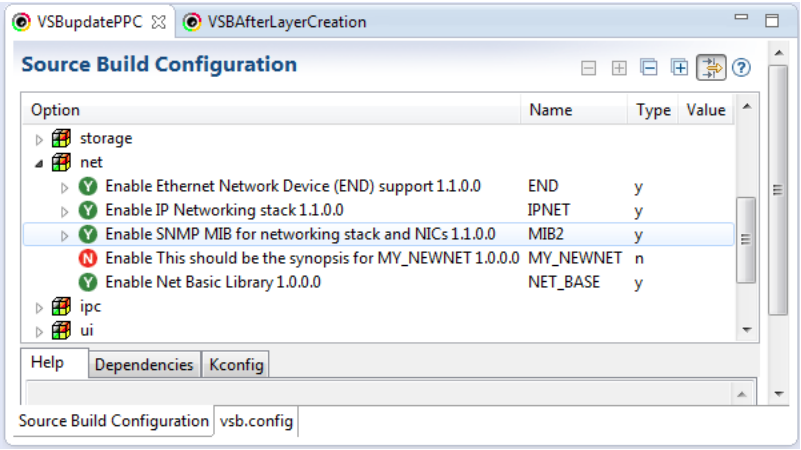
Both layers and VSB projects are either created using Workbench on page 38 or Command Line on page 44. For more detailed information refer to *VxWorks 7 Configuration and Build Guide*. Once a new layer has been created, and is visible to the VSB build system, it will appear as an option in all VSB configurations.

## VSB Projects that Pre-date New Layers

When we open a VSB project and view the configuration, it shows all the VxWorks layers currently available in the Workbench layers path. Assume that we have a VSB project that pre-dates a new layer project. Previously, before the new layer project existed, the VSB project configuration tool did not show this layer within its configuration because the layer didn't exist. Once the new layer exists it appears in the VSB project configurations as an option.

The newly created layer is disabled by default in the VSB source configuration when we open it. This will be the case even if the **DEFAULT** metadata element in **layer.vsbl** is set to **YES**. The layer is disabled, because the VSB project pre-dates the new layer, and so has functioned successfully without this layer up to this point. If you want the new layer enabled in the VSB project, you must manually add it.

For example, the image below shows the source configuration of a VSB where the VSB pre-dates the creation of the layer:

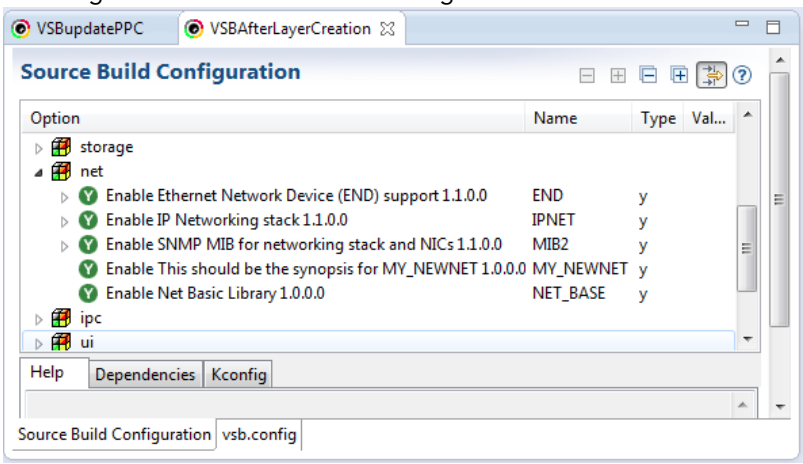



## Layers that Pre-date New VSB Projects

Assume now that it is the layer project that pre-dates a new VSB project. Will the layer be enabled by default in your new VSB project configuration? The answer is - it depends.

Once the VSB project is created, open the VSB configuration tool. Only the layers with a **DEFAULT** metadata element in **layer.vsbl** which is set to **YES** will be enabled by default in all VSB projects that are subsequently created after the layer.

The image below shows the source configuration of a VSB where the VSB was created after the new layer was added.




## Purpose of the DEFAULT Metadata Element

In every layer there is a **layer.vsbl** file, and in this file you will find the following line. Some layers will set it to **YES**, for other layers it will be set to **NO**.

```
DEFAULT             YES
```

The purpose of the DEFAULT metadata element is to control whether or not this layer is automatically part of every new VSB project that is created or not. If it is not, then the user has to manually include this layer into every new VSB project they create. If it is then the VSB will include the layer by default. The layer can be removed manually by the user if required.

When you are developing a custom layer, you must decide whether or not your layer should be enabled by default. If in doubt, set **DEFAULT** to **NO** in **layer.vsbl**.

```
DEFAULT             NO
```

Refer to Layer Metadata Elements Reference on page 98 for further details about the metadata elements in **layer.vsbl**.

# 6. Using Layers at a Custom Location

To create a layer outside of the *installDir* **/vxworks-7/pkgs_v2** directory, also known as the outside layer, the environment variable *WIND_LAYER_PATHS* must include the location of the layer.

## About This Task

For a detailed outside layers tutorial, refer to *VxWorks 7 Layer Version Management Tutorial*.

In the following example, the layer is found at **/Xyz/myLayer**.

## Procedure

1. Create or copy the layer to **/Xyz/myLayer**.

   The **layer.vsbl** file should exist in **/Xyz/myLayer/layer.vsbl**. Provide a unique name for the layer in **layer.vsbl** for example *XYZ_MY_LAYER_NAME*, unless it is a new version of an existing layer.

2. Add the path created in step 1 on page 50 to *WIND_LAYER_PATHS*. One of the methods of adding the path on Windows is as follows :
   1. Go to **Control Panel > System**.
   2. In the left pane, click on **Advanced system settings**.
   3. On the Advanced pane, click **Environment Variables**.
   4. In the **System Variables** area, add a new variable called *WIND_LAYER_PATHS*.
   5. Set this new variable to your custom path. Click OK. Save and exit as appropriate.

   Note: If *WIND_LAYER_PATHS* exists, and already contains a path, append a semicolon to the pre-existing path and insert the new custom path after the semicolon for Windows systems. For Linux systems, use colon as path separators. Do not leave space before the new custom path.

   Note: You may need to reboot your host workstation in order for the *WIND_LAYER_PATHS* environment variable change to take place.

   Now when you use Workbench or command line the environment variable *WIND_LAYER_PATHS* will include this custom path and the path that already existed in the **package.properties**.

   Note: **package.properties** also sets the other environment variables, for example it sets the compiler paths and BSP paths as shown below:

   ```
   WIND_BSP_PATHS=D:\NoScan\vxworks\vxworks-7\pkgs_v2\os\board
   WIND_COMPILER_PATHS=D:\NoScan\vxworks\compilers
   ```

   If the user wants to use alternate compilers or BSPs, they can be installed outside of the VxWorks installation. The environment variable for the compilers and the BSPs can be modified in the same way as the *WIND_LAYER_PATHS* shown above.



WNDRVR

If the user is on linux host OS, use the **.login** script to establish the custom path for the environment variables and wrenv will modify it and add it to the **package.properties** paths as required.

3. Verify that the layer can be found.

   Run wrenv and search the output listing for your new layer directory.

```
installDir> wrenv -p vxworks-7
...
WIND_LAYER_PATHS=c:\Xyz;c:\windriver\vxworks-7\pkgs_v2
...
```

4. Verify that the layer is available to your development tools.

   You have two choices:

   - Open Workbench and create a **Source Build Configuration for a VSB project**. The new layer should be available for selection.
   - From a command line, navigate to a Source Build project and run wrtool. From the wrtool shell, enter a command similar to the example below.

```
wrtool> prj vsb info XYZ_MY_LAYER_NAME
Name:                       XYZ_MY_LAYER_NAME
VERSION:                    1.0.0.0
TYPE:                       GENERIC
...
```

# 7. Creating Alternate Versions of Layers

You can create an alternate version of an existing layer, for example to implement your own versions of APIs, and then have both versions available as options for VSB configuration.

## About This Task

The method described here can be used if you want to implement alternate versions of layer code for Wind River layers or for custom layers. If you want to add or subtract code modules, you must also make changes to the makefile system (which is not described here).

**CAUTION:** **Modifying code used by the VxWorks operating system should only be done with extreme care. Modification or replacement of Wind River code may result in unpredictable behavior.**

For a detailed tutorial on managing layer versions, refer to *VxWorks 7 Layer Version Management Tutorial*.

## Procedure

1. Copy and rename the layer directory, so that the copy is parallel to the original layer directory.

   For example, if you want to implement your own versions of standard C library APIs, copy and rename **pkgs_v2/os/lang_lib/libc/libc_std_1_0_0_0** to **pkgs_v2/os/lang_lib/libc/libc_std_1_0_0_0_custom** (the name you use for the alternate-version directory is up to you).

2. Edit the **layer.vsbl** metadata file in the new directory to give it meaningful version and descriptive information.

   Edit the SYNOPSIS, HELP, and VERSION elements. For example:

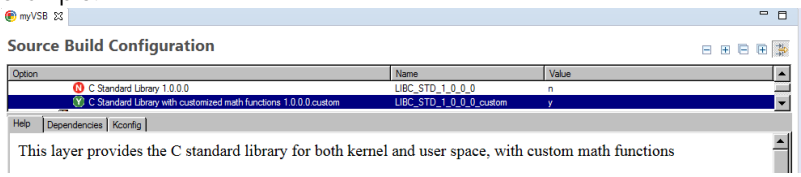
```
Layer LIBC_STD
    {
    SYNOPSIS        C Standard Library with customized math functions

    HELP                This layer provides the C standard library for both kernel and user spa
ce, with custom math functions
    VERSION         1.0.0.0.custom
    VENDOR          Wind River - customized by me

    FEATURE         COREOS
    OPTIONAL        NO
    DEFAULT         YES
    DOC_BUILD       HTML
    }
```

3. Modify the layer API code that you need to change in the layer's **src** subdirectory.

   For example, replace Wind River math function code such as **sqrt.c** with your own.

4. Verify that the new layer is available to the VSB project configuration tools.

   - For Workbench use, create a VSB project, and open the Source Build Configuration dialog, and search on your layer name. For example:




   - For wrtool, use the command wrtool prj vsb info *layerName* to display information about the original layer and your alternate version.

### Results

You may now use your alternate version of the layer in a VSB project.

# 8. Updating Layers

After you have created and built a VSB, you may at some point install layer updates into your system, making the older versions of the layer out of date.

### About This Task

It is useful to know what new layers are available after a VSB has been built so you can perform informed updates to your project.

## Before You Begin

This procedure assumes that an initial version of the layer exists, has been included in the VSB and can be found by development tools such as Workbench and wrtool.

## Procedure

1. List new layer versions.

   The following command-line example using wrtool utility lists new versions of an enabled layer, **ESL_1_0_0_0**. Use the command prj vsb listNew to show all the new layers that are available based on layers that have already been enabled in the VSB

```
wrtool> prj vsb listNew

Newer versions available for this VSB:
ESL_1_0_0_0
    ESL_1_0_10_0
```

2. Select the new version of the layer in your VSB.

   The following command-line example adds the new layer ESL_1_0_10_0 to the VSB.

```
wrtool> prj vsb add ESL_1_0_10_0

Now adding:
ESL_1_0_10_0

done
VSB configuration changed: override: reassigning to symbol ESL_1_0_10_0
VSB configuration changed: override: ESL_1_0_10_0 changes choice state

This VSB supports the following BSPs: itl_64_1_0_0_0
VSB configuration has changed.
On the next build the existing VSB library will be rebuilt.
```

3. Alternatively a single command can be used to add the latest versions of all the enabled layers to your VSB.

   The following command-line example adds the new layer ESL_1_0_10_0 to the VSB.

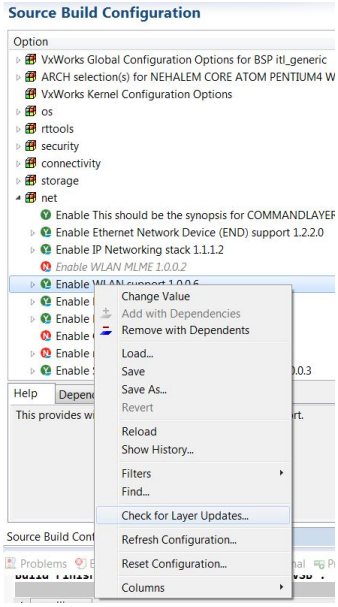
```
wrtool> prj vsb addNew

Now adding:
ESL_1_0_10_0

done
VSB configuration changed: override: reassigning to symbol ESL_1_0_10_0
VSB configuration changed: override: ESL_1_0_10_0 changes choice state

This VSB supports the following BSPs: itl_64_1_0_0_0
VSB configuration has changed.
On the next build the existing VSB library will be rebuilt.
```

   Note: This functionality will not bring in any new layers to the VSB configuration, unless the new version of an enabled layer has a dependency on a new layer.



WNDRVR

Note: Within the Workbench **Source Build Configuration** tool, you can also check for updates to existing layers and easily add them to the VSB. This assumes that all available layers have already been downloaded using the **Product Maintenance** tool first.




# 9. VxWorks 7 Subprojects

Subproject feature allows VxWorks source layers to provide source code that is added and compiled into a VIP, or configuration files that are packaged into the VIP.

The subproject directories and source files are copied from the layer containing the subproject into the VIP directory and can be modified by the user for configuration or customization as required. A subproject can be added and removed from a VIP just like a kernel configuration component.

## Common Use Cases for Subprojects

- **Configuration Files**

  You can add configuration file templates to a VIP project by creating a subproject in a layer. These configuration files can be modified by the user and be used by the software at runtime to define parameters. The subproject mechanism allows the configuration files to be copied from the VSB into the VIP at build time.
- **Customizing Code for a VIP**

  A VSB layer may need to compile some code based on VIP or BSP configuration options and link back into the final VIP image. Code added by a subproject in this manner can incorporate VIP specific config files (for example **prjParams.h**) to customize aspects of the subproject's code, behavior, or VIP configuration. In this use case the subproject copies code into the VIP which is built during the VIP build process and links the resulting objects into the final VIP image.
- **Adding Prebuilt Objects to a VIP**

  A VSB binary layer may contribute prebuilt objects to a VIP by using a subproject instead of relying on normal VSB library linkages. This use case can be appropriate for third party code that we wish to link into the final VIP image.
- **Adding Other Builds to a VIP**

  A VSB layer may provide a standalone binary or other element that can be compiled separately from the VIP, but may depend on VIP specific configuration. This use case allows the code to be copied from the layer into the VIP and gets built during the

VIP build but the resulting artifacts are not linked to the VIP. For example, this use case is used by the boot loader layer to build a secure bootloader that works with the VIP.

## Setting up a Basic Subproject

To setup a subproject in a layer, the layer must provide a **.cdf** configuration file. The **.cdf** file must contain a subproject definition block which includes defining a directory containing the subproject source files. A component which **REQUIRES** the defined subproject must be provided. This component will control when the subproject needs to be included and any other dependencies on other CDF components. When the component is included in the VIP, these definitions will allow the files from the subproject to be copied into the VIP.

For example, the layer would include the following file structure:

- *layerDir* **/layer.vsbl**

  The file **layer.vsbl** is the layer definition file.
- *layerDir* **/subprojects/exampleSubproject**

  The **exampleSubproject** directory contains one subproject's source files, makefiles, make fragments (**defs.sub.mk**), and any other files that need to be copied to the VIP. The directory tree under **exampleSubproject** may follow any structure the developer requires as long as it is self-contained. The subproject source files may **#include** the VIP configuration files or other VIP accessible configuration.
- *layerDir* **/cdf/01testLayer.cdf**

  The **01testLayer.cdf** file contains the subproject and component definitions. The subproject definition will define the **NAME** of the subproject used in **defs.sub.mk**. The subproject **DIRECTORY** definition will refer to the **exampleSubproject** directory in the layer containing the subproject.

## Connecting a Subproject to a VIP Build

If the subproject layer contains customizing source code, the subproject directory must contain the file **defs.sub.mk** which will allow the VIP to build the subproject. The VIP build will use this **defs.sub.mk** file with the build rules to the build the subproject.

The **defs.sub.mk** file must define the following:

- either a target to build or object file(s) that must be added to the VIP.
- rules to build the specified target or object(s).
- a clean rule to remove the subproject objects.

Note: The subproject's clean rule must be added to the common clean rule as a dependency.

## Variables & Rules in defs.sub.mk

The CDF definition for a subproject *<name>* must be used in the definition of values that are specified in the **defs.sub.mk** variables and rules.

If the subproject is providing an object to be linked to the VIP, then the **defs.sub.mk** must:

- define a variable *<name>* **_OBJS** where *<name>* corresponds to the name of the subproject specified in the CDF **SubProject** definition block.

  For example, if the name specified in the **.cdf** file **SubProject** definition block is **SUBPROJECT_CODE_EXAMPLE** then objects should be assigned to the variable **SUBPROJECT_CODE_EXAMPLE_OBJS**.
- define a make rule to create the objects specified in the *<name>* **_OBJS** variable and locate them in the VIP project directory with other objects for linking.

> **Note:** One or more rules may be used to accomplish this as long as proper make dependencies are used to trigger rule execution.

- define a clean rule specified in **clean**_*<name>* for the subproject objects.

  The subproject's clean rule should be added to the common clean rule as a dependency. For example, if the name of the subproject layer is **SUBPROJECT_CODE_EXAMPLE** then the clean rule should be **clean_SUBPROJECT_CODE_EXAMPLE**.

If the subproject is providing a target to be built, then the **defs.sub.mk** is required to:

- provide a single make rule to build the subproject targets appended to **SUBPROJECT_TARGETS** specified as **subproject**_*<name>*.

  For example, if the name of the subproject layer is **SUBPROJECT_CODE_EXAMPLE** then the target rule should be **subproject_SUBPROJECT_CODE_EXAMPLE**.
- append the target to be built to the variable **SUBPROJECT_TARGETS**.
- define a clean rule specified in **clean**_*<name>* for the subproject objects.

  The subproject's clean rule should be added to the common clean rule as a dependency. For example, if the name of the subproject layer is **SUBPROJECT_CODE_EXAMPLE** then the clean rule should be **clean_SUBPROJECT_CODE_EXAMPLE**.

### Adding a Subproject to a VIP

A subproject can be added to an existing VIP just like a component. Like the **Component** class object, **SubProject** class objects can be added explicitly or indirectly as dependencies when required by other components. When the VIP builds:

- subproject directories and their contents will be copied from the VSB to the VIP.
- the subproject files will be built if there are targets or objects defined in the sublayer's **defs.sub.mk** file.
- if proper dependencies are specified in **defs.sub.mk**, the subproject clean target will run if the VIP build clean command is run

### Updating the Layer Makefile

You must specify the subproject directory in the layer makefile. For the directory *layerDir* **/subprojects**, add the following line to *layerDir* **/Makefile**:

```
KERNEL_SUBPROJECTS = subprojects
```

### Removing a Subproject from a VIP

A subproject can be removed from an existing VIP just like a component. Like the Component class object, SubProject class objects can be removed explicitly or indirectly as dependencies when required by other components. Removing the subproject from the VIP configuration will not delete the subproject files from the VIP immediately. Instead the next time the VIP is built, a clean build of the VIP occurs and this will execute the clean rule provided by the subproject. Based on the clean rule provided by the subproject, the build artifacts generated by the subproject may be deleted.

> **Note:** Subproject source files including any user modifications will not be removed from the VIP. Even though the source files are in the VIP, they will not be built into the VIP.

# 10. Creating a Layer Subproject Containing Customizing Code

## Before You Begin

These steps assume you have previously created a layer project. For details, refer to Creating a Layer Project Using the Command Line on page 44.

## About This Task

In this example, we will create a subproject in a source layer containing a source file. This code is intended to be linked into a VxWorks image project (VIP) and modifies it in some way. You will then create a VIP and include the layer subproject into it. The subproject will be called **SUBPROJECT_CODE_EXAMPLE** and it will contain one source file **hello.c**.

The workstation command shell has been configured as a VxWorks wrtool development shell:

```
$ wrtool -data installDir\workspace
installDir> cd workspace
```

## Procedure

1. Create a subproject directory and configuration file directory structure as follows:
   - create *layerDir* **/subprojects**
   - create *layerDir* **/subprojects/code_etc**
2. Add a source file to the *layerDir* **/subprojects/code_etc** directory.

   *layerDir* **/subprojects/code_etc/hello.c**:

   ```
   #include "vxworks.h"
   #include "stdio.h"

   void hello_world()
   {
       printf("Hello world\n");
   }
   ```

   Note: The subproject directory can contain any number of source files written in C, C++ or assembly language. The subproject can also contain other files. The files will be simply copied into the VIP when the subproject is included into a VIP.

   A single layer can contain multiple subproject directories if there is a need to separate the subprojects in groups. For example, a layer containing two subprojects in a single subproject directory would look like this:

   *layerDir* **/**<*layer subproject directory*>**/**<*subproject 1 directory*>**/**<*subproject 1 files*>

   *layerDir* **/**<*layer subproject directory*>**/**<*subproject 2 directory*>**/**<*subproject 2 files*>

   Both <*subproject 1 directory*> and <*subproject 2 directory*> must contain an appropriate **defs.sub.mk** file.

3. Create the **defs.sub.mk** file in the *layerDir* **/subprojects/code_etc** directory.

   The file **defs.sub.mk** must exist in a subproject containing code. It specifies a make fragment that lists the object files to be built within the VIP. It also specifies the clean rules for the subproject.

   *layerDir* **/subprojects/code_etc/defs.sub.mk**:

   ```
   SUBPROJECT_CODE_EXAMPLE_OBJS = hello.o
   ```

```
clean: clean_SUBPROJECT_CODE_EXAMPLE
clean_SUBPROJECT_CODE_EXAMPLE: rm -fr *.o
```

Note: If multiple object files will be built, specify them on the top line. Separate each file name with a space.

4. Create a CDF file **02comp_code.cdf** in the *layerDir* **/cdf** directory.

   The subproject must be defined as a required component for the layer component. To ensure the subproject is added to the VIP, its location must be specified by use of the **_CHILDREN** macro. In this example the VIP component folder **FOLDER_BSP_CONFIG** is used. Other VIP folders can be used.

   *layerDir* **/cdf/02comp_code.cdf**:

```
SubProject SUBPROJECT_CODE_EXAMPLE {
    NAME        my subproject example
    SYNOPSIS    This is my subproject example
    DIRECTORY   code_etc
    MODULES     hello.o
    _CHILDREN   FOLDER_BSP_CONFIG
}

Component INCLUDE_CODE_COMMON {
    NAME        Example code common configuration
    SYNOPSIS    Includes common config configuration
    REQUIRES    SUBPROJECT_CODE_EXAMPLE
    CONFIGLETTES
    CFG_PARAMS
}
```

5. Add a reference to the subproject to the layer **Makefile**.

   Add the following to *layerDir* **/Makefile**:

```
KERNEL_SUBPROJECTS = subprojects
POST_NOBUILD_CDFDIRS = cdf
```

6. Create and build a VxWorks source build (VSB) project in the workspace directory.
   1. Create the VSB project.

```
workspace> prj vsb create -bsp vxsim_windows VxSimVSB -S
workspace> cd VxSimVSB
```

   2. Build the VSB project.

```
VxSimVSB> prj build
VxSimVSB> cd ..
```

7. Create a VxWorks image project (VIP) in the workspace directory.

```
workspace> prj vip create -vsb VxSimVSB vxsim_windows llvm VxSimVIP
workspace> cd VxSimVIP
```

8. Add the subproject component to the VIP and build it:

   A subproject can be added to an existing VIP just like a component. Add **INCLUDE_CODE_COMMON** to the VIP. From the command line, type the following:

1. Add the component from the command line:

```
VxSimVIP> prj vip component add INCLUDE_CODE_COMMON
```

> **Note:** When a subproject is added to a VIP, the subproject files are not copied immediately into the VIP. Instead it will be copied into the VIP when the project is built. In the current implementation, only one instance of a given subproject can be added to a VIP.

2. Build the VIP:

```
VxSimVIP> prj vip build
```

9. Verify that the subproject component **INCLUDE_CODE_COMMON** is included in the VIP.

```
VxSimVIP> prj vip component list
DRV_BUS_FDT_ROOT
DRV_BUS_FDT_SIMPLE
...
INCLUDE_CODE_COMMON
...
SUBPROJECT_CODE_EXAMPLE
VxSimVIP>
```

> **Note:** Both the component **INCLUDE_CODE_COMMON** and the subproject **SUBPROJECT_CODE_EXAMPLE** are shown. This verifies they are included in the VIP. However, these components do not appear in the graphical Workbench VIP Configuration tool.

10. Locate the compiled subproject object file in the VIP.

```
VxSimVip> cd default
default> ls
...
hello.d
hello.o
...
default>
```

> **Note:** The presence of hello.o verifies the source file was successfully compiled and linked at VIP build time.

## Next Steps

The subproject is removed from the VIP by removing the component:

```
VxSimVIP> prj vip component remove INCLUDE_CODE_COMMON
VxSimVIP> prj vip build
```

When removed from the VIP, the subproject content is not deleted from the layer directory. The subproject can be added into another VIP.



WNDRVR

Note: If a new version of a layer is installed and added to the VSB, a new layer version of the subproject will not be copied automatically into the VIP. The user must remove the subproject directory manually from the VIP (all user modifications will be lost at this point). Then the build system will automatically re-install the new layer version of the subproject when the VIP is built again.

Also after a subproject is removed, the next VIP build will clean the subproject directory by executing the clean_*<subproject>* target to remove any of the objects built.

# 11. Creating a Layer Subproject Containing Configuration Files

## Before You Begin

These steps assume you have previously created a layer project. For details, refer to Creating a Layer Project Using the Command Line on page 44.

## About This Task

In this example, you will create a subproject in a source layer. You will then create a VxWorks image project (VIP) and include the layer subproject into it. The subproject will be called **SUBPROJECT_CFGFILE_EXAMPLE** and contain one configuration file **info.cfg**.

The workstation command shell has been configured as a VxWorks wrtool development shell:

```
$ wrtool -data installDir\workspace
installDir> cd workspace
```

## Procedure

1. Create a subproject directory and configuration file directory structure as follows:
   - create *layerDir* **/subprojects**
   - create *layerDir* **/subprojects/cfg_etc**
2. Add the text config file to the directory *layerDir* **/subprojects/cfg_etc** directory.

   *layerDir* **/subprojects/cfg_etc/info.cfg**

   The file **info.cfg** is a text file that can containing anything. The subproject can contain any number of files. The files will be simply copied into the VIP when the subproject is included into a VIP.

   Note: All source files for a given subproject must be located in a single directory within the source layer. The entire directory and its contents will be copied into the VIP when the subproject is added to the VIP. A single layer can contain multiple subproject directories if there is a need to separate the subprojects in groups.

   A single layer can contain multiple subproject directories if there is a need to separate the subprojects in groups. For example, a layer containing two subprojects in a single subproject directory would look like this:

   *layerDir* **/***<layer subproject directory>* **/***<subproject 1 directory>* **/***<subproject 1 files>*

   *layerDir* **/***<layer subproject directory>* **/***<subproject 2 directory>* **/***<subproject 2 files>*

3. Create a CDF file **02comp_cfg.cdf** in the *layerDir* **/cdf** directory.

   *layerDir* **/cdf/02comp_cfg.cdf**:



WNDRVR

The subproject object must be described in a CDF file. The contents of the file **02comp_cfg.cdf** is shown below:

```
SubProject SUBPROJECT_CFGFILE_EXAMPLE {
            NAME my subproject myexample
            SYNOPSIS This is my subproject example
            DIRECTORY cfg_etc
            _CHILDREN FOLDER_BSP_CONFIG
}

Component INCLUDE_CFGFILE_COMMON {
            NAME config common configuration
            SYNOPSIS Includes common config file configuration
            REQUIRES SUBPROJECT_CFGFILE_EXAMPLE
            CONFIGLETTES
            CFG_PARAMS
}
```

Note: A subproject must be located in the VIP component folder structure. In this example, **FOLDER_BSP_CONFIG** has been chosen to contain the component. Other VIP folders can be used.

4. Add a reference to the subproject in the layer **Makefile**.

   The layer subproject directory you created earlier must be added to the layer **Makefile**:

```
KERNEL_SUBPROJECTS = subprojects
POST_NOBUILD_CDFDIRS = cdf
```

5. Create and build a VxWorks source build (VSB) project in the workspace directory.
   1. Create the VSB project.

```
workspace> prj vsb create -bsp vxsim_windows VxSimVSB -S
workspace> cd VxSimVSB
```

   2. Build the VSB project.

```
VxSimVSB> prj build
VxSimVSB> cd ..
```

6. Create a VxWorks image project (VIP) in the workspace directory.

```
workspace> prj vip create -vsb VxSimVSB vxsim_windows llvm VxSimVIP
```

7. Add the subproject to the VIP.

   A subproject can be added to an existing VIP just like a component.

```
workspace> cd VxSimVIP
VxSimVIP> prj vip component add INCLUDE_CFGFILE_COMMON
```

Note: When a subproject is added to a VIP, the subproject files are not copied immediately into the VIP. Instead it will be copied into the VIP only when it is built. In the current implementation, only one instance of a given subproject can be added to a VIP.

8. Verify the component **INCLUDE_CFGFILE_COMMON** is included in the VIP.

```
VxSimVIP> prj vip component list
DRV_BUS_FDT_ROOT
DRV_BUS_FDT_SIMPLE
...
INCLUDE_CFGFILE_COMMON
...
SUBPROJECT_CFGFILE_EXAMPLE
VxSimVIP>
```

Note: Both the component **INCLUDE_CFGFILE_COMMON** and the subproject **SUBPROJECT_CFGFILE_EXAMPLE** are shown. This verifies they are included in the VIP. However, these components do not appear in the graphical Workbench VIP Configuration tool.

9. Build the VIP.

```
VxSimVIP> prj build
```

10. Verify the location of the subproject configuration file in the VIP.

```
VxSimVIP> cd cfg_etc
cfg_etc> ls
info.cfg
cfg_etc>
```

## Next Steps

The subproject is removed from the VIP by removing the component:

```
myVIP> prj vip component remove INCLUDE_CFGFILE_COMMON
myVIP> prj vip build
```

When removed from the VIP, the subproject content is not deleted from the layer directory. The subproject can be added into another VIP.

Note: If a new version of a layer is installed and added to the VSB, a new layer version of the subproject will not be copied automatically into the VIP. The user must remove the subproject directory manually from the VIP (all user modifications will be lost at this point). Then the build system will automatically re-install the new layer version of the subproject when the VIP is built again.

Also after a subproject is removed, the next VIP build will clean the subproject directory by executing the clean_<*subproject*> target to remove any of the objects built.

# 15. PREPARING TO TEST A LAYER

A layer can only be tested once it is built into a VxWorks system and booted on a VxWorks target.

## Before You Begin

To follow the instructions in this section, you will need a target on which to test your layer. Depending on the hardware requirements of your layer, you may be able to use vxsim for some or all of your testing. These instructions assume vxsim will be used.

If you decide to use a different target then these instructions may be adapted for that scenario. However, you must ensure that you have:

- a target board with a CPU architecture supported by VxWorks.
- a VxWorks BSP for the target board.

## About This Task

In order to build a layer project and test that it functions as expected, we must add the layer to an appropriate VSB project. For a detailed description of VSB and VIP projects and how to use them, refer to the *VxWorks 7 Configuration and Build Guide*. For further detailed tutorials that describe the process of building and running layers, refer to *VxWorks 7 Inside Layers Tutorial* and *VxWorks 7 Migrating an RTP Project into a Layer Tutorial*.

> Note: It is strongly recommended that all the code you have added to your layer has been tested prior to its migration to the layer. The layer testing process should simply involve checking that what already worked will also function in the layer.

## Procedure

1. Setup a new wrtool session to manage your Workbench workspace.

```
$ cd installDir
$ wrenv -p vxworks-7
$ wrtool -data workspace
$ cd workspace
```

> Note: Our layer is positioned in *installDir* **/vxworks-7/pkgs_v2/custom/myLayer**, but for the purposes of building a VSB to test our layer, we will return to the Workbench workspace for the creation of our projects. Usually, this is located at *installDir* **/workspace**.

2. List the available BSPs.

```
workspace> prj vsb listBsps -all

Valid BSPs :
    vxsim_windows_1_0_2_7

workspace>
```

3. Create a VSB project.

```
workspace> prj vsb create -bsp vxsim_windows_1_0_2_7 myVSB
```

4. Verify that your layer appears in the VSB project configuration.

1. List all the layers visible to the VSB build system.

```
workspace> prj vsb listAll -f -latest
```

Note: It is useful to redirect wrtool system output to a text file when listing layer information. This is done from a normal shell prompt, not a wrtool command prompt.

```
$ wrtool -data workspace prj vsb listAll -f -latest > log.txt
```

2. List myLayer details.

```
workspace> cd myVSB
myVSB> prj vsb info MYLAYER

Name:                       MYLAYER_CUSTOM
VENDOR                      <company-name>
VERSION:                    1.0.0.0
FEATURE:                    NOT_DEFINED
TYPE:                       GENERIC
Target Name:                COMPANY_CUSTOM
Full Name:                  MYLAYER_CUSTOM_1_0_0_0

SYNOPSIS:                   This should be the synopsis for MYLAYER_CUSTOM

VSB_REQUIRES:               !MYLAYER_CUSTOM_DISABLED
LAYER_REQUIRES:
SELECT:
FEATURE_PROVIDES:
FEATURE_REQUIRES:
OPTIONAL:                   YES
DEFAULT:                    YES

FASTBUILD:                  YES

Content:                    SOURCE
DOC_BUILD:                  NO
Location:                   installDir/vxworks-7/pkgs_v2/custom/myLayer
HELP:                       This should be the descriptive help for MYLAYER_CUSTOM


-----

myVSB>
```

5. Add the layer to your VSB project configuration.

   Notice that the name of your layer is listed in the previous step.

```
Name:                       MYLAYER_CUSTOM
```

   Add your layer to your VSB project. If the layer is already enabled in the VSB project, wrtool will warn you.

```
myVSB> prj vsb add MYLAYER_CUSTOM
Nothing to add all requested layers enabled
```



WNDRVR

```
done


Checking layers required by BSP vxsim_windows_1_0_2_7.
Nothing to add all requested layers enabled

done



This VSB supports the following BSPs: vxsim_windows_1_0_2_7

myVSB>
```

6. Build the VSB project containing your layer.

```
myVSB> prj build
```

Once the VSB project has built one time, it may be possible to speed up subsequent builds of your layer by restricting your build to only your layer and nothing else. Be aware that this approach can cause difficulty if your layer depends on other layers which also need to be rebuilt before it will build and run.

To restrict the build to just your layer:

```
my VSB> prj build myVSB -- __LAYERS_TO_BUILD=MYLAYER_CUSTOM
```

7. Deploy and test.

Refer to the *VxWorks 7 Configuration and Build Guide* for a description of how to build and boot a VIP project based upon your VSB project.

Once the target is booted, check to see whether or not your layer is functioning as expected. Check the log messages your layer generates, or run the layer test routines that you added to your layer.

# 16. DIFFERENCES AND BENEFITS BETWEEN VXWORKS 6.9 AND VXWORKS 7

The directory structure and build process of VxWorks 7 is quite different from VxWorks 6.9 and earlier releases.

## Building and Modifying VxWorks Kernel Source

Under VxWorks 6.9 it was possible from the host command line to navigate to *installDir* **/vxworks-6.9/target/src** and build the target libraries directly by running **make**. This approach was often recommended when patching the VxWorks 6.9 installation with changes to driver source or header files.

One drawback to the VxWorks 6.9 approach was that, once changes were made to files in the **src** directory structure, it was very difficult and time consuming to reverse those changes again. Often it was easier to simply delete the Wind River installation completely and install again from scratch.

This approach is no longer supported under VxWorks 7 and no source files will exist in *installDir* **/vxworks-7/target/src**. One can no longer simply build the VxWorks source tree from the host command line under VxWorks 7. Instead, a VxWorks source build (VSB) must be created in the workspace either using the command line or Workbench.

However, the productivity benefits of VxWorks 7 layers become clear when you consider that multiple parallel layer versions can co-exist in your Wind River installation at the same time. This means that we can modify the VxWorks kernel, for example, by downloading a new version of the kernel layers without modifying the original layer version source files at all. This is what the Wind River **Product Maintenance** tool does when you instruct it to download the latest VxWorks 7 updates. Following the update, you are left with different versions of layers; an original version and a new updated version. Consequently, you no longer face difficulty in rolling back the changes you have made to the VxWorks kernel because the original layers remain untouched in your installation. To rollback your changes, simply revert to the original layer version in the VSB Configuration tool.

## Kernel Source Customisation Workflow

Unlike VxWorks 6.9, the workflow for customising the source code of the kernel, or any of the components around it, is simple and straightforward for the user.

1. Take a copy of the original layer directory within *installDir* **/vxworks-7/pkgs_v2** and give it an updated name.
2. Modify the new layer's **layer.vsbl** meta-data to increment the layer's version number.
3. Update the layer source and header files as appropriate.
4. Select the new layer version in the VSB Source Build Configurator.
5. Rebuild both VSB and VIP projects and test.

## Kernel Source Modification Rollback Workflow

Rolling back your changes is also much simpler in VxWorks 7.

1. Open the Workbench Source Build Configurator and select the original layer version.
2. Rebuild both VSB and VIP projects and test.

## Board Support Packages

VxWorks 6.9 shipped with BSPs installed in *installDir* **/WindRiver/vxworks-6.9/target/config** directory, although third party BSPs could be located anywhere on the host filesystem. There was not a single, reliable and recommended way for distributing and installing VxWorks 6.9 BSPs.

BSPs under VxWorks 7 are packaged as layers and found at *installDir* **/vxworks-7/pkgs_v2/os/board**. This means that BSPs may be packaged within RPMs and so installed by the Wind River installer.

BSPs also have a simpler structure than the VxWorks 6.9 equivalents.

# 17. PACKAGE MANAGEMENT OVERVIEW

RPM packages are a portable, multi-platform format for distributing complex, structured file sets such as project source files or finished software products to developers or end users.

RPM-based installations are easy to maintain over time, and ensure the integrity of common management tasks such as updates and package removal. Visit rpm.org for more information on RPMs.

You can use the Wind River Workbench export wizard to create an RPM package and or RPM **.spec** file, or you can create an RPM from the command line based on an existing **.spec** file. A spec file contains the instructions for creating a given RPM package.

**Creating an RPM Project with Workbench on page 68**To create an RPM, start a new VxWorks RPM project, select the project files to include, provide details required for the **.spec** file, and build the RPM.
**Exporting an RPM Package with Workbench on page 72**You can use the export wizard in Workbench to export resources, for example, projects and files, as an RPM package.
**Exporting an RPM Package from an RPM SPEC File on page 74**You can export resources such as projects and files from Workbench as an RPM. The Export wizard enables you to create an RPM using an RPM SPEC file.
**Creating a New RPM Package from the Command Line on page 76**You can create an RPM package from the command line using the wrrpm utility.

## 1. Creating an RPM Project with Workbench

To create an RPM, start a new VxWorks RPM project, select the project files to include, provide details required for the **.spec** file, and build the RPM.

### Before You Begin

You should be familiar with RPM. You should already have a VxWorks project, e.g., VxWorks Layer Project on page 2, which you want to export as an RPM package.

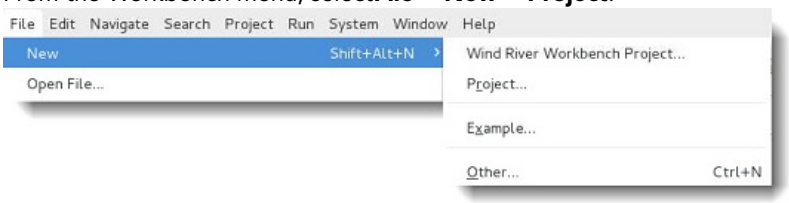
**CAUTION:** **You should never create your RPM as root user on a Linux host. You must always build as a normal user.**

### About This Task

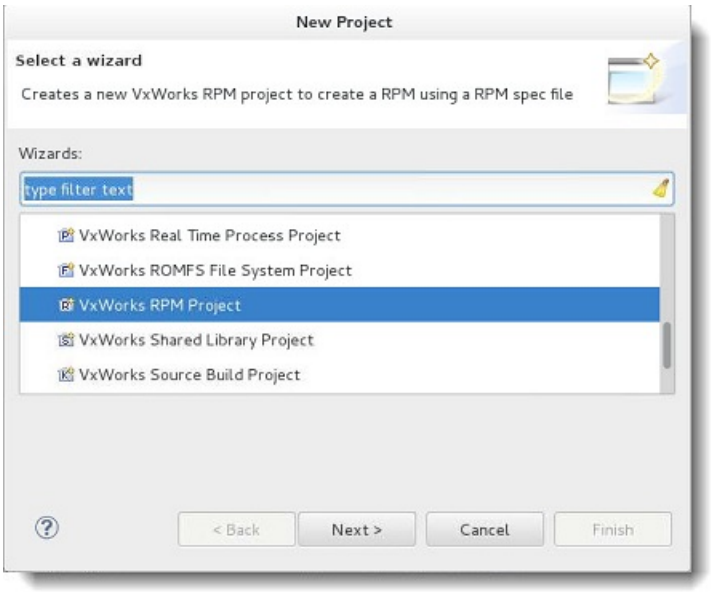
Perform the following steps to create a new VxWorks RPM project to generate an RPM using an RPM spec file.

### Procedure

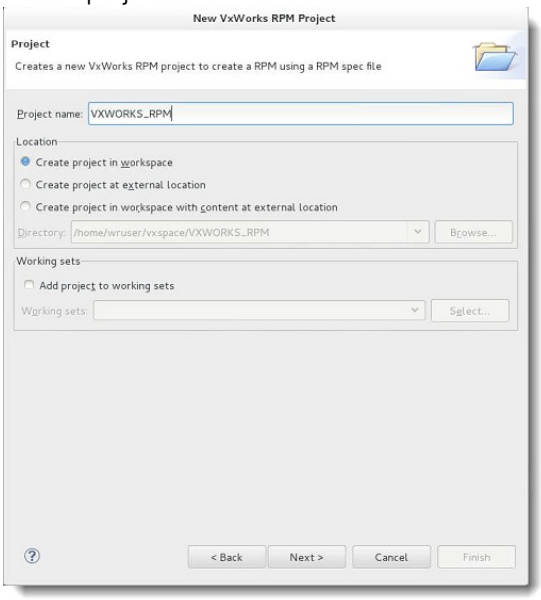
1. From the Workbench menu, select**File > New > Project**.



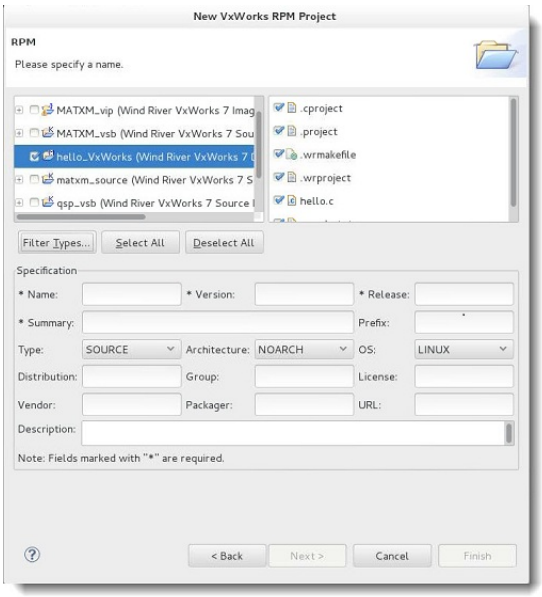

2. Select **VxWorks RPM Project** from a VxWorks folder and click **Next**.



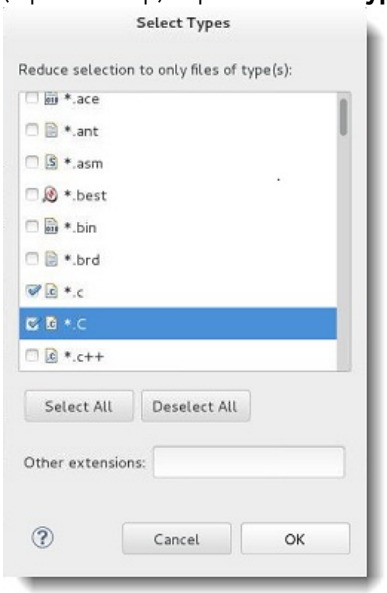

3. Enter a project name.




4. Choose the location where your project will be created.
5. (Optional step). If you want to add the project to working sets, select the **Add project to working sets** option.
6. Click **Next**.
7. Select a development project to add to the RPM.




8. (Optional step). Open the **Filter Types...** tab to select the file types to include in the RPM package.




9. Provide RPM specification information and click **Next**.

   This information is used to generate the **.spec** file, where each field corresponds to a **Tag**. Provide In the Specification area, provide the information for the RPM being created. Fields marked with an asterisk (*) are required.

   Name

   The (base) name of the package, the name of the software being packaged.

   Version

   The version of the software being packaged.

   Release

   The version of the generated package.

Summary

A brief description of the packaged software.

Prefix

Used when relocatable package is to be built. The data specified to the **prefix** tag should be the part of the package's path during installation.

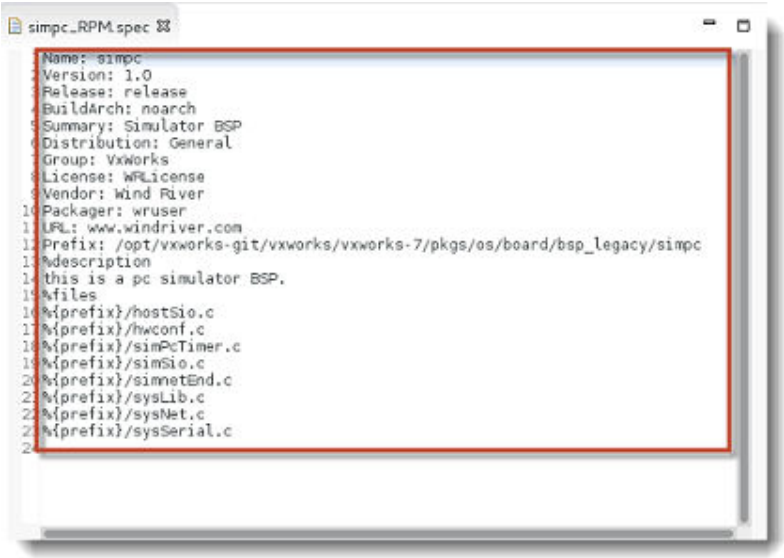
10. (Optional step). In the Project Structure dialog, select the referenced subproject to your project and click **Finish**.

    In Workbench, the Project Explorer view displays the newly created RPM project. You can now build the project, which results in an **.rpm** file. By default, the RPM project contains the **.spec** and **.rpm** files, and a Makefile providing build support.

11. Verify the contents of the **.spec** file.

    The RPM and the **.spec** file are available from the Project Explorer. Open the **.spec** file

    The information and files you added in the previous steps are included.




    You can directly edit the content in this file, for example, by updating the version as your development proceeds and you want to create an RPM of the new version without overwriting the original. Then you can use the **.spec** file to generate an RPM without the need to repeat all of the previous steps. Refer to Exporting an RPM Package from an RPM SPEC File on page 74 for details.

12. (Optional step) Change the value of one of the tags in the **.spec** file in the Editor. For example, change the version value to "1.1".
13. (Optional step) Build the project.

    In Project Explorer, right-click the RPM project and select **Build Project**.

    When the build finishes successfully you will see a new RPM package with the updated version in the project tree.

**Parent topic:** Package Management Overview on page 68

**Related information**

Exporting an RPM Package with Workbench on page 72
Exporting an RPM Package from an RPM SPEC File on page 74

Creating a New RPM Package from the Command Line on page 76

# 2. Exporting an RPM Package with Workbench

You can use the export wizard in Workbench to export resources, for example, projects and files, as an RPM package.

## Before You Begin

Before exporting an RPM, you must have created a VxWorks project, such as a VxWorks Layer Project on page 2, that you want to export as an RPM package.

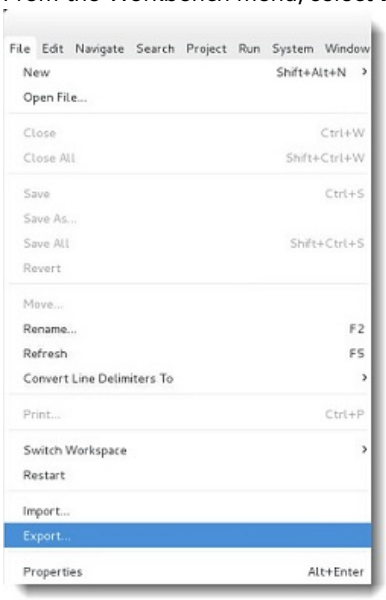
**CAUTION:** **If you create RPMs on a Linux host, you must always build as a normal user. Never create your RPM as root user.**

## About This Task

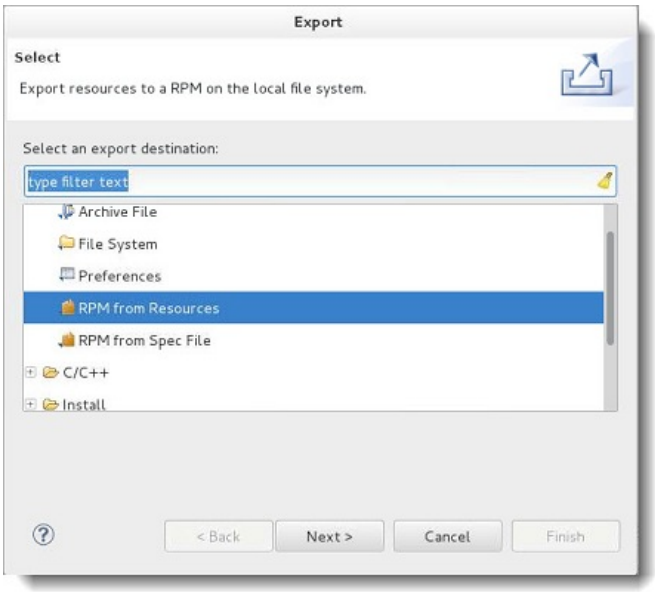
You can use the export tool in Workbench to export resources, such as VxWorks projects or files systems, to an RPM package.

## Procedure

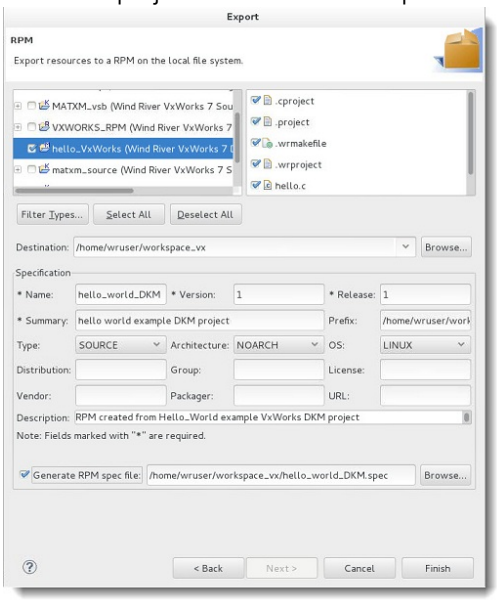
1. From the Workbench menu, select **File > Export...**.




2. Select **General > RPM from Resources** in the Export dialog and click **Next**.

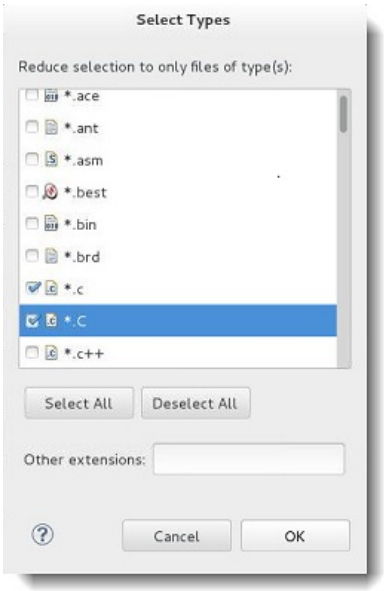



3. Select the project as a resource to export.




4. (Optional step). Apply a filetype filter.

   Open the **Filter Types...** tab to select the file types to include in the RPM package. Those files will be included in the RPM package for installation later when your RPM package is installed.




5. Enter the path to the location where the RPM should be exported in the **Destination** field.
6. In the Specification tab, provide the information for the RPM being created. Fields marked with an asterisk (*) are required. The information entered is used to create the **.spec** file. Click **Next**.
7. (Optional step). Select the check box **Generate RPM spec file** to have all the information entered in step 6 on page 74 stored in the **.spec** file.

   You can directly edit the content in this file, for example, by updating the version as your development proceeds and you want to create an RPM of the new version without overwriting the original. Then you can generate an RPM by using the **.spec** file without the need to repeat all of the previous steps. Refer to Exporting an RPM Package from an RPM SPEC File on page 74 for details.

8. Click **Finish**.

**Parent topic:** Package Management Overview on page 68

**Related information**

Creating an RPM Project with Workbench on page 68
Exporting an RPM Package from an RPM SPEC File on page 74
Creating a New RPM Package from the Command Line on page 76

# 3. Exporting an RPM Package from an RPM SPEC File

You can export resources such as projects and files from Workbench as an RPM. The Export wizard enables you to create an RPM using an RPM SPEC file.

## Before You Begin

Before beginning these steps you must already have a VxWorks project, such as a VxWorks Layer Project on page 2, that you want to export as an RPM package.

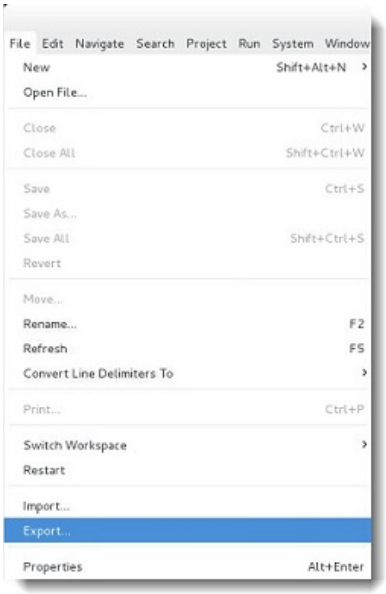
| | |
|---|---|
| **CAUTION:** | **If you create RPMs on a Linux host, you must always build as a normal user. Never create your RPM as root user.** |

## About This Task

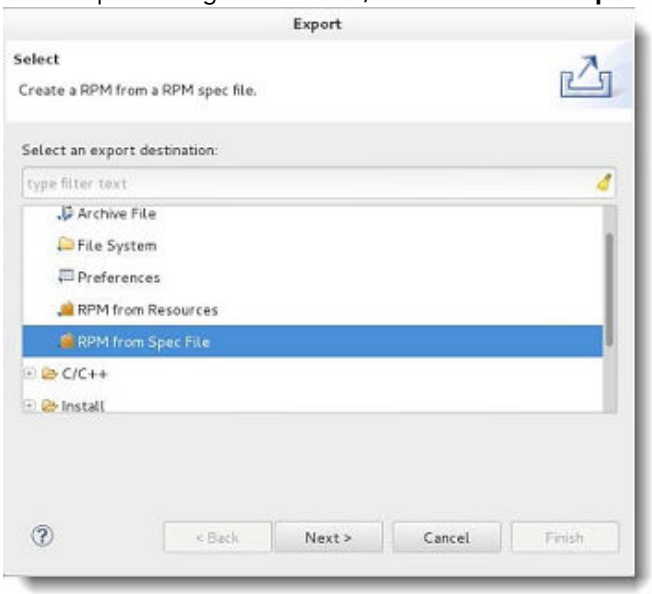
You can use the export tool in Workbench to create a new RPM package using an RPM spec file. The RPM spec file can be created with the RPM Wizard in Workbench (see Creating an RPM Project with Workbench on page 68), with the RPM Export Wizard in Workbench (see Exporting an RPM Package with Workbench on page 72), or from other resources.

## Procedure

1. From the Workbench menu, select **File > Export...**.




2. In the Export dialog's **Select** field, choose **RPM from Spec File** and click **Next**.




3. Give the export file a name.

In the Export dialog, enter the name of the RPM spec file or click the **Browse** button to select the path to the file.

4. Enter the path to the build root directory or click the **Browse** button to select the path.
5. Enter the path to the destination directory or click the **Browse** button to select the path.

   The RPM will be generated and stored in that directory.

6. Click **Finish**.

**Parent topic:** Package Management Overview on page 68

**Related information**

Creating an RPM Project with Workbench on page 68
Exporting an RPM Package with Workbench on page 72
Creating a New RPM Package from the Command Line on page 76

# 4. Creating a New RPM Package from the Command Line

You can create an RPM package from the command line using the wrrpm utility.

## Before You Begin

You should already have a VxWorks project, for example a VxWorks Layer Project on page 2, that you want to export as an RPM package.

| | |
|---|---|
| **CAUTION:** | **If you create RPMs on a Linux host, you must always build as a normal user. Never create your RPM as root user.** |

## About This Task

The wrrpm tool in VxWorks takes a RPM spec file and destination location, and optionally build root location, in order to create an RPM. It can be used manually from within any development shell, allowing you to integrate the wrrpm tool into your build system. It is also used to build RPM projects (the Makefile provides the corresponding build rules and is set into context with corresponding information from the RPM project build properties). See Creating an RPM Project with Workbench on page 68 for more details.

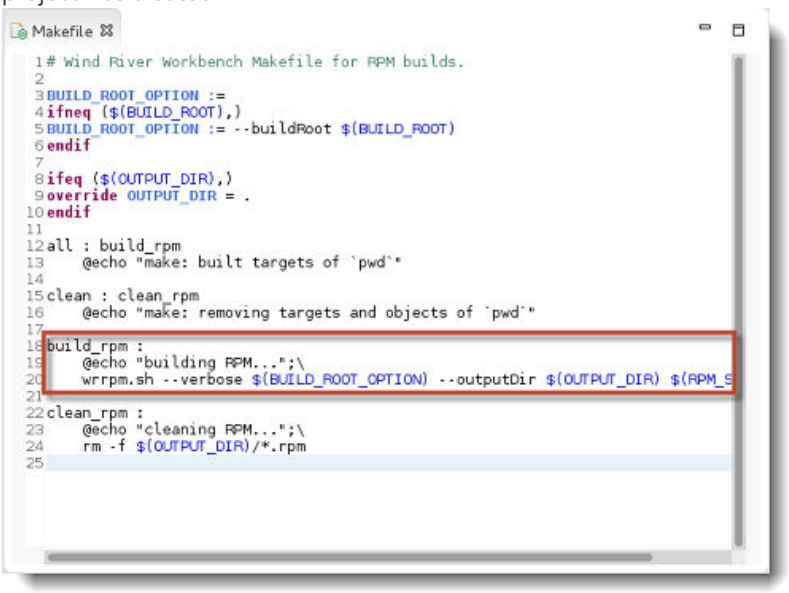
## Procedure

1. In Project Explorer, browse to the RPM project you created followed the steps in Creating an RPM Project with Workbench on page 68.
2. Open the project's Makefile in the Editor.



WNDRVR

Note: The **build_rpm** rule specifies arguments passed to the wrrpm tool. These arguments were defined when the RPM project was created.




3. Open a shell.

   You have three options.

   - From Project Explorer, Right-click on an RPM project and select **Open Wind River VxWorks 7 Development Shell**.
   - From the installation root directory in a native command line, enter the following.

     ```
     $ wrenv -p vxworks-7
     ```

     provided shortcut on the desktop or start men
   - Click the desktop shortcut or **Start** menu option.

   For more information, refer to the *VxWorks User's Guide* to set up the environment on your system

   Once within the shell, you can use **wrrpm** without the need to locate it; it will be available in the **PATH** environment.

4. Execute the command wrrpm --help to print the usage of the command.

   Note: **wrrpm** is placed on your *PATH* environment and you can call it without entering its full path.

```
Usage: wrrpm [OPTION...] file

Options:
  General:
    --help, -h, -?                print this message
    --verbose, -v                 provide detailed output
  Creation:
    --buildRoot, -b [directory]   the build root directory for content generatio
n
    --outputDir, -o [directory]   the directory in which the RPM file will be cr
```

```
eated
  Query:
    --list, -l                    show content information about the specified R
PM file
File:                             the file to be used:
                                    - an RPM spec file for creation
                                    - an RPM file to query
```

5. Create an RPM.

```
wrrpm --verbose --buildRoot ${buildroot_option} --outputDir ${output_option} $(RPM_SPEC_FILE}
```

where:

- *${buildroot_option}* is the path to the **buildroot** directory for content generation
- *${outputDir}* is the path to where the RPM package will be created and stored
- *${RPM_SPEC_FILE}* is the path to the **.spec** file that will be used to generate the RPM package

6. Inspect the new RPM.

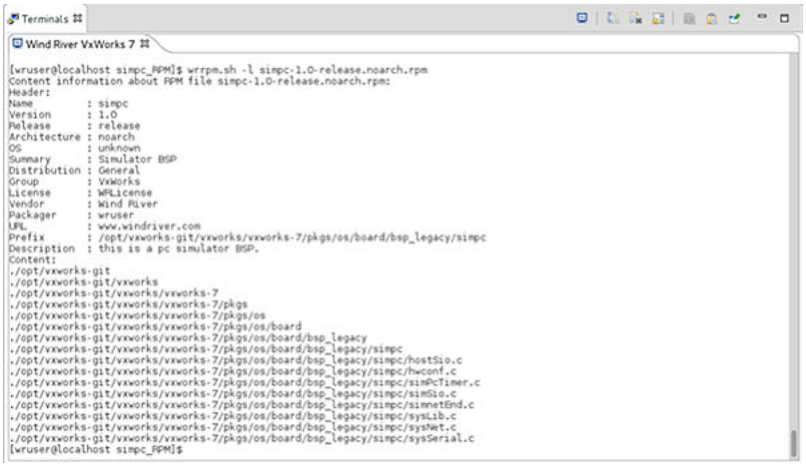
Inspection allows you to view the contents (files packed in the RPM) and meta information. The wrrpm tool provides a --list or -l option. Execute the following:

```
wrrpm --list ${RPM}
```

where *${RPM}* is the name and path to the RPM to be inspected.

If you want to review detailed information, add the --verbose or -v option:

```
wrrpm --list --verbose ${RPM}
```




**Parent topic:** Package Management Overview on page 68

**Related information**

Creating an RPM Project with Workbench on page 68
Exporting an RPM Package with Workbench on page 72

Exporting an RPM Package from an RPM SPEC File on page 74

# 18. LAYER-SPECIFIC VSB CONFIGURATION OPTIONS REFERENCE

This reference lists layer specific configuration options when creating or managing a VxWorks Source Build (VSB) project using wrtool command-line utility.

VxWorks provides the wrtool command-line utility to create, build, and manage VxWorks projects from the command-line interface (CLI). The syntax is wrtool prj vsb *subcommand option_flags* .

The wrtool utility is tightly coupled with Wind River Workbench. You can use Workbench and the wrtool utility interchangeably on the same projects. Any changes that are made through the wrtool utility will be seen in the Workbench GUI, and vice versa.

For example, to start using the wrtool command-line utility, one of the methods is to open the development shell and run the command:

```
$ wrtool -data <path to the workspace>
```

## add

Adds a layer and all layer dependencies to a VSB project.

Syntax: wrtool prj vsb add *layer*

Example:

```
wrtool> prj vsb add CAN
Now adding:
CAN
CAN_2_0_0_0

done
```

In the above example, the user asked to add the layer CAN, and the add process added the parent layer CAN_2_0_0_0.

## addNew

Adds the latest versions of enabled layers to a VSB project.

Syntax: wrtool prj vsb addNew

Example:

```
wrtool> prj vsb addNew
Now adding:
FS_1_0_0_0_NFS_1_0_0_3

done
```

Note: This functionality will not bring in any new layers to the VSB configuration, unless the new version of an enabled layer has a dependency on a new layer.

### list

Lists the layers in the VSB project.

Syntax: wrtool prj vsb list

```
wrtool> prj vsb list

Selected in this VSB:
ACPI_6_1_0_0
BDM_2_0_0_0
...
```

### listAll

Lists all layers in the installation directory.

Syntax: wrtool prj vsb listAll

Example:

```
wrtool> prj vsb listAll

Installed:
ACPI_6_1_0_0
ARM_2_0_0_0
AUDIO_2_0_0_0
BDM_2_0_0_0
CAN_2_0_0_0
CIVETWEB_1_9_1_10
...
```

### listFeatures

Lists all features and the layers that provide them

Syntax: wrtool prj vsb listFeatures

Example:

```
wrtool> prj vsb listFeatures
Features installed:
ADD_HW_BP
--- ARM_1_0_1_0_KERNEL_1_0_1_0_ARMV7A_1_0_1_0

ADD_IMX_ECSPI
--- FSL_IMX_1_1_1_0
...
...
...
XHCI_VXBUS
--- USB_1_1_0_0_CTLR_1_1_0_0_XHCI_1_0_1_1
```

### listNew

Lists all the newer versions of the layers that are in the VSB

Syntax: wrtool prj vsb listNew

Example:

```
wrtool> prj vsb listNew
Newer versions available for this VSB:
ESL_1_0_0_0
    ESL_1_0_10_0
```

### layerclean

Cleans a layer from a VSB target.

Syntax: wrtool prj vsb layerclean *layer*

Example:

```
wrtool> prj vsb layerclean USB_2_0_0_0
cd mk && make -f Makefile.prenobuild clean __LAYERS_TO_BUILD=USB_OTG VSB_DIR=/home/wruser/workspac
e/myVsbProj
make[1]: Entering directory '/home/wruser/workspace/myVsbProj/mk'
rm -f /home/wruser/workspace/myVsbProj/krnl/cdf/40usb_otg.cdf
rm -f /home/wruser/workspace/myVsbProj/krnl/configlette/usrUsbOtgInit.c
clean done
make[1]: Leaving directory '/home/wruser/workspace/myVsbProj/mk'
```

The above command is typically followed by the build command like wrtool prj vsb build USB_1_0_0_0_OTG_1_0_0_0

### remove

Removes a layer from the VSB project.

Syntax: wrtool prj vsb remove *layer*

Example:

```
wrtool> prj vsb remove CAN_2_0_0_0
Now removing:
CAN_2_0_0_0
CAN

done
```

# 19. LAYER MAKEFILE MACROS REFERENCE

A layer has a makefile in its top directory. When a VSB includes this layer, the makefile macros determine how the layer is built.

Three types of layer makefile macros exist:

Static Content Macro

Some macros in the makefile specify static content to be delivered to a VSB project like headers and component descriptor files. These static content files setup the VSB project build environment. This macro determines which source directory the content is located in. The content is copied to the VSB project build area during the build. An example of this type of macro is **PRE_BUILD_GENH_DIRS**.

Directory Specifications Macro

Some macros specify source directories that code is built in. The macro assigns the layer to the appropriate sub-stage of the VSB project build. The specified directories must contain a makefile in the appropriate format for the library or project source it contains. An example of this type of macro is **POSTBUILD_DKM_DIRS**.

Toolchain Specification Macro

Sometimes you may wish to override the default compiler settings of the VSB project. This is done using a Toolchain Specification macro that forces the layer compilation to use a different compiler at the selected build stage. An example of this type of macro is **PRE_BUILD_TOOLS**.

Layers are built as part of a VxWorks Source Build (VSB) project build that happens in stages. Each layer makefile macro relates to a different stage in this VSB build pipeline. A complete VSB build is comprised of three stages.

- Stage 1 - PREBUILD
- Stage 2 - BUILD
- Stage 3 - POSTBUILD

These build stages are subdivided into a 7 sub-stage pipeline and each stage is denoted by a macro. The VSB build pipeline starts at **PRE_BUILD_GENH_DIRS**, which begins the **PREBUILD** stage, and the pipeline ends with **POSTBUILD_RTP_DIRS**, which completes the **POSTBUILD** stage. All the macros listed below are optional, you only use what you need to describe the layer content.

Table 1. Layer Makefile Macro Overview - Build Stages

| Build Stage | Build Sub-Stage | Macro Name | Overview |
|---|---|---|---|
| **PREBUILD** | 1 | **PRE_BUILD_GENH_DIRS** | Directory Specification Macro<br>Assemble header files in these directories.<br>Recommended layer directory name is **genh**. |
| | 1 | **KERNEL_PUBLIC_H_DIRS** | Static Content Macro<br>Public header files needed during kernel space build are found in these directories.<br>Recommended layer directory name is **h**. |
| | 1 | **USER_PUBLIC_H_DIRS** | Static Content Macro<br>Public header files needed during user space build are found in these directories.<br>Recommended layer directory name is **usr_h**. |

| Build Stage | Build Sub-Stage | Macro Name | Overview |
|---|---|---|---|
| | 1 | **SHARED_PUBLIC_H_DIRS** | Static Content Macro<br>Public header files needed during both kernel and user space build are found in these directories.<br>Recommended layer directory name is **share_h**. |
| | 1 | **KERNEL_PUBLIC_H_FILES** | Static Content Macro<br>Specific public header files for kernel build are listed here. |
| | 1 | **USER_PUBLIC_H_FILES** | Static Content Macro<br>Specific public header files for user space build are listed here. |
| | 1 | **SHARED_PUBLIC_H_FILES** | Static Content Macro<br>Specific public header files for both kernel and user space build are listed here. |
| | 1 | **KERNEL_PROTECTED_H_DIRS** | Static Content Macro<br>Protected header files needed during kernel space build are found in these directories.<br>Recommended layer directory name is **prot_h**. |
| | 1 | **USER_PROTECTED_H_DIRS** | Static Content Macro<br>Protected header files needed during user space build are found in these directories.<br>Recommended layer directory name is **usr_prot_h**. |
| | 1 | **SHARED_PROTECTED_H_DIRS** | Static Content Macro<br>Protected header files needed during both kernel and user space build are found in these directories.<br>Recommended layer directory name is **share_prot_h**. |
| | 1 | **KERNEL_PROTECTED_H_FILES** | Static Content Macro<br>Specific protected header files for kernel build are listed here. |
| | 1 | **USER_PROTECTED_H_FILES** | Static Content Macro<br>Specific protected header files for user space build are listed here. |
| | 1 | **SHARED_PROTECTED_H_FILES** | Static Content Macro<br>Specific protected header files for both kernel and user space build are listed here. |
| | 1 | **KERNEL_SYSTEM_H_DIRS** | Static Content Macro<br>System header files needed during kernel space build are found in these directories. |

| Build Stage | Build Sub-Stage | Macro Name | Overview |
|---|---|---|---|
| | 1 | **KERNEL_SYSTEM_H_FILES** | Static Content Macro<br>Specific system header files for kernel space build are listed here. |
| | 1 | **POST_NOBUILD_CDFDIRS** | Static Content Macro<br>Directories containing cdf files to be used when building a VIP.<br>Recommended layer directory name is **cdf**. |
| | 2 | **PRE_BUILD** | Directory Specification Macro<br>Directories that contain the source of small, stand alone libraries.<br>Recommended layer directory name is **pre_src**. |
| **BUILD** | 3 | **BUILD_DIRS** | Directory Specification Macro<br>Directories containing code to be built into kernel libraries.<br>Recommended layer directory name is **src**. |
| | 4 | **BUILD_USER_DIRS** | Directory Specification Macro<br>Directories containing code to be built into user space libraries.<br>Recommended layer directory name is **usr_src**. |
| **POSTBUILD** | 5 | **POSTBUILD_USERLIB_DIRS** | Directory Specification Macro<br>Directories containing code to be built and linked with user space libraries. |
| | 5 | **POST_NOBUILD_CFGDIRS** | Static Content Macro<br>Directories containing configelette files to be used when building a VIP.<br>Recommended layer directory name is **configlette**. |
| | 6 | **POSTBUILD_DKM_DIRS** | Directory Specification Macro<br>Directories containing code to be built into DKM object files.<br>Recommended layer directory name is **dkm**. |
| | 7 | **POSTBUILD_RTP_DIRS** | Directory Specification Macro<br>Directories containing code to be built into RTP vxe files.<br>Recommended layer directory name is **rtp**. |

Table 2. Layer Makefile Macro Overview - Toolchain

| Macro Name | Build Sub-Stages | Description |
|---|---|---|
| **PRE_BUILD_TOOLS** | 1, 2 | Defines the tools used for content specified by the **PRE_BUILD_GENH_DIRS** and **PRE_BUILD_DIRS** macros. |
| **BUILD_TOOLS** | 3 | Defines the tools used for content specified by the **BUILD_DIRS** macro. |
| **BUILD_USER_TOOLS** | 4,5 | Defines the tools used for content specified by the **BUILD_USER_DIRS** and **POSTBUILD_USERLIB_DIRS** macros. |
| **DKM_TOOLS** | 6 | Defines the tools used for the content specified by the **POSTBUILD_DKM_DIRS** macro. |
| **RTP_TOOLS** | 7 | Defines the tools used for the content specified by the **POSTBUILD_RTP_DIRS** macro. |

The value assigned to the toolchain macro is one of the following:

- **$(_WRS_CONFIG_TOOL)** is the default setting and selects the compiler as chosen in the VSB configurator tool.
- **$(_WRS_CONFIG_APP_TOOL)** selects the compiler as chosen in the VSB configurator tool.
- **$(_WRS_CONFIG_ALL_TOOLS)** selects the compiler as chosen in the VSB configurator tool.
- **$(_WRS_CONFIG_gnu_VERSION)** selects the GNU GCC compiler only.
- **$(_WRS_CONFIG_llvm_VERSION)** selects the Clang (LLVM) compiler only.
- If no value is assigned; the default compiler as chosen in the VSB configurator tool is used.

If you intend to use the toolchain macros, you must ensure that the following makefile rule is included first at the top of your layer makefile:

**include $(WIND_KRNL_MK)/defs.layers.mk**

Refer to Layer Makefile Macros Reference on page 83 for a description of the available makefile rule files.

When deciding your choice of layer makefile macros, it is important to remember the sequential nature of VSB project build stages. Your layer is built as part of a wider VSB project build, and so you will need to decide upon the appropriate build stage(s) for your layer. To ensure that your layer builds properly, all dependant libraries must have been built by the time the VSB build system reaches your layer.

Most kernel layer code can be built using macro **BUILD_DIRS** at build sub-stage 3, and most user layers will be built using macro **BUILD_USER_DIRS** at build sub-stage 4. However, later build stages are available to you if you encounter VSB build errors and need to move your code there.

## Layer Makefile Macros Detailed Description

Table 3. Layer Makefile Directory Specification Macros - Kernel Space Description

| Build Sub-Stage | Makefile Macro | Required Directory Name | Description |
|---|---|---|---|
| 1 | **PRE_BUILD_GENH_DIRS** | **genh** | **Build directory that is primarily for header files although kernel source files are sometimes built here too.**<br>**The headers built or generated here can be either kernel or user header files.**<br>**The header files built or generated here must exist before the subsequent library builds start.**<br>**Specified directories contain headers and/or source files and a source makefile that includes** *$(WIND_KRNL_MK)***/rules.library.mk.** |
| 2 | **PRE_BUILD_DIRS** | **pre_src** | **Build directory for generated tools, source or headers that need to be created before other source is built. For example; yacc code generation.**<br>**Recommended for advanced customer layers.**<br>**Content varies depending on the file generated. Generated files should be placed in the VSB tree to keep layer content pristine and read only. This is typically done by build targets that are prefixed $(OBJ_DIR) or similar makefile macro.**<br>**Directories that contain a source makefile must ensure that it includes** *$(WIND_KRNL_MK)***/rules.library.mk.** |
| 3 | **BUILD_DIRS** | **src** | **The source-code directory for kernel libraries.**<br>**The source code is built into the VSB as libraries. The library names are derived from the macro LIB_BASE_NAME. LIB_BASE_NAME must be defined in a source makefile within the source directory of the kernel library.**<br>**Recommended for customer layers.**<br>**Specified directories contain source files and a source makefile that includes** *$(WIND_KRNL_MK)***/rules.library.mk.** |
| 6 | **POSTBUILD_DKM_DIRS** | **dkm** | **Specifies a source-code directory that compiles to a downloadable kernel modules (DKM).** |

| Build Sub-Stage | Makefile Macro | Required Directory Name | Description |
|---|---|---|---|
| | | | **Each directory contains the kernel source that will build to a partially linked object capable of being processed by the kernel runtime link-loader.**<br>**Specified directories contain source files and a source makefile that includes** *$(WIND_KRNL_MK)* **/rules.dkm.mk.** |

Table 4. Layer Makefile Directory Specification Macros - User Space Description

| Build Sub-Stage | Makefile Macro | Required Directory Name | Description |
|---|---|---|---|
| 1 | **PRE_BUILD_GENH_DIRS** | **genh** | **Build directory that is primarily for header files although kernel source files are sometimes built here too.**<br>**The headers built or generated here can be either kernel or user header files.**<br>**The header files built or generated here must exist before the subsequent library builds start.**<br>**Specified directories contain headers and/or source files and a source makefile that includes** *$(WIND_KRNL_MK)* **/rules.library.mk.** |
| 2 | **PRE_BUILD_DIRS** | **pre_src** | **Build directory for generated tools, source or headers that need to be created before other source is built. For example; yacc code generation.**<br>**Recommended for advanced customer layers.**<br>**Content varies depending on the file generated. Generated files should be placed in the VSB tree to keep layer content pristine and read only. This is typically done by build targets that are prefixed $(OBJ_DIR) or similar makefile macro.**<br>**Directories that contain a source makefile must ensure that it includes** *$(WIND_KRNL_MK)* **/rules.library.mk.** |
| 4 | **BUILD_USER_DIRS** | **usr_src** | **Contains the source-code directory for user libraries or applications that will be linked to RTPs.**<br>**The source code is built into the VSB as libraries. The library names are derived from the macro LIB_BASE_NAME. LIB_BASE_NAME must be defined in a source makefile within the source directory of the user library.** |

| Build Sub-Stage | Makefile Macro | Required Directory Name | Description |
|---|---|---|---|
| | | | **This macro is usually used for a code base which is linked statically to an RTP at compile time.**<br>**It may also be used to build an RTP shared library. However, ensure that any additional libraries your code depends on have already been built. For example, low level C compiler library archive libc_internal.a is built at this build stage. If your layer depends on the availability of this archive, the VSB project build will fail if the archive has not yet been built.**<br>**If you encounter VSB build errors using this macro, relocate your code to either POSTBUILD_USRLIB_DIRS or POSTBUILD_RTP_DIRS and rebuild the VSB project.**<br>**The specified directories contain source and a source makefile that includes** *$(WIND_USR_MK)***/rules.library.mk.** |
| 5 | **POSTBUILD_USRLIB_DIRS** | **usr_lib** | **Contains the source-code directory for user libraries or applications that have a dependency on BUILD_USER_DIRS libraries.**<br>**This macro is recommended for use with custom user space layers that fail to build at BUILD_USER_TOOLS stage due to dependency problems on other user space layers that have not yet been built.**<br>**The specified directories contain source and a source makefile that includes** *$(WIND_USR_MK)***/rules.library.mk.** |
| 7 | **POSTBUILD_RTP_DIRS** | **rtp** | **Specifies source-code directories for real-time processes (RTP).**<br>**May also be used to specify the source-code directory for an RTP shared library. An RTP shared library is essentially another type of RTP. RTP applications link to RTP shared libraries dynamically at run time.**<br>**Each directory contains the source for one RTP and will build to an RTP vxe file.**<br>**Specified directories contain source and a source makefile that includes** *$(WIND_USR_MK)***/rules.rtp.mk.** |

Table 5. Layer Makefile Static Content Macros Description

| Build Sub-Stage | Makefile Macro | Recommended Directory Name | Description |
|---|---|---|---|
| 1 | **KERNEL_PUBLIC_H_DIRS** | **h** | **Public header file directories.**<br>**Contains public header files for kernel code.**<br>**Content is public and shared with all code during kernel space build.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**Directories do not contain a makefile.** |
| 1 | **USER_PUBLIC_H_DIRS** | **usr_h** | **Public header file directories.**<br>**Contains public header files for RTP and user space library code.**<br>**Content is public and shared with all user space code.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**Directories do not contain a makefile.** |
| 1 | **SHARED_PUBLIC_H_DIRS** | **share_h** | **Public header file directories.**<br>**Contains public header files for kernel, RTP, and user space library code.**<br>**Content is public and shared with all user space and kernel space code.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**Directories do not contain a makefile.** |
| 1 | **KERNEL_PUBLIC_H_FILES** | | **Lists individual public header files, not directories.**<br>**The header files specified are for kernel code.**<br>**Content is public and shared with all kernel code.**<br>**File names should be unique.**<br>**The file path for each header file starts from the top of the layer. For example: agent/src/tcflib/tcfLib.h agent/src/tcflib/debugAgent.h where the top of the layer is** *installDir* **/vxworks-7/pkgs_v2/os/debug_analysis/debug-2.0.0.0.**<br>**Recommended for exporting one or two header files used to describe the public interface of the layer.** |

| Build Sub-Stage | Makefile Macro | Recommended Directory Name | Description |
|---|---|---|---|
| 1 | **USER_PUBLIC_H_FILES** | | **Lists individual public header files, not directories.**<br>**The header files specified are for RTP and user space library code.**<br>**Content is public and shared with all user space code.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**The file path starts from the top of the layer. For example: coredump/userh where the top of the layer is** *installDir* **/ vxworks-7/pkgs_v2/os/debug_analysis/debug-2.0.0.0.**<br>**Recommended for exporting one or two header files used to describe the public interface of the layer.** |
| 1 | **SHARED_PUBLIC_H_FILES** | | **Lists individual public header files, not directories.**<br>**The header files specified are for kernel, RTP and user space library code.**<br>**Content is public and shared with all kernel and RTP and user space code.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**The file path starts from the root of the layer. For example: coredump/shareh where the root of the layer is** *installDir* **/ vxworks-7/pkgs_v2/os/debug_analysis/debug-2.0.0.0.**<br>**Recommended for exporting one or two header files used to describe the public interface of the layer.** |
| 1 | **KERNEL_PROTECTED_H_DIRS** | **prot_h** | **Protected header file directories.**<br>**Contains protected header files for kernel code.**<br>**Content is protected and shared with a select set of layers known as friends during kernel space build.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**Directories do not contain a makefile.** |
| 1 | **USER_PROTECTED_H_DIRS** | **usr_prot_h** | **Protected header file directories.**<br>**Contains protected header files for RTP and user space library code.**<br>**Content is protected and shared with a select set of layers known as friends during user space build.**<br>**File names should be unique.** |

| Build Sub-Stage | Makefile Macro | Recommended Directory Name | Description |
|---|---|---|---|
| | | | **Recommended for customer layers.**<br>**Directories do not contain a makefile.** |
| 1 | **SHARED_PROTECTED_H_DIRS** | **share_prot_h** | **Protected header file directories.**<br>**Contains protected header files for kernel, RTP, and user space library code.**<br>**Content is protected and shared with a select set of layers known as friends during both kernel and user space build.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**Directories do not contain a makefile.** |
| 1 | **KERNEL_PROTECTED_H_FILES** | | **Lists individual protected header files, not directories.**<br>**The header files specified are for kernel code.**<br>**Content is protected and shared with a select set of layers known as friends in kernel space.**<br>**File names should be unique.**<br>**The file path for each header file starts from the top of the layer. For example: agent/src/tcflib/tcfLib.h agent/src/tcflib/debugAgent.h where the top of the layer is** *installDir* **/vxworks-7/pkgs_v2/os/debug_analysis/debug-2.0.0.0.**<br>**Recommended for exporting one or two header files used to describe the protected interface of the layer.** |
| 1 | **USER_PROTECTED_H_FILES** | | **Lists individual protected header files, not directories.**<br>**The header files specified are for RTP and user space library code.**<br>**Content is protected and shared with a select set of layers known as friends in user space.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**The file path starts from the top of the layer. For example: coredump/userh where the top of the layer is** *installDir* **/vxworks-7/pkgs_v2/os/debug_analysis/debug-2.0.0.0.**<br>**Recommended for exporting one or two header files used to describe the protected interface of the layer.** |
| 1 | **SHARED_PROTECTED_H_FILES** | | **Lists individual protected header files, not directories.**<br>**The header files specified are for kernel, RTP and user space library code.** |



WNDRVR

| Build Sub-Stage | Makefile Macro | Recommended Directory Name | Description |
|---|---|---|---|
| | | | **Content is protected and shared with a select set of layers known as friends in kernel, RTP and user space.**<br>**File names should be unique.**<br>**Recommended for customer layers.**<br>**The file path starts from the root of the layer. For example: coredump/shareh where the root of the layer is** *installDir* **/ vxworks-7/pkgs_v2/os/debug_analysis/debug-2.0.0.0.**<br>**Recommended for exporting one or two header files used to describe the protected interface of the layer.** |
| 1 | **KERNEL_SYSTEM_H_DIRS** | | **Directory containing header files for kernel space.**<br>**Supported when GNU GCC compiler is being used.**<br>**The GNU GCC compiler suppresses any warnings on the content of these header files.**<br>**These header files are copied into the VSB build last, after all the other headers have been copied. This uses the compiler's -isystem command-line option.**<br>**If -isystem is unsupported on the toolchain used to compile the layer, the VSB build system will default these header file directories to behave like KERNEL_PUBLIC_H_DIRS.** |
| 1 | **KERNEL_SYSTEM_H_FILES** | | **Lists individual header files, not directories.**<br>**Supported when GNU GCC compiler is being used.**<br>**The GNU GCC compiler suppresses any warnings on the content of the header files listed.**<br>**These header files are copied into the VSB build last, after all the other headers have been copied. This uses the compiler's -isystem command-line option.**<br>**If -isystem is unsupported on the toolchain used to compile the layer, the VSB build system will default these header files to behave like KERNEL_PUBLIC_H_FILES.** |
| 1 | **POST_NOBUILD_CDFDIRS** | **cdf** | **Directory containing configuration descriptor files (cdf).**<br>**These text files use the cdf file extension. They declare components related to this layer that will appear as options in the configuration of a VxWorks Image Project (VIP) when it is built from a VSB containing this layer.**<br>**These files will be copied from the layer into the VSB after the build is complete.** |

| Build Sub-Stage | Makefile Macro | Recommended Directory Name | Description |
|---|---|---|---|
| 5 | **POST_NOBUILD_CFGDIRS** | **cfg** | **Directory containing configlette source files that will be compiled in the VIP based on cdf selection.**<br>**A configlette is a small C source file that is not compiled by the VSB project build. Instead, it is compiled as part of the VIP when the cdf component that references it is included in the VIP kernel configuration. It is typically used for complex component initialization.**<br>**These files will be copied from the layer into the VSB after compilation.** |

For a tutorial that shows how layer makefile macros are used in layer creation, refer to *VxWorks 7 Migrating an RTP Project into a Layer Tutorial*.

## Header File Namespace

All header files referenced by the Statc Content Macros must have unique file names. They will each be copied from the layer header file directories into a VSB project header directory at the start of the VSB project build. It is important then that the layer header files all have unique names across the complete VSB project namespace.

If header files exist in your layer that have the same name as another header file that appears elsewhere in the VSB, then you will probably experience strange and unexpected build problems as you build your VSB project.

For example, assume your layer contains a heavily modified header file called **vxworks.h**. Build problems may arise because the original system **vxworks.h** header file will be overwritten by your modified version at that start of the VSB build containing your layer. All layers that are built as part of the VSB project will use your **vxworks.h** header file instead of the original one.

If you must create a duplicate header file for use in your layer, then the recommended procedure is as follows:

1. Create a uniquely named sub-directory in your layer header directory and locate the customised header file there.

   Note: For example, in layer directory *installDir* **/vxworks-7/pkgs_v2/myLayer/h**

   ```
   $tree -L 2
   .
   ├── unique_header_name1.h
   ├── unique_header_name2.h
   ├── layer_XXX_duplicate_headers
       ├── vxworks.h
       ├──
       ├──
       ├──
   ```

2. Point the Static Content Macro at the layer header file directory as normal.
   Because the layer header directory **h** contains header files and a uniquely named sub-directory containing your customised header file, the VSB project build system will copy this sub-directory into the VSB project header directory along with the header files in **h**. The original header file will be preserved, and the customised header file will also be available within the VSB project filesystem and namespace.

For example, the layer makefile would contain this line:

```
KERNEL_PUBLIC_H_DIRS = h
```

3. All layer source files that include your customised header file must include the full path.

   For example, a layer source file would include this line:

```
include "layer_XXX_duplicate_headers/vxworks.h"
```

# 20. LAYER MAKEFILE RULES REFERENCE

Wind River supplies makefile rules that control the stages of the VSB project build and so these rules must be included in layer and source makefiles.

VxWorks layer directories contain two types of makefile:

- A layer makefile.
- One or more source makefiles.

For more details on the differences between these types of makefile, refer to Layer Makefile on page 17. It is important that layer makefiles only include makefile rules intended for that type of makefile. The same applies to source makefile rules. If the incorrect makefile rules are included, then the results during the VSB project build are undefined and unpredictable.

## Detailed Description

Table 1. Makefile Build Rules for Layers

| Rule Path | Layer or Source Makefile Rule | Rule Description |
|---|---|---|
| $(WIND_KRNL_MK)/rules.layers.mk | Layer | Must be included at the bottom of every root layer makefile. |
| $(WIND_KRNL_MK)/defs.layers.mk | Layer | **When included, gives the layer makefile visibility of the VSB project configuration. Specifically, it is granted access to the _WRS_CONFIG_XXX macros (e.g. _WRS_CONFIG_CERT) build system macros.**<br>**Must be included at the top of every layer makefile that uses the Toolchain Macro to change the compiler choice for the layer.** |
| $(WIND_KRNL_MK)/rules.library.mk | Source | **Important for customer developed layers.**<br>**Must be included in every source makefile that builds for a kernel space library.** |
| $(WIND_KRNL_MK)/defs.library.mk | Source | **Important for customer developed layers.**<br>**Must appear in a source makefile that is building a library in kernel space.**<br>**Will usually appear in the makefile along with the macro LIB_BASE_NAME.**<br>**Is found in layers that submit source code directories to be built at stage 3 of the build pipeline (BUILD_DIRS).** |

| Rule Path | Layer or Source Makefile Rule | Rule Description |
|---|---|---|
| $(WIND_KRNL_MK)/rules.binlayer.mk | Source | Included at the bottom of a binary layer root makefile. Usually binary layer makefile's are auto-generated. |
| $(WIND_KRNL_MK)/rules.dkm.mk | Source | Must be included at the bottom of a layer makefile that is building source code at stage 6 of the build pipeline (**POSTBUILD_DKM_DIRS**). |
| $(WIND_USR_MK)/rules.library.mk | Source | **Important for customer developed layers.**<br>**Must be included in every source directory makefile that builds for a user space library (BUILD_USER_DIRS or POSTBUILD_USERLIB_DIRS).** |
| $(WIND_USR_MK)/defs.library.mk | Source | **Important for customer developed layers.**<br>**Must appear in a source makefile that is building a library in user space.**<br>**Will usually appear in the makefile along with the macro LIB_BASE_NAME.**<br>**Is found in layers that submit source code directories to be built at stage 4 of the build pipeline (BUILD_USER_DIRS).** |
| $(WIND_USR_MK)/rules.rtp.mk | Source | Must be included at the bottom of a layer makefile that is building source code at stage 7 of the build pipeline (**POSTBUILD_RTP_DIRS**). |

# 21. LAYER METADATA ELEMENTS REFERENCE

This topic provides a reference on layer metadata elements.

Some of the metadata elements are required fields. The other metadata elements are optional and can be left blank.

The metadata is in the **layer.vsbl** file in the root directory of the layer.

Table 1. Metadata Elements

| Element | Descriptions | Examples |
|---|---|---|
| *LayerName*<br>Required | **Must be unique among layers in the same directory tree or container.**<br>**Sublayers in other container layers can have the same name.**<br>*LayerName* **is case-sensitive.**<br>**The recommended naming conventions are upper case characters and alphanumeric with "_" between terms.**<br>**Cannot contain dashs (" -") , hash marks (" #"), commas (" ,") , or periods (" . ")** | **Layer USB** |
| **SYNOPSIS**<br>Required | **This is the text field that will be used to describe the presented choice.**<br>**Should be less than 40 characters and descriptive.** | **SYNOPSIS USB system sk** |
| **HELP**<br>Required | **This text field is displayed in the VSB help for the layer.**<br>**All characters are permitted with the exception of single quotes.**<br>**The help field should be as descriptive as required; this is the online reference for the layer.** | **HELP This includes all USB selected automatically if a enabled.** |
| **VERSION**<br>Required | **Provides the layer's version information.**<br>**Must have 4 numerical digits, each separated by a period.** | **VERSION 1.0.0.0** |
| **VENDOR**<br>Required | **Provides manufacture information for the layer.**<br>**This is an alphanumeric field.**<br>**All layers provided by Wind River have the text Wind River.** | |
| **FEATURE**<br>Required | **Provides a category for the layer.**<br>**This is an alphanumeric field.** | |

| Element | Descriptions | Examples |
|---|---|---|
| **VSB_REQUIRES**<br>Optional | **Provides the VSB attributes required for the layer.**<br>**Supports multiple VSB attributes.**<br>**Supports boolean operators: ( ), \|\|, !, and &&** | **VSB_REQUIRES ARCH_pp**<br>**(FEATURE_IPNET_INET4_**<br>**FEATURE_IPNET_INET6) &**<br>**COMPONENT_IPCRYPTO_** |
| **LAYER_REQUIRES**<br>Optional | **Lists the layers that this layer depends on.**<br>**Supports conditional syntax as described in** Layer Metadata File on page 12**.** | **LAYER_REQUIRES CORE_**<br>**FDT VXBUS_BUSLIB** |
| **OPTIONAL**<br>Required | **Indicates whether the layer is optional.**<br>**Assign either YES or NO.**<br>**The default is NO if not specified** | **OPTIONAL YES**<br>**OPTIONAL NO** |
| **DEFAULT**<br>Required | **Indicates whether the layer should be automatically included in the VSB configuration.**<br>**The latest version of the same layer is chosen if there are multiple versions and the DEFAULT is set to YES.**<br>**The latest version is determined by processing VERSION in alphanumeric order.**<br>**Assign either YES or NO.**<br>**Default is NO if not specified.** | **DEFAULT YES**<br>**DEFAULT NO** |
| **PARENT**<br>Optional | **Assigns the layer's container layer.**<br>**Required only if the layer is not directly under the container layer's directory.**<br>**Default is blank.** | **PARENT IPNET_1_0_0_0_** |
| **FEATURE_REQUIRES**<br>Optional | **A feature is a specific API or set of APIs or functionality that a layer requires.**<br>**Layers can be made dependent on layer+feature pairs where the specified layer provides the specified feature. It will match the most recent versions of a layer that has the specific feature.** | **FEATURE_REQUIRES FEA**<br>**FEATURE_REQUIRES {LAY**<br>**{LAYER_X FEATURE_Y} **<br>**{LAYER_Z FEATURE_A}** |
| **FEATURE_PROVIDES**<br>Optional | **A feature is a specific API or set of APIs or functionality provided by a layer.**<br>**FEATURE_PROVIDES works in conjunction with FEATURE_REQUIRES. The FEATURE_PROVIDES data is used to indicate to other layers that these APIs exist in this particular version of a layer. It enables other layers to say it requires layer** *FOO* **with feature** *BAR* **by writing FEATURE_REQUIRES** *FOO* | **FEATURE_PROVIDES FEA**<br>**FEATURE_PROVIDES FEA** |

| Element | Descriptions | Examples |
|---|---|---|
| | *BAR*. **FEATURE_REQUIRES elements are used in place of explicitly stating a LAYER_REQUIRES** *FOO* **_1_0_0_0. If two or more versions of the layer** *FOO* **have a FEATURE_PROVIDES** *BAR* **element, then any of these** *FOO* **versions would be acceptable to a layer that has FEATURE_REQUIRES** *FOO BAR*.<br>**Feature names have the same restrictions as layer names and should be uppercase.** | |
| **FRIEND**<br>Optional | **Lists the layers that are friends to this layer.**<br>**The protected interfaces of this layer can only be exported to the select set of layers known as friends listed here.** | **Layer IKE** |
| **SELECT**<br>Optional | **Will automatically enable other layers when enabled in layer.vsbl.**<br>**Layer is always visible and can be enabled at any time when layer uses SELECT.**<br>**Similar to the metadata elements LAYER_REQUIRES or FEATURE_REQUIRES with one major distinction, the use of SELECT will not make this layer dependent on other layers, it will only enable the layer. When a layer uses either LAYER_REQUIRES or FEATURE_REQUIRES the layer will remain hidden until all dependent layers listed in the LAYER_REQUIRES and FEATURE_REQUIRES have been enabled.**<br>⚠ Warning: If **SELECT** is misused it can totally alter the fundamental nature of a VSB. **SELECT** is not a recommended element in layer metadata.<br>**Any layer using a SELECT must have the same restrictions as the layer SELECT is enabling. For example if a layer is only viable for ARCH_ppc then the layer doing the SELECT must also be only viable for ARCH_ppc.** | **SELECT {ILP32_DRV, F1, F**<br>**SELECT FO \ F3_SUB \ FE**<br>**{VXBUS_DRV, F4_ZZ, F5_** |
| **DOC_BUILD**<br>Optional | **Controls whether this layer appears in the Document generation process.**<br>**If a layer is internal and not to be published, then set DOC_BUILD NO.** | **DOC_BUILD YES**<br>**DOC_BUILD NO** |
| **FASTBUILD**<br>Optional | **Controls whether this layer supports parallel compilation during Workbench VSB Build.**<br>**If a layer has complex dependencies, it may be necessary to set to NO if build problems arise.** | **FASTBUILD YES**<br>**FASTBUILD NO** |



WNDRVR

| Element | Descriptions | Examples |
|---|---|---|
| **RETROFIT**<br>Optional | **Allows the current layer to modify the metadata for earlier versions of the same layer.**<br>**The modification can be specified to an individual or a range of previous layer versions. Wildcards are supported.**<br>**All payloads must be encapsulated in {} characters.**<br>Multiple RETROFIT payloads must be specified on a multiple lines by joining the payloads with the **/** character.<br>Note: **All the elements of a layer.vsbl can only occur once in the layer.vsbl. If an element is repeated, only the last one will be used.**<br>**The RETROFIT is only applied from the newest version of a layer, so 1.0.0.1 can modify 1.0.0.0 only until 1.0.0.2 is present at which point only the RETROFIT statement provided by 1.0.0.2 will be used.**<br>**When the layer specifying the RETROFIT is removed, the metadata modifications to the earlier layer versions are also removed.** | **RETROFIT {REPLACE, 1.0. FASTBUILD, NO}**<br>**On all of the layer version 1.2.1.9, set FASTBUILD to**<br>**RETROFIT {REPLACE, 1.0. FASTBUILD, NO}/ {ADD, 1 LAYER_REQUIRES, SYSTE**<br>**On all of the layer version 1.2.1.9, set FASTBUILD to layer dependency on the SYSTEMVIEWER_AGENT** |
| **SOFT_LAYER_REQUIRES**<br>Optional | **Provides a way to specify optional layers.**<br>**The main use of SOFT_LAYER_REQUIRES is in the BSP layer metadata bsp.vsbl files. Optional layers are added when the VSB is first created based on the BSP as a convenience. Unlike LAYER_REQUIRES, if an optional layer specified by SOFT_LAYER_REQUIRES is removed, the BSP will still be supported by the VSB.**<br>**Supports conditional syntax as described in** Layer Metadata File on page 12**.** | |
| **SOFT_FEATURE_REQUIRES**<br>Optional | **Provides a way to specify optional layers that have a specific feature. The main use of SOFT_FEATURE_REQUIRES is in the BSP layer metadata bsp.vsbl files. Optional layers are added when the VSB is first created based on the BSP as a convenience. Unlike LAYER_REQUIRES, if an optional layer specified by SOFT_LAYER_REQUIRES is removed, the BSP will still be supported by the VSB.**<br>**Syntax is SOFT_FEATURE_REQUIRES {***flag2***, {***uuu value***}, {***vvv value***}} {***www value***}**<br>**Where** *uuu***,** *vvv***, and** *www* **are layer names and** *value1***,** *value2***, and** *value3* **are FEATURE_PROVIDES from the preceding layer.**<br>**Supports conditional syntax as described in** Layer Metadata File on page 12**.** | |

For a tutorial showing the use of layer metadata elements, refer to *VxWorks 7 Building Custom Layers with Dependencies Tutorial*.

For a detailed outside layers tutorial, refer to *VxWorks 7 Outside Layers Tutorial*.



WNDRVR

# 22. NAMING AND VERSIONING OF PACKAGES AND LAYERS REFERENCE

This reference lists naming conventions to be used for packages, and versioning guidelines for layers and packages in VxWorks.

Layers and Packages have versions. This section describes some guidelines for naming packages and modifying version numbers of layers and packages. Any time layer code is modified, it triggers an event that requires that the layer and the associated package versions be incremented. The characteristics of the package are controlled with the **.spec** file. Layers are defined by the **.vsbl** file. Each package can contain one or more layers.

Package here refers to the RPM package that a customer receives.

## Filename Syntax for Packages

**<packagename><_optionalcputool|hostos>-<version><_optional fixid>-<distribution field><_datestamp>.noarch.rpm**

## Character Rules

- Use all lowercase characters.
- Do not use extended characters (ASCII a-z, 0-9).
- Separate words with underscores.
- Fields must be separated with a **-**.

## Package Name

The name of the package should describe the content of the package succinctly, so that a user can understand easily. It should not use company specific internal names.

Example:

- tipc
- drv_gei
- bsp_itl_unified

## CPU/TOOL

An optional field that describes the CPU/TOOL combination (if applicable) for binary content within the package. Binary distributions will always be released for the entire top level architecture family (like ARM, Intel, PPC, or RISC-V). If the processor family supports multiple variants, all variants will ship with the package.

For example:

- tipc_ppc_gnu
- tipc_x86_llvm

## Host OS

An optional field for host tools to describe the host operating system the packaged tools are for.

Valid values are:

- _windows
- _linux

## Version

Versions have the format **#.#.#.#**, where **#** is a number (or numbers). The leftmost (most significant) number is the major release version, the second number is the minor release version, the third number is the update release version, and the fourth (least significant) number is the maintenance or service release version.

By simple review of the field that changed a user should be able to understand the scope of the change from the previous revision.

- A version should only be incremented once per release.
  - A release is the event of code being delivered to a customer.
  - If a more significant change is made after an initial change, the version should be updated again to accurately represent the changes in a release.
- A version change of a layer requires the same level of increment in the package that contains the changed layer. For example, if you make a second digit change in a sublayer, the package that contains the sublayer must also have its second digit incremented.
- When you increment a first, second or third digit of a version, please zero out all lesser version numbers. (For example, if you have version 1.0.0.1 and you make a version 1.0.1.x, the final number should be zero, 1.0.1.0, and not 1.0.1.1. Likewise, a second digit move from 1.0.2.4 would yield 1.1.0.0)
- The RPM version should always match the top level layer version except for container layers, where the **layer.vsbl** version should never change.
- Unrelated and unmodified layer versions should not change.
- This policy does allow releasing version 1.2.3.5 after 1.2.4.0 has been released. Care should be taken as this will create multiple maintenance branches.

## Rules for Incrementing Digits in the Version

**Digit 4 or maintenance release version is incremented when:**

- An update fixes one or more defects but no new functionality is added.

**Digit 3 or update release version is incremented when:**

- An update adds new functionality without any dependancies on a new component.
- If the only practical implementation for a bug fix breaks binary compatibility (even though no new functionality is being introduced).

**Digit 2 or minor release version is incremented when:**

- An update adds functionality and becomes dependent on a new component or a pre-existing component.

**Digit 1 or major release version is incremented when:**

- An update adds a new feature which breaks source code backwards compatibility.

## Custom Layer Versioning

The last digit of the layer version must be extended by a new **.#**, where **#** is a number (or numbers).

Extending the version digit must always happen as a new suffix and not a prefix.

So for example once customized, a layer with a version 1.0.4 will become version 1.0.4.1.

> **Note:** There is no limit to the number of extensions that can be added.

If these custom layers are released to other customers, the product team making the modification can add their company abbreviation to the layer version. So for example once customized, a layer with a version 1.0.4 will become version 1.0.4.1.LTC (where LTC stands for Looney Tunes Corp).

### Fix ID

An optional field used for customer specific fixes (patches). This is used only to be distributed to individual customers, and never to an entire customer base. This is not expected to be a normal distribution mechanism, but is only for special customer specific fixes.

Example:

_WIND00112233

### Distribution Field

This field is used to represent the distribution of the VxWorks product this package is created for.

### Distribution Build Timestamp

This filed is used to support traceability of the multiple builds and internal releases of RPMs that will occur during development. This field will contain a datestamp for the RPM build and will make the name unique. This is added automatically during the RPM creation step for vx7 RPM in the following format **_YYYYMMDD**

Example:

_20130101 for January 1 2013

_20130522 for May 22 2013

> **Note:** This field is populated automatically by the packaging tools.

### Host Arch Field Value

This field is hardcoded to **noarch** to align with the standard RPM naming conventions for both usability and conformance with the existing RPM tools.

### File Extension

Always use **.rpm** for file extensions.

> **Note:** There will be no **.srpm** files.

### Example Package Names

- itl_generic-2.0.0.0-vx7_20180922.noarch.rpm
- drv-2.0.0.0-vx7_20180922.noarch.rpm



WNDRVR

- intrinsics_gnu-1.0.0.0-vx7_20180922.noarch.rpm
- llvm_vxworks_linux-6.0.0.2-vx7_20180906.noarch.rpm

### Third Party Package Naming Convention

Prepend the package name with the vendor name.

Example:

curtisswright_bsp_foo_1.0.0.0.rpm

datalight_flashfx_1.0.1.123.rpm

### Mapping of Package fields to RPM Specfile fields

| Package | RPM Specfile |
|---|---|
| Filename Field | Specfile Field |
| <packagename><_optionalcputool or hostos> | Name |
| <version><_optional fixid> | Version |
| <distribution field> | Release |
| noarch | BuildArch |